

Métodos Computacionais para Economia

**Faculdade de Economia, Administração, Contabilidade e Atuária Universidade de
São Paulo**

[Danilo Souza](#)

20 de janeiro, 2026

Índice

Sobre este material	3
Como usar	3
Histórico de atualizações	4
Agradecimentos e reprodução	4
Prefácio	5
I Alfabetização computacional e lógica básica	7
1 Fundamentos de computação	10
1.1 Introdução	10
1.1.1 Pré-requisitos	10
1.2 Como um computador funciona	10
1.2.1 Sistema computacional	12
1.2.2 Arquitetura de um computador	13
1.2.3 Bits e bytes	14
1.2.4 Processamento, memória e armazenamento na prática	15
1.3 Arquivos e diretórios	17
1.3.1 Estrutura hierárquica	17
1.3.2 Extensões de arquivos	18
1.4 Terminal	20
1.5 Linguagens de programação	22
1.5.1 Sintaxe e semântica	22
1.5.2 Diferenças entre linguagens naturais e formais	23
1.5.3 Alto nível X baixo nível	23
1.5.4 Linguagem Compilada X Interpretada	24
1.5.5 Paradigmas	24
1.5.6 Tipagem	25
1.5.7 Python	25
1.6 Exercícios	25
2 Primeiros passos no Python	26
2.1 O Python	26
2.1.1 Porque escolher	26

2.1.2	Como instalar	27
2.1.3	Anaconda e suas particularidades	30
2.1.4	IDEs	30
2.1.5	Spyder X Jupyter	33
2.2	Construindo o primeiro programa	34
2.2.1	Operadores aritméticos	34
2.2.2	Instruções de atribuição	36
2.2.3	Nomes de variáveis	36
2.2.4	Expressões e instruções	37
2.2.5	Pacotes de terceiros e funções não-nativas	38
2.3	Exercícios	39
3	Programação e IA	41
3.1	Introdução	41
3.1.1	Pré-requisitos	41
3.2	O que é Inteligência Artificial?	41
3.3	ChatGPT	43
3.3.1	Acesso à plataforma e primeiro uso	43
3.3.2	Conceitos básicos de engenharia de prompt	44
3.3.3	Alguns exemplos	44
3.3.4	Integração com ferramentas e APIs externas	44
3.4	StackOverflow	45
3.5	Exercícios	46
4	Tipos primitivos e objetos básicos	47
4.1	Introdução	47
4.1.1	Pré-requisitos	47
4.2	Strings	47
4.2.1	Operações básicas	48
4.2.2	Métodos de strings	50
4.2.3	Formatação e a instrução print	52
4.2.4	Introdução a expressões regulares	52
4.3	Listas	55
4.3.1	Operações básicas	56
4.3.2	Métodos de listas	57
4.3.3	Operadores lógicos e variáveis booleanas	59
4.3.4	Execução condicional	61
4.3.5	Execução alternativa	62
4.3.6	Condicionais encadeadas	62
4.4	Tuplas	63
4.4.1	Operações básicas	64
4.4.2	Atribuição de tuplas	65
4.4.3	Tuplas como valores de retorno	67

4.4.4	Operações nativas com tuplas	68
4.5	Dicionários	69
4.5.1	Operações básicas	71
4.5.2	Métodos de dicionários	73
4.6	Exercícios	74
5	Controle de fluxo e iteração	76
5.1	Introdução	76
5.1.1	Pré-requisitos	76
5.2	Repetição simples e a instrução <i>for</i>	76
5.2.1	<i>for</i> loop e instruções condicionais	79
5.2.2	Usando <i>enumerate</i> e <i>zip</i>	79
5.2.3	<i>For</i> e <i>list comprehensions</i>	82
5.3	Reatribuição e atualização de variáveis	83
5.4	Repetição condicional e a instrução <i>while</i>	84
5.4.1	Formas de Interromper um Loop	86
5.5	Aplicação: Teorema Central do Limite	88
5.6	Exercícios	92
6	Funções	93
6.1	Introdução	93
6.1.1	Pré-requisitos	93
6.2	O que é uma função?	93
6.3	Nossa primeira função	94
6.4	Argumentos de uma função	95
6.4.1	<i>Keyword arguments</i> e <i>default values</i>	95
6.4.2	Argumentos arbitrários	96
6.4.3	Elementos dentro da função: execução condicional	98
6.4.4	Aplicação: Teorema Central do Limite - Parte 1	98
6.4.5	Elementos dentro da função: iteração	99
6.4.6	Aplicação: Teorema Central do Limite - Parte 2	100
6.5	Valor de retorno de uma função	101
6.5.1	Expectativa de vida de variáveis dentro da função	102
6.5.2	Aplicação: Teorema Central do Limite - Parte 3	102
6.6	Documentação	105
6.7	Funções anônimas	111
6.8	Exercícios	112

II	Computação numérica, análise de dados e visualização	113
7	Arrays, matrizes e álgebra linear	115
7.1	Introdução	115
7.1.1	Pré-requisitos	115
7.2	O que é o NumPy?	115
7.3	Elementos básicos do NumPy	116
7.3.1	Arrays unidimensionais	116
7.3.2	Arrays multidimensionais	118
7.3.3	Propriedades de arrays	120
7.4	Operações básicas com arrays	121
7.4.1	Operações aritméticas simples	121
7.4.2	Funções universais	126
7.4.3	Arrays e listas	128
7.5	Aplicação: solução de sistema de equações lineares	129
7.5.1	Algoritmo de eliminação de Gauss-Jordan	129
7.5.2	Solução de sistemas exatamente identificados	133
7.6	Exercícios	135
8	Gestão e análise de dados	136
8.1	Introdução	136
8.1.1	Pré-requisitos	136
8.2	O que é o Pandas?	136
8.3	Objetos no Pandas	137
8.4	<i>DataFrames</i> : o coração do Pandas	139
8.4.1	Importação de dados externos	139
8.4.2	Características básicas	141
8.4.3	Indexação, seleção e filtragem	145
8.4.4	Operações com colunas numéricas	149
8.4.5	Operações com colunas contendo strings	152
8.4.6	Agrupamento e junção de <i>DataFrames</i>	155
8.4.7	Exportando o <i>DataFrame</i>	160
8.5	Eficiência e <i>dtypes</i>	161
8.6	Exercícios	171
9	Visualização	172
9.1	Introdução	172
9.1.1	Pré-requisitos	172
9.2	O que é o Matplotlib?	172
9.3	Construindo nosso 1º gráfico	173
9.3.1	Base de dados	173
9.3.2	Função <i>plot</i>	175
9.4	Personalização	178

9.5	Gráficos múltiplos	187
9.5.1	Várias séries em um mesmo <i>plot</i>	187
9.5.2	Vários <i>plots</i> dentro da mesma figura	191
9.6	Outros tipos de gráficos	195
9.6.1	Gráfico de dispersão: <i>scatter</i>	195
9.6.2	Gráfico de barra: <i>bar</i> e <i>barh</i>	197
9.6.3	Histograma: <i>hist</i>	201
9.7	Seaborn	204
9.8	Plotly	204
9.9	Visualização e <i>storytelling</i>	204
9.10	Exercícios	205
10	Análise empírica integrada	206
10.1	Introdução	206
10.1.1	Pré-requisitos	206
10.2	O que é o NumPy?	206
10.3	Exercícios	206
III	Temas complementares	207
11	Introdução à programação orientada a objetos (OOP)	209
11.1	Introdução	209
11.1.1	Pré-requisitos	209
11.2	Exercícios	209
12	Introdução ao R	210
12.1	Introdução	210
12.1.1	RStudio	210
12.1.2	Pacotes de terceiros	210
12.1.3	Pré-requisitos	210
12.2	Primeiros passos	211
12.2.1	Diferenças de sintaxe em relação ao Python	211
12.2.2	R como calculadora	211
12.3	Objetos	211
12.3.1	Vetores e matrizes	211
12.3.2	Dataframes	211
12.4	Iteração	211
12.5	Funções	211
12.6	Gestão e análise de dados	211
12.6.1	Manipulação e pipelines	211
12.6.2	Visualização	211
12.7	Exercícios	211

Sobre este material

Este website reúne o material didático da disciplina [EAE1106 — Métodos Computacionais para Economia](#), ministrada no Departamento de Economia da Universidade de São Paulo. Tal disciplina faz parte da grade de matérias obrigatórias e é oferecida no 3º semestre do Bacharelado em Ciências Econômicas.

A disciplina tem o objetivo de introduzir os estudantes ao raciocínio computacional, aos conceitos relevantes de computação, e às suas aplicações para resolver problemas relevantes no campo da Economia. Como linguagem principal para introdução à lógica de programação e análise de dados, o curso utilizará o **Python**, amplamente utilizado em diversas áreas das ciências e do mercado de trabalho. Espera-se que ao final do curso o estudante tenha, além do domínio sobre a sintaxe da linguagem utilizada no curso, capacidade de formular e resolver problemas desde a sua concepção até a forma de apresentação, numérica e visual, das análises.

O conteúdo aqui disponível foi desenvolvido para apoiar as aulas ao longo do semestre e servir como referência permanente para os estudantes. As notas combinam texto explicativo, exemplos de código executável e aplicações empíricas, com foco no desenvolvimento de habilidades práticas em programação e análise de dados. O material está organizado como um livro digital, estruturado em capítulos que acompanham a progressão da disciplina. Cada capítulo pode ser lido de forma sequencial ou consultado de maneira independente, conforme a necessidade do leitor.

Como usar

Este material foi desenvolvido para ser utilizado em conjunto com as aulas presenciais. Os capítulos seguem, em linhas gerais, a ordem do curso. Recomenda-se que o estudante leia o capítulo correspondente a cada aula, executando os exemplos de código em seu próprio computador ou outro ambiente de programação configurado conforme indicado ao longo do curso. Ler código sem executá-lo raramente é suficiente para desenvolver fluência em qualquer linguagem de programação; a compreensão dos conceitos e implementação prática depende da experimentação direta e contínua.

O código apresentado ao longo do texto é executável e foi concebido para ser modificado. Alterar o código, cometer erros e observar o comportamento do programa são etapas naturais e desejáveis no aprendizado de programação. Erros não indicam falha; eles fornecem informação sobre como o código funciona.

Os capítulos incluem exemplos, observações e exercícios. Esses elementos têm objetivos distintos:

- Exemplos ilustram conceitos apresentados no texto.
- Observações destacam detalhes importantes ou armadilhas comuns.
- Exercícios têm como objetivo consolidar o conteúdo do capítulo e preparar o estudante para aplicações mais complexas. **Não deixe de fazê-los.**

Este material não foi pensado para ser lido do início ao fim de forma linear, como um texto tradicional, e tampouco substitui as aulas. Ele pode ser consultado conforme a necessidade, servindo como referência contínua durante o curso e após seu término. O objetivo é que, ao final da disciplina, o estudante se sinta mais confortável em utilizar linguagens de programação como ferramentas para organizar dados, explorar informações e apoiar análises empíricas em economia.

Histórico de atualizações

Este website é atualizado continuamente, refletindo *feedbacks* a respeito do conteúdo e incorporação de novos pacotes e atualizações.

- **23 de Fevereiro, 2026:** Disponibilização da primeira versão do material, com a integridade dos capítulos 1-6.

Agradecimentos e reprodução

Esse material deve muito ao Prof. [Claudio Ribeiro de Lucinda](#). Parte importante do conteúdo foi desenvolvida de forma conjunta entre eu e o Prof. Claudio ao longo do segundo semestre de 2022, ano em que o curso foi oferecido pela primeira vez como uma matéria obrigatória de 30h no Bacharelado em Ciências Econômicas da FEA-USP. Agradecimento especial a Tomás Polido Garcia, que, enquanto monitor da matéria no primeiro semestre de 2026, ajudou muito na elaboração de exercícios e *feedbacks* em relação aos capítulos após a conversão do curso em uma matéria de 60h.

Este material é de propriedade intelectual do autor e não deve ser copiado ou reproduzido para fins comerciais. Não houve, até o momento, revisão técnica do material, de modo que o uso para fins acadêmicos é de inteira responsabilidade daquele que decidir por assim utilizar o material. Eventuais erros ou imprecisões são de responsabilidade exclusiva do autor do material.

Sugestões de ajuste e melhorias são muito bem-vindas, inclusive através de *issues* via GitHub. Caso queira, você pode me contactar em: danilosouza@usp.br

Danilo Souza Prof. Associado em Economia FEAUSP, Fevereiro 2026

Prefácio

Este material foi desenvolvido como apoio didático para a disciplina EAE1106 – Métodos Computacionais para Economia, ministrada pelo Departamento de Economia da Universidade de São Paulo. A disciplina tem por objetivo oferecer aos estudantes uma introdução sistemática à lógica de programação e às técnicas de análise de dados utilizando linguagens de programação, com ênfase em **Python** e uma breve introdução a **R** ao final do semestre.

O curso parte da motivação de que grande parte das perguntas contemporâneas em economia – desde a exploração de grandes bases de dados até a implementação de simulações e modelos numéricos – exige o uso de ferramentas computacionais para organizar, manipular, visualizar e interpretar dados de forma eficiente. Consequentemente, a capacidade de expressar uma sequência de operações de forma reproduzível e automatizada é uma competência essencial para economistas que desejam conduzir análises empíricas rigorosas e comunicar resultados de forma clara e estruturada.

Este material foi concebido com duas orientações principais:

- **Progressão pedagógica gradual.** O conteúdo inicia por fundamentos de lógica de programação, tipos de dados e objetos dentro da linguagem, controle de fluxo e funções, e evolui para tópicos aplicados de manipulação de dados, álgebra linear, visualização e análise integrada de dados. A estrutura dos capítulos foi planejada para acompanhar as etapas do raciocínio computacional e da análise empírica.
- **Aplicação orientada a problemas econômicos.** Sempre que possível, os exemplos e exercícios são contextualizados em questões empíricas de interesse econômico, tais como manipulação e análise de bases de dados administrativas, cálculo de estatísticas descritivas e visualização exploratória. Isso visa reforçar a conexão entre métodos computacionais e aplicações reais em pesquisa econômica.

Além dessas orientações gerais, o curso está organizado em três grandes blocos de conteúdo, que estruturam tanto as aulas quanto este material:

1. **Alfabetização computacional e lógica básica**, que introduz os conceitos fundamentais de computação, a interação entre linguagens de programação e modelos de inteligência artificial, tipos primitivos e objetos básicos dentro do Python, além de como implementar controle de fluxo, iteração, funções personalizadas dentro da linguagem.

2. **Computação numérica, análise de dados e visualização**, com foco em estruturas matriciais e álgebra linear através do uso do pacote NumPy, manipulação eficiente de dados em dataframes com Pandas, construção e personalização de gráficos, e princípios básicos de visualização de dados, culminando em uma análise empírica integrada que enfatiza organização de projetos, reprodutibilidade e construção de pipelines completos.
3. **Temas complementares**, que incluem uma pequena introdução a tópicos mais avançados, como a programação orientada a objetos, e uma breve introdução à linguagem R, com o objetivo de familiarizar o estudante com outro ecossistema amplamente utilizado em análise empírica e principal linguagem utilizada ao longo da sequência de cursos obrigatórios de Econometria.

De forma geral, o conteúdo foi organizado como um livro digital interativo, com exemplos de código e saídas esperadas, visualizações e anotações integradas. O uso de linguagens de programação neste formato permite que o leitor aprenda de forma construtiva e ativa, facilitando a compreensão da sintaxe, lógica e aprendendo a avaliar erros e como corrigí-los. Espero que este material seja útil tanto para estudantes que se aproximam pela primeira vez da programação aplicada à economia quanto para aqueles que desejam reforçar e aplicar os conceitos em trabalhos empíricos e projetos de pesquisa.

Parte I

Alfabetização computacional e lógica básica

Esta parte inicial do curso é dedicada à alfabetização computacional e à construção dos fundamentos da lógica de programação. O objetivo é desenvolver, de forma gradual, a capacidade de compreender, escrever e interpretar código, preparando o terreno para aplicações empíricas mais avançadas nas etapas seguintes do curso.

Nesta etapa, trabalharemos com os conceitos básicos da linguagem Python, sem assumir conhecimento prévio em programação. O foco está menos em bibliotecas especializadas e mais na compreensão dos elementos fundamentais que estruturam qualquer programa: sintaxe, tipos de dados e objetos, operações, controle de fluxo, funções e boas práticas de código. Listo abaixo alguns dos principais conceitos trabalhados nos capítulos que compõem esta parte do curso:

- **Capítulo 1 – Fundamentos de computação.** Introduz noções básicas sobre como computadores funcionam, o que significa programar e quais as características que diferenciam as várias linguagens de programação disponíveis. O objetivo é construir uma base conceitual mínima para entender o que acontece “por trás” do código.
- **Capítulo 2 – Primeiros passos no Python.** Apresenta o Python como linguagem de programação, discutindo sua sintaxe básica, a execução de comandos simples e a interação com o ambiente de desenvolvimento. Este capítulo marca o primeiro contato prático com a escrita e execução de código.
- **Capítulo 3 – Programação e Inteligência Artificial.** Discute, de forma introdutória, a relação entre programação, automação e modelos de inteligência artificial. O objetivo não é aprofundar técnicas de IA, mas contextualizar o uso de algoritmos, modelos e linguagens de programação em aplicações modernas, incluindo suas limitações e implicações.
- **Capítulo 4 – Tipos primitivos e objetos básicos.** Introduz os principais tipos de dados do Python – como números, *strings* e booleanos – e os objetos básicos utilizados para armazenar e manipular os diferentes tipos de informação. Este capítulo estabelece a base para compreender como a informação é representada em programas.
- **Capítulo 5 – Controle de fluxo e iteração.** Apresenta estruturas fundamentais para controlar a execução de um programa, como condicionais e laços de repetição (*loops*, em inglês). São discutidas formas de automatizar tarefas repetitivas e de implementar lógica condicional, elementos centrais em qualquer aplicação computacional.
- **Capítulo 6 – Funções.** Este capítulo tem por objetivo desenvolver o conceito de funções como forma de organizar código, evitar repetição e tornar programas mais legíveis e reutilizáveis. Discute definição, uso e boas práticas na criação de funções, preparando o aluno para estruturar programas mais complexos.

Juntos, esses seis capítulos estabelecem as bases conceituais e práticas da programação, permitindo que o aluno desenvolva familiaridade com o raciocínio computacional e com a linguagem Python. Ao final desta parte do curso, mais do que memorizar comandos específicos, espera-se que o estudante seja capaz de ler, compreender e construir pequenos programas, criando uma

base sólida para o uso de ferramentas computacionais em análise de dados e aplicações empíricas em economia nas etapas seguintes da disciplina.

1 Fundamentos de computação

1.1 Introdução

Antes de escrever qualquer linha de código, é importante entender o ambiente no qual a programação acontece. Programar não é apenas aprender uma linguagem específica, mas também compreender como o computador organiza arquivos, executa comandos e interage com o sistema operacional. Muitos dos erros e dificuldades enfrentados por iniciantes em programação não estão relacionados à lógica do código em si, mas ao uso do computador como ferramenta de trabalho: navegar por diretórios, localizar arquivos, executar programas e interpretar mensagens do sistema. Esses aspectos são frequentemente tratados como triviais, mas são fundamentais para o uso eficiente de qualquer linguagem de programação.

Este capítulo introduz conceitos fundamentais sobre o funcionamento de computadores, a representação de informações em bits e bytes, a organização de arquivos e a forma como interagimos com o sistema operacional via terminal. Esses elementos servirão como infraestrutura conceitual para todo o restante do curso. O foco aqui não é aprofundar detalhes técnicos de hardware ou sistemas operacionais, mas fornecer uma visão prática e funcional do ambiente computacional que será utilizado ao longo da disciplina.

1.1.1 Pré-requisitos

Neste capítulo introdutório não utilizaremos nenhum programa ou linguagem. Ele foi escrito para que mesmo o aluno sem nenhum tipo de conhecimento em computação e com o mínimo de conhecimento a respeito de operações matemáticas elementares tenha capacidade de acompanhar. Basta curiosidade!

1.2 Como um computador funciona

Um computador realiza duas coisas e apenas duas coisas: faz operações e guarda de forma eficiente o resultado dessas operações. No entanto, ao fazer essas duas coisas de forma extremamente eficiente essa máquina chamada computador nos permite receber, armazenar, processar e transmitir informações. Em essência, o computador nos serve ao propósito de **resolução de problemas**. Isso significa que, através de uma linguagem formal (especificamente operações de computação), podemos formular problemas e programar meios de resolver de



Figura 1.1: Setup usual de um computador pessoal no começo dos anos 2000. Fonte: *Wikimedia Commons*

forma automatizada tais problemas. No fim das contas, programar é o ato de criar programas, isto é, estabelecer uma sequência de instruções em linguagem formal que especifica como executar uma determinada operação no computador.

Mas antes de avançar na ideia de programa e algoritmos, quais são as características de um computador como o conhecemos?

1.2.1 Sistema computacional

Um sistema computacional é resultado da integração de componentes atuando como uma entidade, com o propósito de processar dados, isto é, realizar algum tipo de operação lógica envolvendo os dados, de modo a produzir diferentes níveis de informações. Tais sistemas são compostos por 3 principais elementos: hardware, software e o componente humano. O hardware inclui os componentes físicos enquanto o software é o conjunto de instruções que diz ao hardware o que fazer. No entanto, o software não elabora e decide sozinho o tipo de instrução a ser enviada ao hardware, é só através do componente humano que as instruções e objetivos são definidos.

- **Hardware:** componente físico de um sistema de computação, isto é, todos os equipamentos utilizados pelo usuário nas ações de entrada, processamento, armazenamento e saída de dados. Exemplos:
 - Teclado, mouse, impressora
 - Unidade central de processamento (CPU, em inglês) ou processador
 - Placa mãe
 - Placa de vídeo (GPU)
 - Memória RAM
 - Disco rígido de armazenamento
- **Software:** componente lógico de um sistema de computação, isto é, séries de instruções que fazem o computador funcionar. São os famosos programas de computador, que podem ser (i) proprietários e utilizáveis a partir da compra de uma licença ou (ii) gratuitos. Exemplos:
 - Editores de texto (e.g., Microsoft Word)
 - Editor de planilhas (e.g., Microsoft Excel)
 - Browsers para navegação na internet (e.g., Safari e Google Chrome)
- **Componente humano:** as pessoas, que utilizam o computador basicamente como ferramenta para atingir determinado fim, para resolver problemas. Sem a ação de um indivíduo o sistema computacional não funciona, já que (ainda) depende das instruções e direcionamento humano para definir o que o software deve fazer e como deve interagir com o hardware.

1.2.2 Arquitetura de um computador

A arquitetura de um computador define como os componentes estão interconectados e como eles colaboram para executar tarefas. Os computadores, como os conhecemos hoje, são estruturados em cima da lógica proposta inicialmente por John Von Neumann, matemático húngaro que viveu durante a primeira metade do século XX e que contribuiu imensamente em várias áreas das ciências. Embora tenhamos hoje uma complexidade maior nos elementos que compõe a arquitetura de um computador, a estrutura básica por trás dos computadores mais modernos continua sendo aquela proposta por Von Neumann. Essa arquitetura pode ser representado pelo diagrama abaixo:

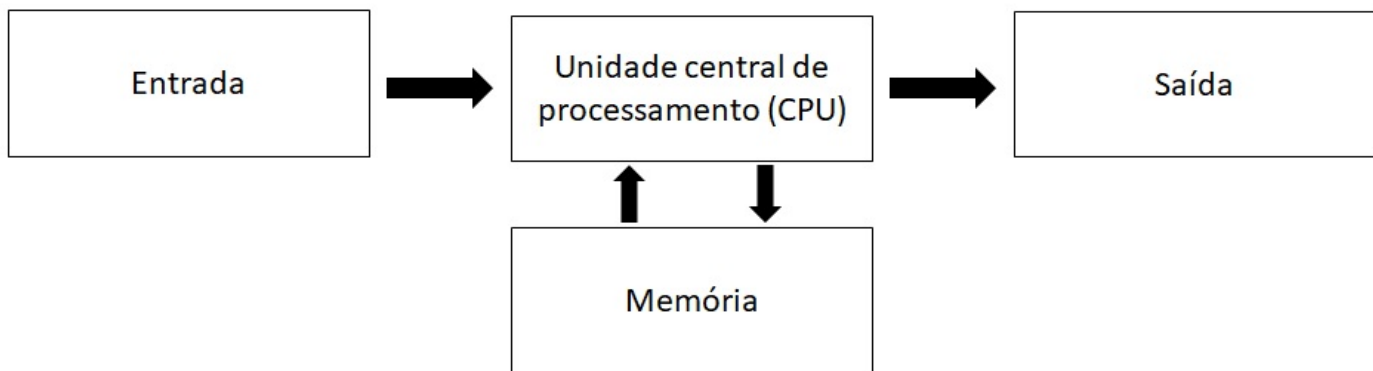


Figura 1.2: Diagrama que representa o funcionamento da arquitetura Von-Neumann

Essencialmente, o hardware que compõe as partes da arquitetura de um computador podem ser divididas em 3 partes:

- **Dispositivos de entrada e saída:** é através deles que o usuário dialoga com a máquina. Através do teclado e mouse, por exemplo, o usuário fornece informações ao computador a partir das quais processos serão realizados e seus resultados serão percebidos pelos dispositivos de saída (monitor e impressora, por exemplo).
- **Processador ou CPU:** é o cérebro do sistema computacional, é nele que serão executados cálculos e instruções lógicas dos usuários.
- **Memória:** aqui são armazenados temporariamente os resultados dos cálculos, dados e instruções. Esse armazenamento na memória, porém, não é feito de forma permanente, ele serve apenas como suporte ao processador. É como se o processador utilizasse a memória de caderno de anotações, retomando o resultado dos cálculos já realizados sempre que necessário.

! Importante

Não confunda *memória* com o que chamamos de *armazenamento*. A memória do computador, normalmente na forma da chamada memória RAM (sigla para *Random Access Memory*), é responsável por armazenar temporariamente os dados de dos programas que estão em uso, permitindo acesso rápido pelo processador. É também chamada de memória volátil: todo o seu conteúdo é perdido quando o computador é desligado.

Já as informações armazenadas no disco rígido (*Hard Disk Drive*, HDD) ou na unidade de estado sólido (*Solid-State Drive*, SSD) não são perdidas quando o computador é desligado. Esses dispositivos mantêm os dados armazenados de forma persistente, funcionando como a “memória de longo prazo” do computador. Por isso, esse tipo de armazenamento é chamado de memória não volátil.

Fazendo uma analogia com o funcionamento do nosso cérebro, a memória RAM armazena informações de curto prazo, como o que você almoçou hoje. Esse tipo de informação é importante ao longo do dia, por exemplo para saber se você já se alimentou bem ou se ainda está longe da sua meta diária de proteína, mas costuma ser esquecida depois que você dorme.

Já o HDD ou SSD armazena informações de longo prazo, como memórias da infância ou experiências passadas. Essas informações não são utilizadas constantemente no dia a dia, mas permanecem disponíveis e podem ser úteis em conversas com amigos ou como aprendizado acumulado, por exemplo na hora de decidir como cuidar dos seus filhos quando chegar a hora.

1.2.3 Bits e bytes

Para entender como um computador armazena e processa informações, é necessário compreender o conceito de sistema binário ou sistema de base 2¹. Diferentemente do sistema decimal, que utilizamos no dia a dia e possui dez dígitos (de 0 a 9), os computadores operam utilizando apenas dois estados possíveis: 0 e 1. Pense nisso como um interruptor, em que 1 representa o estado “ligado” e 0 o estado “desligado”. Os processadores atuais de um computador são resultado da junção de bilhões desses minúsculos “interruptores”, também chamados de transistores.

Esses dois estados, 0 ou 1, também são chamados de bits (do inglês *binary digits*). Um bit representa a menor unidade de informação que um computador pode armazenar. Qualquer informação processada por um computador – números, textos, imagens ou vídeos – é, em última instância, representada como uma sequência de bits.

Como trabalhar com bits individualmente seria pouco prático, eles são agrupados em unidades maiores chamadas de bytes. Um byte é composto por 8 bits e permite representar um conjunto maior de valores. Um byte pode representar 256 ($2^8 = 256$) combinações diferentes, o que é suficiente para representar todas as letras maiúsculas e minúsculas, números e símbolos básicos

¹Para entender um pouco mais sobre a história do sistema binário acesse o [link](#).

do alfabeto inglês. Não à toa, o byte se tornou a unidade padrão de medida quando o assunto é capacidade de armazenar informação em um computador. A Tabela 1.1 mostra como converter alguns números do sistema decimal para o sistema binário de 8 bits, em que cada dígito binário indica a presença ou ausência de uma potência de 2.

Tabela 1.1: Representação de números decimais no sistema binário e detalhamento do cálculo para conversão.

Número decimal	Número binário	Cálculo para conversão
0	00000000	$0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 0$
1	00000001	$0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1$
5	00000101	$0 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$
48	00110000	$0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 48$
137	10001001	$1 \cdot 2^7 + 0 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 137$
255	11111111	$1 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 255$

A partir dos bytes, surgem unidades maiores, amplamente utilizadas para medir o tamanho de arquivos, a quantidade de memória e a capacidade de armazenamento: quilobytes, megabytes, gigabytes, terabytes e petabytes. Essas unidades indicam, essencialmente, quantos bytes são necessários para representar uma determinada quantidade de informação.

- Kilobyte (KB): 1,024 (ou 2^{10}) bytes.
- Megabyte (MB): 1,048,576 (ou 2^{20}) bytes.
- Gigabyte (GB): 1,073,741,824 (ou 2^{30}) bytes.
- Terabyte (TB): 1,099,511,627,776 (ou 2^{40}) bytes.
- Petabyte (PB): 1,125,899,906,842,624 (ou 2^{50}) bytes.
- Exabyte, Zettabyte, Yottabyte...

Compreender bits e bytes é fundamental para entender por que arquivos têm tamanhos diferentes, por que algumas operações exigem mais memória do que outras e por que certas tarefas computacionais são mais custosas do ponto de vista de processamento. Esses agrupamentos de bits (bytes) são as unidades básicas que a memória e o armazenamento usam para guardar informação. A próxima seção explica o papel desses componentes e como eles impactam o desempenho prático.

1.2.4 Processamento, memória e armazenamento na prática

Podemos entender o funcionamento de um computador como um fluxo contínuo de **leitura**, **processamento** e **armazenamento** de bits. Quando observamos as especificações de um computador pessoal, é comum encontrar informações como número de núcleos do processador (*cores*), frequência do processador em gigahertz (GHz), quantidade de memória RAM e capacidade de armazenamento em disco. Esses números descrevem, essencialmente, a velocidade e a

escala com que o computador consegue manipular grandes volumes de informação binária em cada etapa desse fluxo contínuo.

Processamento refere-se à execução de instruções sobre dados representados em bits. O processador é o componente responsável por realizar operações elementares que, em última instância, são operações lógicas sobre sequências de zeros e uns. A frequência do processador indica aproximadamente quantos ciclos de processamento podem ser realizados por segundo. Frequências mais altas permitem que mais instruções sejam executadas em menos tempo, embora o desempenho final dependa também da organização do código, do tipo de tarefa e do acesso à memória. Além disso, processadores modernos possuem múltiplos *cores* ou núcleos. O uso de múltiplos núcleos permite paralelismo, isto é, a execução simultânea de diferentes partes de uma tarefa, de forma análoga a dividir um trabalho entre várias pessoas para concluí-lo mais rapidamente.

Memória (RAM) é o espaço onde os dados e instruções ficam armazenados temporariamente enquanto estão sendo processados. É na memória RAM que o computador mantém os bytes que serão acessados repetidamente pelo processador. Quanto maior a quantidade de RAM disponível, maior é o volume de dados que pode ser mantido “em uso” ao mesmo tempo. Quando a RAM é insuficiente, o computador passa a usar o disco como apoio, o que reduz drasticamente o desempenho.

Armazenamento, por sua vez, diz respeito ao local onde os dados são guardados de forma permanente. HDDs e SSDs armazenam grandes volumes de bytes mesmo quando o computador está desligado. A diferença principal é que SSDs permitem acesso muito mais rápido aos dados, reduzindo o tempo necessário para carregar programas, abrir arquivos e transferir grandes conjuntos de dados para a memória.

Vistos em conjunto, processamento, memória e armazenamento formam um sistema integrado: dados são lidos do disco, carregados na memória, processados pela CPU e, quando necessário, gravados novamente no armazenamento. Para tarefas simples, esse fluxo passa despercebido. Para programação, análise de dados e computação numérica compreender essas etapas ajuda a interpretar erros, gargalos de desempenho e limitações práticas do ambiente computacional.

💡 O que significam as especificações do seu computador?

Considere um computador com processador com 16 núcleos e frequência de 3.2 GHz, 32 GB de memória RAM e 1 TB de SSD. Esses números descrevem como ele lida com bits e bytes em diferentes etapas do processamento.

- **16 núcleos (cores)** indicam que o processador pode executar várias tarefas ao mesmo tempo. Em aplicações que permitem paralelismo – como certas operações numéricas ou análises sobre grandes conjuntos de dados – isso é semelhante a dividir o trabalho entre dezesseis pessoas em vez de apenas uma.
- **3.2 GHz** refere-se à velocidade com que cada núcleo executa instruções elementares.

Em termos simples, indica quantos passos o processador consegue dar por segundo ao manipular dados representados em sistema binário.

- **32 GB de RAM** significam que até esse volume de dados pode permanecer disponível para acesso rápido enquanto programas estão em execução. Bases de dados grandes, matrizes extensas e múltiplos processos consomem memória rapidamente.
- **1 TB de SSD** indica a capacidade de armazenamento permanente.

Em análise de dados e programação, problemas de desempenho raramente estão ligados apenas à velocidade do processador. Com frequência, eles surgem porque os dados são grandes demais para caber na memória ou porque operações são realizadas de forma pouco eficiente. Entender essas especificações ajuda a interpretar erros, lentidão e limitações práticas do ambiente computacional.

1.3 Arquivos e diretórios

Mais do que armazenar informação, é importante entender *onde* e *como* a informação é armazenada. Assim como na vida cotidiana, é preciso ter organização no armazenamento para saber como e onde procurar determinada informação no computador. Se você é uma pessoa organizada, você sabe que o iogurte estará sempre guardada na geladeira e em algum pote fechado, não é preciso procurar debaixo do chuveiro em um saco de pão.

1.3.1 Estrutura hierárquica

Os diretórios servem justamente para organizar o armazenamento do computador. É como se você pudesse separar o seu SSD primeiro em cômodos, depois em armários dentro de cada cômodo, em gavetas dentro de cada armário e assim por diante. Formalmente, um diretório pode ser entendido como um contêiner no qual arquivos e outros diretórios podem estar organizados, formando uma estrutura hierárquica. Essa organização permite localizar, acessar e manipular arquivos de forma sistemática, especialmente em projetos maiores.

Por exemplo, a organização de um projeto de análise de dados pode ter a seguinte forma:

```
C:\
  projeto\
    data\
      dados.csv
    scripts\
      analise.py
```

```
resultados\  
figura1.png
```

Nesse exemplo, `projeto` é um diretório que está dentro da partição `C` e que contém outros diretórios. O arquivo `dados.csv`, por exemplo, está dentro do diretório `data`, que é um diretório localizado dentro do diretório `projeto`. Para acessar esse arquivo `dados.csv` dentro dessa estrutura hierárquica de diretórios utilizamos duas formas principais: o caminho absoluto ou o caminho relativo.

- **Caminho absoluto:** descreve a localização completa a partir da raiz do sistema de arquivos. Nesse caso, o caminho absoluto seria `C:\projeto\data\dados.csv`.
- **Caminho relativo:** descreve a localização em relação ao diretório atual. Isso quer dizer que, se estivermos trabalhando em um programa a partir do diretório `projeto`, para acessar o arquivo de dados basta descrever o caminho relativo a partir dali: `data/dados.csv`.

1.3.2 Extensões de arquivos

Para além do *onde*, a informação em si pode ser armazenada de várias formas. Você pode guardar o iogurte em uma jarra de vidro ou mantê-lo no recipiente original. Um conjunto de informação é armazenada em um arquivo que pode ter vários formatos ou **extensões**. A extensão do arquivo corresponde ao sufixo após o ponto no nome do arquivo e indica como os dados estão organizados internamente e quais programas sabem interpretá-los corretamente. Do ponto de vista conceitual, a extensão não “muda” o dado em si, mas define a estrutura lógica usada para representar a informação. O mesmo conjunto de informações pode ser salvo em formatos diferentes, dependendo do objetivo (leitura humana, processamento automático, troca entre sistemas etc.).

Um formato muito comum em análise de dados é o CSV (`.csv`, *Comma-Separated Values*). Trata-se de um arquivo de texto em que os dados são organizados em linhas e colunas, sendo as colunas separadas por um delimitador, geralmente uma vírgula. Cada linha representa uma observação e a primeira linha costuma conter os nomes das variáveis. Arquivos CSV são amplamente utilizados porque (i) são simples e legíveis; (ii) podem ser abertos em editores de texto, planilhas e softwares estatísticos; e (iii) são facilmente importados por linguagens como Python.

Outro formato bastante utilizado é o JSON (`.json`, *JavaScript Object Notation*). Embora também seja um arquivo de texto, sua estrutura é diferente. Em vez de linhas e colunas, os dados são organizados como pares chave-valor, de forma semelhante a um dicionário, podendo conter listas e estruturas aninhadas. Esse formato é muito usado para troca de dados entre sistemas e aplicações, além do uso para armazenamento de configurações.

Outras extensões comuns incluem:

- `.txt`: texto simples, sem estrutura fixa.
- `.xlsx`: arquivos de planilha do Microsoft Excel.
- `.py`: arquivos de código Python.

É importante enfatizar que a extensão não garante o conteúdo do arquivo: um `.csv` mal formatado pode não ser lido corretamente, e um arquivo `.txt` pode conter dados altamente estruturados. Ainda assim, seguir convenções de extensão é essencial para organização de projetos, reprodutibilidade e interoperabilidade entre ferramentas. Considere a tabela de notas abaixo:

nome	nusp	ingresso	materia	media
Joao	12345678	2026	EAE1106	8
Miguel	5253678	2023	EAE1106	4
Alice	9743678	2025	EAE1106	9

O armazenamento desses dados em formato CSV e JSON é feito da seguinte maneira:

- Formato CSV

```
nome,nusp,ingresso,materia,media
Joao,12345678,2026,EAE1106,8
Miguel,52543678,2023,EAE1106,4
Alice,9743678,2025,EAE1106,9
```

- Formato JSON

```
[
  {
    "nome": "Joao",
    "nusp": 12345678,
    "ingresso": 2026,
    "materia": "EAE1106",
    "media": 8
  },
  {
    "nome": "Miguel",
    "nusp": 52543678,
    "ingresso": 2023,
    "materia": "EAE1106",
    "media": 4
  },
  {
    "nome": "Alice",
```

```
"nusp": 9743678,  
"ingresso": 2025,  
"materia": "EAE1106",  
"media": "9"  
}  
]
```

Mas qual a importância disso tudo? Entender como os computadores armazenam números e como arquivos e diretórios são organizados e acessados pelo sistema operacional ajuda a compreender limitações e comportamentos observados na prática. Por exemplo, explica por que um sistema de 32 bits é capaz de alocar apenas até 4 GB de memória RAM ou por que arquivos com milhões de linhas podem ser difíceis de abrir ou processar em um computador local.

Ter um conhecimento mínimo sobre o funcionamento do computador e sobre como os programas interagem com sua arquitetura permite entender o que ocorre “por trás das cortinas”. Com isso, podemos usar o computador como uma ferramenta para resolver problemas — e não como um fim em si. Para chegar a esse ponto, precisamos de meios formais para nos comunicar com o sistema operacional e, posteriormente, expressar instruções de forma lógica e precisa.

1.4 Terminal

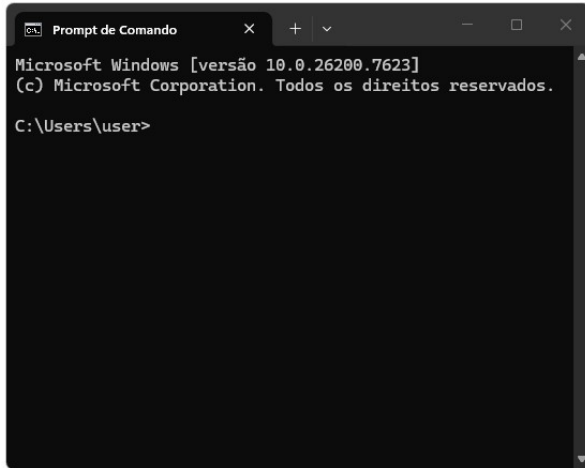
Um passo intermediário entre o funcionamento interno do computador e as linguagens de programação é o terminal. O terminal é uma interface textual que permite interagir diretamente com o sistema operacional por meio de comandos simples. Em vez de clicar em ícones ou navegar por menus gráficos, o usuário descreve explicitamente o que deseja que o computador faça.

O uso do terminal torna visíveis conceitos introduzidos na seção anterior, como arquivos e diretórios, além de fornecer um primeiro contato com a lógica de comandos que será essencial ao longo do curso. A Figura 1.3 ilustra o ambiente típico do terminal nos dois principais sistemas operacionais em uso atualmente.

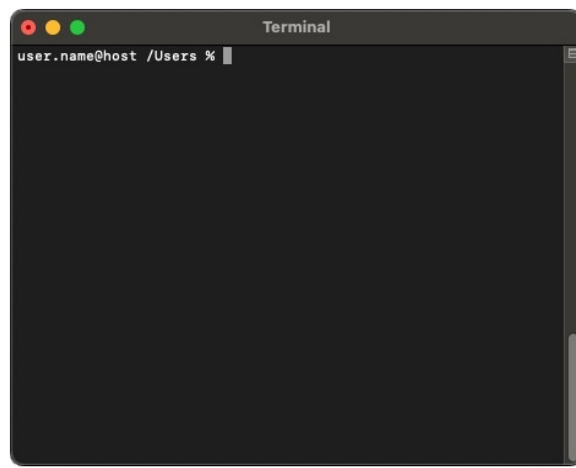
Comandos básicos no terminal (ou prompt de comando) do Windows incluem:

- **dir**: lista arquivos e diretórios no ambiente atual
- **cd**: muda de diretório
- **mkdir**: cria um novo diretório

No exemplo do painel A da Figura 1.3, **cd "C:\projeto"** muda o ambiente atual do diretório **C:\Users\user** para **C:\projeto**. Note, porém, que se a pasta **projeto** estiver em outra partição do seu armazenamento, é preciso primeiro mudar para essa partição para depois utilizar o comando **cd**. A Figura 1.4 ilustra esses dois casos.

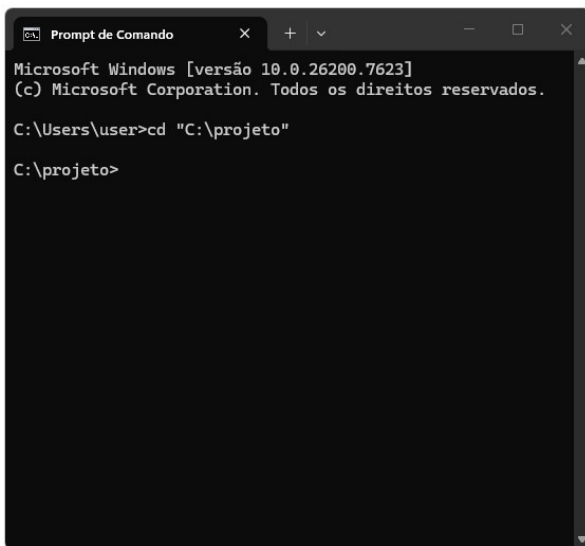


(a) Windows

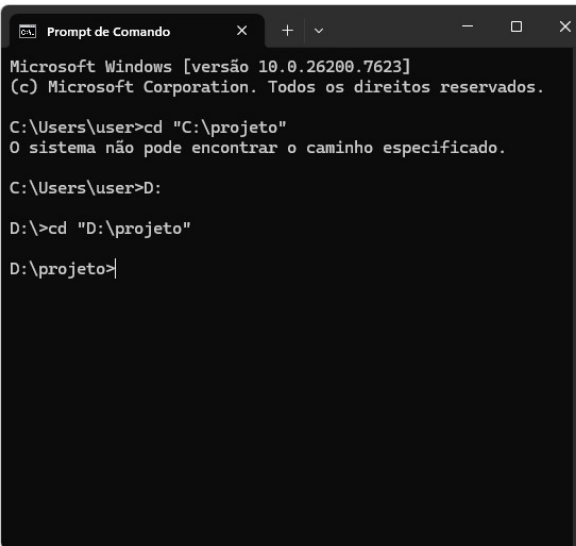


(b) MacOS

Figura 1.3: Interface usual do terminal



(a) Diretório na mesma partição



(b) Diretório em outra partição

Figura 1.4: Exemplo de uso do terminal para mudar o ambiente atual

Mesmo que você utilize interfaces gráficas, compreender o terminal facilita o entendimento de como o computador está organizado e de como o sistema operacional executa tarefas básicas. Ao longo do curso, o terminal será utilizado principalmente para executar programas, navegar entre diretórios e instalar novos pacotes e ferramentas.

Apesar de útil, o terminal ainda exige que as instruções sejam dadas comando a comando, de forma relativamente limitada e pouco reutilizável. Se queremos resolver problemas mais gerais e automatizar tarefas de forma sistemática, é necessário um nível adicional de abstração.

1.5 Linguagens de programação

Para atingir esse nível adicional de abstração, precisamos de um meio de comunicar instruções ao computador de maneira precisa e sem ambiguidades. Esse meio são as linguagens de programação, que definem regras formais para descrever operações, decisões e cálculos que o computador pode executar. Diferentemente das linguagens naturais, como o português que eu e você falamos, as linguagens de programação são linguagens formais, que foram criadas pelas pessoas para aplicações específicas. A notação usada pelos matemáticos é uma linguagem formal adequada para representar relações entre números e símbolos; químicos utilizam linguagens formais para representar a estrutura de moléculas. Utilizamos as linguagens de programação para nos comunicar de forma inequívoca com os computadores.

1.5.1 Sintaxe e semântica

Toda linguagem de programação possui dois níveis fundamentais: sintaxe e semântica.

A **sintaxe** refere-se às regras de escrita da linguagem. Ela define quais símbolos e quais sequências de símbolos são válidas: onde usar parênteses, palavras-chave, operadores e a ordem correta dos elementos. Erros de sintaxe são, em geral, fáceis de identificar porque a execução do programa é interrompida e uma mensagem de erro é apresentada. Exemplos comuns incluem esquecer um caractere obrigatório em determinada sequência, utilizar parênteses, colchetes ou aspas sem o fechamento correspondente ou escrever palavras-chave de forma incorreta. Nesses casos, o computador não consegue sequer interpretar a estrutura do código, independentemente da intenção do programador.

Por outro lado, a **semântica** refere-se ao significado das instruções. Um código pode estar sintaticamente correto, mas produzir resultados errados por não representar corretamente a intenção do programador. Erros semânticos são mais sutis, pois o código respeita todas as regras formais da linguagem e é executado normalmente, mas o resultado obtido está incorreto. Por exemplo, um programa pode calcular uma média dividindo a soma dos valores pelo número errado de observações ou usar uma variável diferente da pretendida em uma expressão. Esses erros não geram mensagens automáticas do computador e só podem ser detectados pela análise crítica dos resultados produzidos.

Essa distinção é central em programação. O computador consegue verificar automaticamente erros de sintaxe, mas não é capaz de julgar se o significado do programa está correto do ponto de vista do problema que se deseja resolver.

1.5.2 Diferenças entre linguagens naturais e formais

Embora as linguagens formais e naturais tenham muitas características em comum – símbolos, estrutura, sintaxe e semântica – há algumas diferenças:

1. **Ambiguidade:** as linguagens naturais são cheias de ambiguidade e as pessoas lidam com isso usando pistas contextuais e outras informações. As linguagens formais são criadas para ser quase ou completamente inequívocas, ou seja, qualquer afirmação tem exatamente um significado, independentemente do contexto
2. **Redundância:** para compensar a ambiguidade e reduzir equívocos, as linguagens naturais usam muita redundância. Por causa disso, muitas vezes são verborrágicas. As linguagens formais são menos redundantes e mais concisas.
3. **Literalidade:** as linguagens naturais são cheias de expressões e metáforas. Se eu digo “caiu a ficha”, provavelmente não há ficha nenhuma na história, nem nada que tenha caído (esta é uma expressão para dizer que alguém entendeu algo depois de certo período de confusão). As linguagens formais têm significados exatamente iguais ao que expressam.

Como todos nós crescemos falando linguagens naturais, às vezes é difícil se ajustar a linguagens formais. As linguagens formais são mais densas que as naturais, então exigem mais tempo para a leitura. Além disso, a estrutura é importante, então nem sempre é melhor ler de cima para baixo e da esquerda para a direita. Em vez disso, aprenda a analisar o programa primeiro, identificando os símbolos e interpretando a estrutura. E os detalhes fazem diferença. Pequenos erros em ortografia e pontuação, que podem não importar tanto nas linguagens naturais, podem fazer uma grande diferença em uma língua formal.

1.5.3 Alto nível X baixo nível

As linguagens de programação também diferem quanto ao seu nível de abstração. Na ciência da computação, **linguagens de programação de alto nível** abstraem detalhes do funcionamento interno do computador, permitindo que o programador se concentre na lógica do problema. São linguagens mais próximas das linguagens humanas e mais distantes das linguagens de máquina (e do sistema binário). Tais linguagens podem usar elementos de linguagem natural, serem mais fáceis de usar, ou podem ocultar totalmente áreas significativas de sistemas de computação, como o gerenciamento de memória. Isso torna o processo de desenvolvimento de um programa mais simples e eficiente. Python, R e Java são alguns exemplos de linguagens de alto nível.

Por outro lado, uma **linguagem de baixo nível** é uma linguagem de programação que fornece pouca ou nenhuma abstração de conceitos de programação e está muito próxima de escrever instruções de máquina reais. A palavra “baixo” refere-se à pequena ou inexistente quantidade de abstração entre a linguagem e a linguagem de máquina; por causa disso, as linguagens de baixo nível são às vezes descritas como *próximas do hardware*. Programas escritos em linguagens de baixo nível tendem a ser relativamente não portáteis e dependentes do computador para o qual foram escritas. Assembly é uma das linguagens de mais baixo nível à disposição do programador.

Em cursos como Economia e outras ciências aplicadas, linguagens de alto nível são preferidas porque reduzem o custo cognitivo, o tempo necessário para produzir código e aumentam a produtividade.

1.5.4 Linguagem Compilada X Interpretada

Outra distinção importante diz respeito à forma como o código é executado. Em **linguagens compiladas** o código é traduzido integralmente para código de máquina antes da execução. Elas também dão ao desenvolvedor mais controle sobre os aspectos de hardware, como gerenciamento de memória e uso da CPU. As linguagens compiladas precisam de uma etapa de “construção” – elas precisam ser compiladas manualmente primeiro. Somente após essa etapa que o código pode rodar por completo. Linguagens como C e C++ seguem esse modelo.

Em **linguagens interpretadas**, as instruções não são executadas diretamente pela máquina de destino, mas lidas e executadas, linha por linha, por algum outro programa, um intérprete. Isso permite maior flexibilidade e facilita a experimentação, embora geralmente com menor desempenho em tempo de máquina. Python e R são exemplos clássicos de linguagens interpretadas.

Embora usualmente mais lentas, essa simplicidade e flexibilidade em escrever código torna linguagens interpretadas particularmente adequadas para aprendizado, análise de dados e trabalho empírico. Lembre-se, no fim das contas, o tempo relevante não é apenas o tempo de máquina, mas o tempo que você leva para escrever e depurar código.

1.5.5 Paradigmas

Linguagens de programação também podem ser classificadas segundo o paradigma que adotam, isto é, o estilo predominante de organização do código. Entre os principais paradigmas de programação podemos citar programação imperativa, procedural, funcional, declarativa e programação orientada a objetos. Na programação imperativa, por exemplo, o programador diz como, o quê e em qual ordem exatamente um programa ou rotina deve realizar. É neste paradigma que surgiram as famosas estruturas condicionais e atribuição de valor à variáveis. Por outro lado, na programação orientada a objetos o programa é estruturado em objetos que

combinam dados e comportamento. Mais detalhes sobre paradigmas de programação podem ser obtidos [aqui](#).

Muitas linguagens modernas, incluindo Python, são multiparadigma, permitindo combinar diferentes estilos conforme o problema.

1.5.6 Tipagem

Por fim, as linguagens diferem quanto à forma como tratam os tipos de dados. Em linguagens de tipagem estática, o tipo de cada variável é definido explicitamente no código e verificado antes da execução. O Python utiliza tipagem dinâmica, isso significa que o próprio interpretador do Python infere o tipo dos dados que uma variável recebe, sem a necessidade que o usuário diga de que tipo determinada variável é. Além disso, o Python é uma linguagem fortemente tipada, o que significa que o interpretador do Python avalia as expressões por conta própria e não faz conversões automáticas de valores entre tipos de dados não compatíveis. Ao fazer operações com tipos incompatíveis, o Python não converte automaticamente esses tipos pra você, ele vai dar erro.

Tipagem dinâmica reduz a quantidade de código inicial e facilita experimentação, mas exige atenção adicional para evitar erros lógicos já que o usuário não define explicitamente o tipo do dado. No entanto, a tipagem forte reduz a chance de resultados inesperados por conta de tipos definidos ou convertidos de forma equivocada já que a linguagem não tenta corrigir o erro por conta própria.

1.5.7 Python

Neste curso, utilizaremos Python, uma linguagem de programação de alto nível, interpretada, multiparadigma e de tipagem dinâmica e forte. Python é amplamente utilizada em análise de dados, ciência de dados, automação, aprendizado de máquina e pesquisa empírica em Economia. Seu principal atrativo é permitir que o aluno foque na lógica do problema, e não em detalhes de implementação de baixo nível, além de contar com um ecossistema extenso de bibliotecas científicas. No próximo capítulo, deixaremos o nível conceitual e começaremos a programar de fato: escrever código, executar comandos, interpretar erros e manipular dados diretamente no computador.

1.6 Exercícios

1. X
2. X

2 Primeiros passos no Python

In summary, this book has no content whatsoever.

2.1 O Python

Python foi criado no final dos anos oitenta(1989) por Guido van Rossum no Centro de Matemática e Tecnologia da Informação (CWI, Centrum Wiskunde e Informatica), na Holanda, como sucessor da linguagem de programação ABC, capaz de lidar com exceções e interagir com o sistema operacional Amoeba. O nome da língua vem do gosto de seu criador pelos humoristas britânicos Monty Python (fonte: <https://blog.vulpi.com.br/python-como-surgiu/>)

Python é uma linguagem de programação de alto-nível, interpretada, multi-paradigma, e que usa tipagem dinâmica e forte.

bla bla bla

2.1.1 Porque escolher

Assim como tudo na vida, o Python é uma linguagem de programação com muitas qualidades mas que também possui seus defeitos (em geral ligados à velocidade de execução dos programas). Em sendo uma linguagem de alto nível, é uma linguagem bastante indicada, por exemplo, para aplicações que demandam replicabilidade, dada a facilidade de escrever e interpretar o código. Por outro lado, aplicações que demandam uma interação mais eficiente entre o código e o gerenciamento de memória da máquina, por exemplo, podem se beneficiar de outras linguagens.

No entanto, o conjunto de qualidades do Python e as inúmeras bibliotecas que foram desenvolvidas nos últimos anos parecem ter mais do que compensado as falhas da linguagem. Hoje o Python é uma das principais linguagens de programação (senão a principal) quando o assunto é ciência dos dados e aprendizado de máquina (*data science* e *machine learning*, respectivamente). É também uma das principais linguagens de programação por trás de vários dos sistemas de grandes empresas, como Uber, GoldmanSachs, Netflix e Google (fonte: [link](#)).

Por todos esses fatores, Python é uma das linguagens com a comunidade mais ativa em fóruns online voltados à programação e é a **linguagem que mais cresce no mundo**. As figuras abaixo ilustram bem esse crescimento da linguagem nos últimos anos:

PYPL Popularity of Programming Language

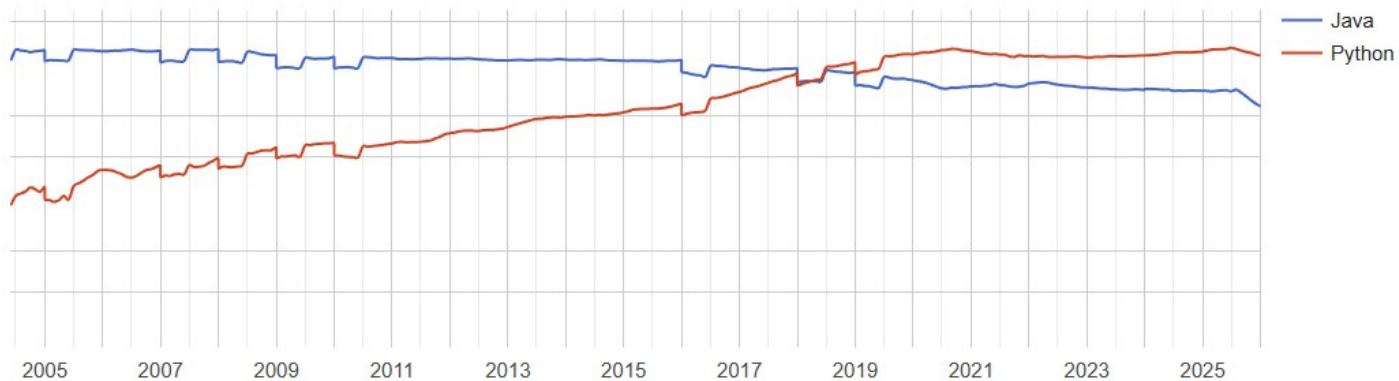


Figura 2.1: Índice PYPL de popularidade de linguagens

2.1.2 Como instalar

Existe mais de uma forma de instalar o Python no seu sistema e ainda mais formas de interagir com a linguagem. Você pode baixar o Python *puro* (<https://www.python.org/downloads/>) e instalá-lo diretamente em sua máquina, porém, essa distribuição vem com poucos pacotes já instalados e instalá-los um a um requer paciência. Além disso, a instalação do Python puro pode demandar algumas alterações em configurações do sistema via prompt de comando do Windows, por exemplo, para que seja possível dialogar com a linguagem.

Uma forma mais simples de instalar o Python em seu computador é através da distribuição Anaconda (<https://www.anaconda.com/>). Além de ser gratuita e conter vários dos pacotes que iremos utilizar, ela nos fornece diversas ferramentas que facilitam nossa interação com a linguagem. É hoje uma das distribuições do Python mais populares no mundo! **Recomendo fortemente utilizar esse caminho.**

O passo a passo a seguir foi feito para o sistema Windows 10. Embora o caminho seja parecido, podem haver algumas divergências em relação ao passo a passo para sistemas Linux e MacOS.

1. Baixar e instalar o Anaconda é fácil. Basta acessar a aba de downloads no site do Anaconda (<https://www.anaconda.com/download>), fornecer um email para cadastro e baixar o arquivo executável necessário para a instalação a depender do seu sistema operacional.
2. Depois de baixado, basta clicar duas vezes sobre o arquivo e ir acompanhando o instalador, mantendo sempre as opções padrão e escolhendo a opção de instalar para todos os usuários do computador, o que requer acesso de administrador.
3. Pronto, o Anaconda está instalado e os principais pacotes e programas que utilizaremos também!

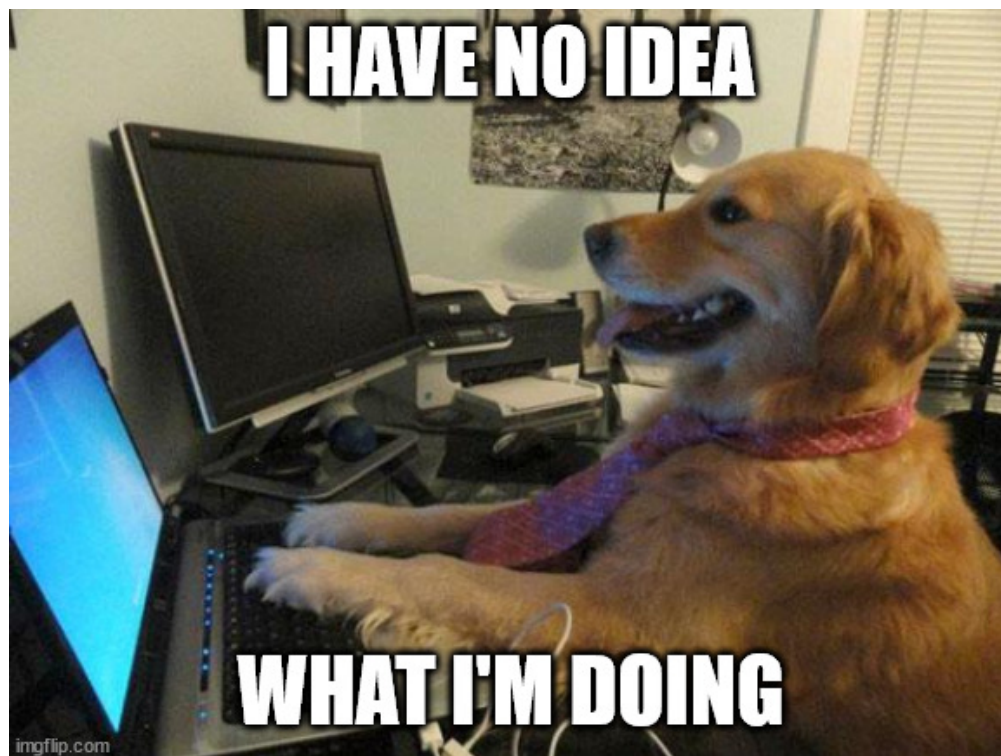


Figura 2.2: Meme

Download Now

For installation assistance, refer to [Troubleshooting](#).

Download Distribution by choosing the proper installer for your machine.




Anaconda Installers



Windows

Python 3.12

📄 64-Bit Graphical Installer (912.3M)



Mac

Python 3.12

📄 64-Bit (Apple silicon) Graphical Installer (704.7M)

📄 64-Bit (Apple silicon) Command Line Installer (707.3M)

📄 64-Bit (Intel chip) Graphical Installer (734.7M)

📄 64-Bit (Intel chip) Command Line Installer (731.2M)



Linux

Python 3.12

📄 64-Bit (x86) Installer (1007.9M)

📄 64-Bit (AWS Graviton2 / ARM64) Installer (800.6M)

📄 64-bit (Linux on IBM Z & LinuxO) Installer (425.8M)

Figura 2.3: Tela de download da distribuição Anaconda

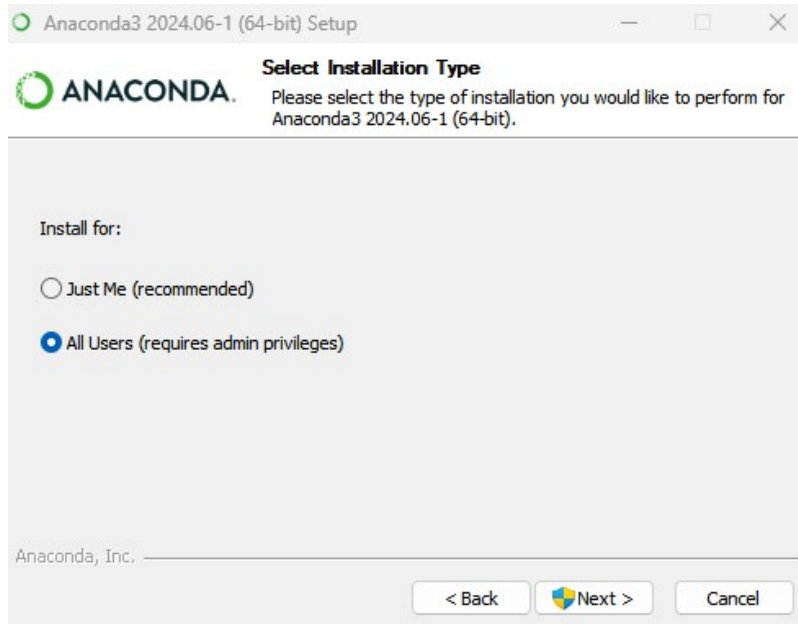


Figura 2.4: Tela de instalação da distribuição Anaconda

2.1.3 Anaconda e suas particularidades

Como dito acima, anaconda é uma distribuição do Python para computação científica (e.g, ciência de dados e aprendizado de máquina), que visa simplificar o gerenciamento e a implantação de pacotes. As versões de pacotes no Anaconda são gerenciadas pelo sistema de gerenciamento de pacotes *conda*. Através do *conda* é possível criar ambientes e instalar pacotes distintos de forma a evitar incompatibilidades entre versões de pacotes já instalados.

Além disso, o anaconda traz consigo o Anaconda Navigator, um ambiente *user-friendly* para a gestão de pacotes e aplicativos necessários para a utilização do Python sem que seja preciso ter qualquer conhecimento acerca da utilização correta de terminais (e.g, *prompt de comando* do Windows). Ao abrir o Anaconda Navigator no Windows, você deve ver algo assim

É possível instalar pacotes e fazer muito mais coisa diretamente pelo Anaconda Navigator. Para os que tiverem interesse em saber um pouco mais, esse [vídeo](#) é um bom começo.

2.1.4 IDEs

IDE é um acrônimo para Integrated Development Environment, em português, Ambiente de Desenvolvimento Integrado. Ele é um programa que reúne ferramentas necessárias para a construção de outros softwares. A utilização de um IDE ajuda muito os programadores e empresas, pois torna mais rápido o desenvolvimento de aplicações, aumentando a produtividade

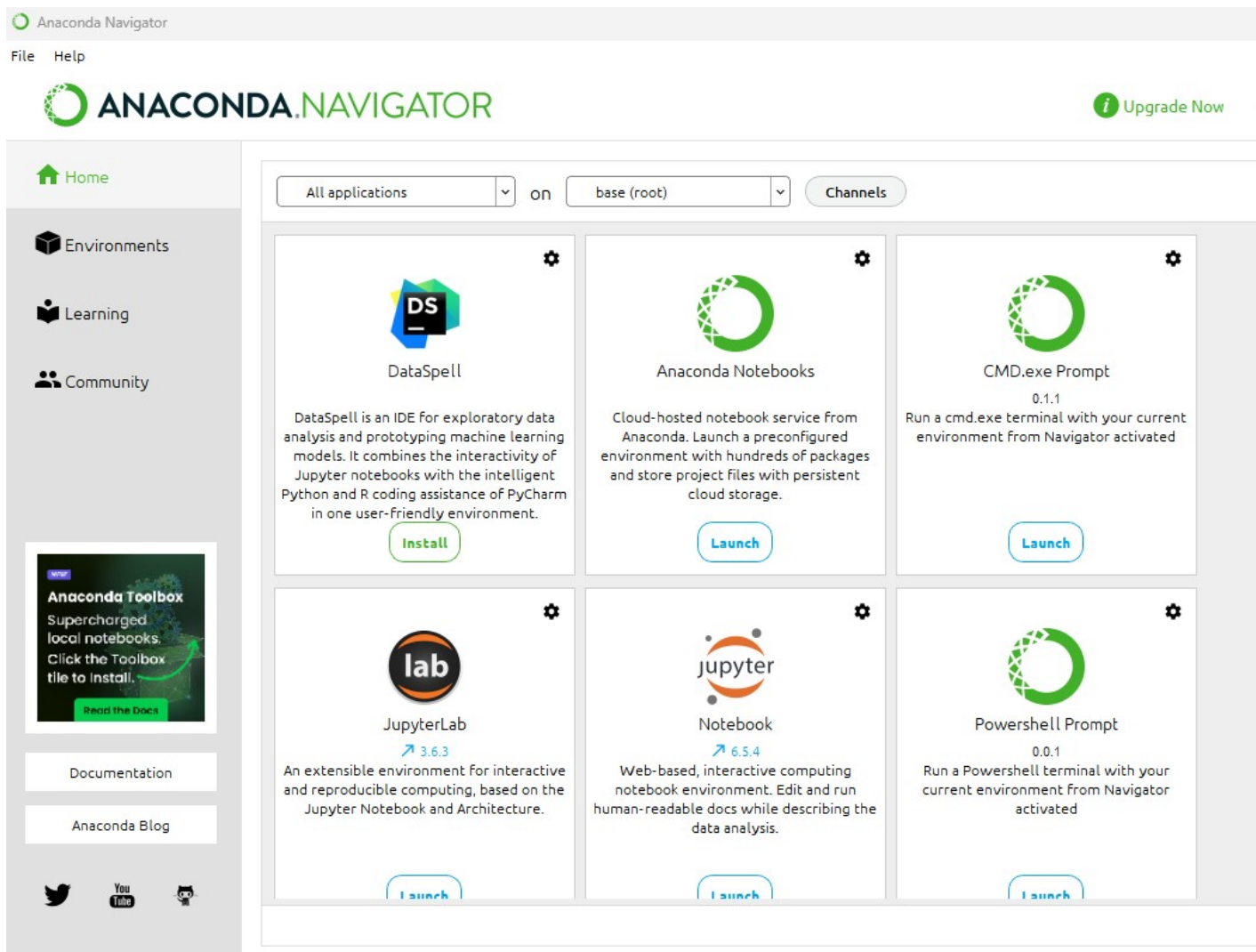


Figura 2.5: Tela inicial do Anaconda Navigator

e reduzindo custos. Existem IDE's específicas para plataformas, e outras que são mais flexíveis. As mais famosas são o Sublime Text e o Visual Studio Code.

Quais são os componentes típicos de uma IDE?

- **Editor de código-fonte:** Permite edição do código nas linguagens de programação suportadas pelo IDE.
- **Preenchimento inteligente:** Esse é um recurso trazido pelos IDEs que agiliza o desenvolvimento, pois escreve automaticamente trechos do código como, por exemplo, comandos de função.
- **Compilador:** O compilador que você escreveu em uma determinada linguagem de programação para a linguagem de máquina, de modo que os computadores o entendam.
- **Debugger:** É outra ferramenta que contribui para no código-fonte, melhorando o desempenho do programa.
- **Geração de código:** Com esse recurso, é possível predefinir trechos de códigos para serem usados de modelo em outros projetos, agilizando o desenvolvimento de trabalhos futuros.

Como podemos ver, os IDEs reúnem diversas ferramentas que tornam mais simples a vida dos programadores. Dentre todos os benefícios que a utilização desses programas pode trazer para seus projetos, podemos citar alguns:

- Reduz o tempo gasto em cada aplicação;
- Permite o desenvolvimento de um código mais limpo, organizado e legível;
- Aumenta a produtividade dos desenvolvedores e das empresas;
- Reduz a quantidade de bugs e falhas no código-fonte;
- Reúne diversas ferramentas em um só lugar.

2.1.4.1 Spyder

O Spyder é uma ferramenta leve, simples e ao mesmo tempo poderosa. É um IDE Python de código aberto que conta com elementos avançados de edição, depuração e testes interativos. Ele é bastante utilizado para o aprendizado de Data Science, apesar de não fornecer ferramentas tão avançadas nesse sentido como outras disponíveis. Mas ele é prático e seu depurador destaca bem funções, variáveis e erros. Conta também com um recurso de exploração de variáveis, que exibe os conteúdos armazenados dentro de cada uma. Isso poupa a escrita de comandos de impressão de variáveis na tela.

Disponível com a instalação do [Anaconda](#), uma das maiores e mais utilizadas plataformas de distribuição do Python.

2.1.4.2 Jupyter

É um IDE Python gratuito, utilizado principalmente na análise e ciência de dados. Ele é fácil e intuitivo, proporcionando um bom ambiente para iniciantes em Python. Também conta com muitos materiais de referência, tornando-se um dos IDEs mais utilizados pela comunidade. Ele trabalha muito bem com grandes conjuntos de dados. Além disso, é ótimo para a estética do código e atua como uma. É possível visualizar e editar facilmente seu código para deixá-lo mais atraente e apresentável.

Além de tudo isso, a estrutura do Jupyter serve muito ao propósito de tornar o código mais facilmente replicável, já que é possível intercalar células de comentários e explicações em linguagem Markdown com células de código propriamente dito, seguidas do resultado (ou erros) das instruções. Ele possui ainda integrações com HTML, por exemplo, que fazem a diferença principalmente na hora de apresentar projetos ou utilizá-los para o aprendizado. O material aula a aula do nosso curso, por exemplo, foi inteiramente desenvolvido e criado no Jupyter!

2.1.4.3 Google Colaboratory

O [Google Colaboratory](#), carinhosamente chamado de Colab, é um serviço de nuvem gratuito hospedado pelo próprio Google para incentivar a pesquisa de Aprendizado de Máquina e Inteligência Artificial.

É uma ferramenta que permite que você misture código fonte (geralmente em python) e texto rico (geralmente em markdown) com imagens e o resultado desse código, assim como o próprio Jupyter e sua estrutura de *notebooks* (“cadernos” em inglês). Uma diferença importante é que no caso do Colab os recursos computacionais utilizados para a execução do código são os da Google e não do seu computador. Apesar da versão gratuita disponibilizar apenas algo como 12GB de RAM, é possível ampliar essa capacidade de processamento da máquina virtual por uma conta paga que começa em \$5 mensais.

2.1.5 Spyder X Jupyter

Alguns dos pacotes mais utilizados no Python hoje em dia, como *Pandas* e *Matplotlib*, já vem instalados com o Anaconda, o que não é verdade na distribuição pura do Python. Além disso, alguns dos programas mais utilizados para interagir com a linguagem, como o *Spyder* e o *Jupyter Notebook* também. Tanto o *Spyder* quanto o *Jupyter* são ótimas IDEs para trabalhar com o Python, embora cada um tenha suas particularidades.

Existem vários comparativos entre essas duas formas (e tantas outras) de trabalhar com o Python (1, 2 e 3), mas no nosso caso talvez seja mais produtivo se formos direto para ambos os programas!

Acabamos de ver que uma das diferenças mais relevantes para nós, iniciantes na linguagem, ao usar o *Spyder* ou o *Jupyter* é a dificuldade, no segundo caso, de fazer uma gestão mais direta das variáveis criadas e da memória utilizada por cada uma delas. É possível, no entanto, superar essa dificuldade através de uma biblioteca que pode ser bastante útil para nós, o `nbextensions` (documentação [aqui](#) e [aqui](#)).

Vamos primeiro instalar essa biblioteca e mais uma outra biblioteca auxiliar que nos permitirá ter um controle maior sobre as extensões. Rode os comandos abaixo no *Anaconda Prompt*

1. `pip install jupyter_contrib_nbextensions`
2. `pip install jupyter_nbextensions_configurator`

Note que uma nova aba *Nbextensions* foi criada na “Home Page” do Jupyter Notebook. Ali é possível ativar e desativar as extensões que possam ser interessante ao nosso propósito. Agora já podemos brincar um pouco com tudo que ela pode nos fornecer!

2.2 Construindo o primeiro programa

Tradicionalmente, o primeiro programa que se escreve em uma nova linguagem chama-se “Hello, World!”, porque tudo o que faz é exibir as palavras “Hello, World!” na tela. No Python, ele se parece com isto:

```
print('Hello, World!')
```

Este é um exemplo de uma instrução `print` (instrução de impressão), embora na realidade ela não imprima nada em papel. Ela exibe um resultado na tela. As aspas apenas marcam o começo e o fim do texto a ser exibido; elas não aparecem no resultado. Os parênteses indicam que o `print` é uma função. No Python 2, a instrução `print` é ligeiramente diferente; ela não é uma função, portanto não usa parênteses.

2.2.1 Operadores aritméticos

Depois do “Hello, World”, o próximo passo é a aritmética. O Python tem operadores, que são símbolos especiais representando operações de computação, como adição e multiplicação. Os operadores `+`, `-` e `*` executam a adição, a subtração e a multiplicação. Finalmente, o operador `**` executa a exponenciação; isto é, eleva um número a uma potência, como nos seguintes exemplos:

```
40 + 2
```

```
43 - 1
```

```
6 * 7
```

```
84 / 2
```

```
6 ** 2 + 6
```

Em algumas outras linguagens, o $\wedge$ é usado para a exponenciação, mas no Python é um operador bitwise, chamado XOR. Se não tiver familiaridade com operadores bitwise, o resultado o surpreenderá. Não abordaremos operadores bitwise neste curso, mas você pode ler sobre eles em <http://wiki.python.org/moin/BitwiseOperators>.

```
6 ^ 2
```

Um valor é uma das coisas básicas com as quais um programa trabalha, como uma letra ou um número. Alguns valores que vimos até agora foram 2, 42.0 e 'Hello, World!'.

Esses valores pertencem a tipos diferentes: 2 é um número inteiro, 42.0 é um número de ponto flutuante e 'Hello, World!' é uma string, assim chamada porque as letras que contém estão em uma sequência em cadeia.

Se não tiver certeza sobre qual é o tipo de certo valor, o interpretador pode dizer isso a você:

```
type(2)
```

```
type(42.0)
```

```
type('Hello World')
```

Nesses resultados, a palavra “class” (classe) é usada no sentido de categoria; um tipo é uma categoria de valores. Como se poderia esperar, números inteiros pertencem ao tipo *int*, strings pertencem ao tipo *str* e os números de ponto flutuante pertencem ao tipo *float*.

E valores como '2' e '42.0'? Parecem números, mas estão entre aspas como se fossem strings:

```
type('2')
```

Um dos recursos mais eficientes de uma linguagem de programação é a capacidade de manipular variáveis. Uma variável é um nome que se refere a um valor.

2.2.2 Instruções de atribuição

Uma instrução de atribuição cria uma nova variável e dá um valor a ela:

```
message = 'And now for something completely different'
n = 17
pi = 3.141592653589793
```

Esse exemplo faz três atribuições. A primeira atribui uma string a uma nova variável chamada `message`; a segunda dá o número inteiro 17 a `n`; a terceira atribui o valor (aproximado) de π a `pi`. Uma forma comum de representar variáveis por escrito é colocar o nome com uma flecha apontando para o seu valor. Este tipo de número é chamado de diagrama de estado porque mostra o estado no qual cada uma das variáveis está (pense nele como o estado de espírito da variável).

2.2.3 Nomes de variáveis

Os programadores geralmente escolhem nomes significativos para as suas variáveis – eles documentam o uso da variável. Nomes de variáveis podem ser tão longos quanto você queira. Podem conter tanto letras como números, mas não podem começar com um número. É legal usar letras maiúsculas, mas a convenção é usar apenas letras minúsculas para nomes de variáveis. O caractere de sublinhar (`_`) pode aparecer em um nome. Muitas vezes é usado em nomes com várias palavras, como *`your_name`* ou *`airspeed_of_unladen_swallow`*.

Se você der um nome ilegal a uma variável, recebe um erro de sintaxe:

```
76trombones = 'big parade'
```

```
more@ = 1000000
```

```
class = 'Advanced Theoretical Zymurgy'
```

`76trombones` é ilegal porque começa com um número. `more@` é ilegal porque contém um caractere ilegal, o `@`. Mas o que há de errado com `class`?

A questão é que `class` é uma das palavras-chave do Python. O interpretador usa palavras-chave para reconhecer a estrutura do programa e elas não podem ser usadas como nomes de variável. O Python 3 tem estas palavras-chave:

<code>and</code>	<code>del</code>	<code>from</code>	<code>None</code>	<code>True</code>
<code>as</code>	<code>elif</code>	<code>global</code>	<code>nonlocal</code>	<code>try</code>
<code>assert</code>	<code>else</code>	<code>if</code>	<code>not</code>	<code>while</code>
<code>break</code>	<code>except</code>	<code>import</code>	<code>or</code>	<code>with</code>
<code>class</code>	<code>False</code>	<code>in</code>	<code>pass</code>	<code>yield</code>
<code>continue</code>	<code>finally</code>	<code>is</code>	<code>raise</code>	
<code>def</code>	<code>for</code>	<code>lambda</code>	<code>return</code>	

Você não precisa memorizar essa lista. Na maior parte dos ambientes de desenvolvimento, as palavras-chave são exibidas em uma cor diferente; se você tentar usar uma como nome de variável, vai perceber.

2.2.4 Expressões e instruções

Uma expressão é uma combinação de valores, variáveis e operadores. Um valor por si mesmo é considerado uma expressão, assim como uma variável, portanto as expressões seguintes são todas legais:

```
n
```

```
n + 25
```

Quando você digita uma expressão no prompt, o interpretador a avalia, ou seja, ele encontra o valor da expressão. Neste exemplo, o `n` tem o valor 17 e `n + 25` tem o valor 42.

Uma instrução é uma unidade de código que tem um efeito, como criar uma variável ou exibir um valor.

```
>>> n = 17
>>> print(n)
```

A primeira linha é uma instrução de atribuição que dá um valor a `n`. A segunda linha é uma instrução de exibição que exibe o valor de `n`. Quando você digita uma instrução, o interpretador a executa, o que significa que ele faz o que a instrução diz. Em geral, instruções não têm valores.

2.2.5 Pacotes de terceiros e funções não-nativas

2.2.5.1 Como instalar

Para instalar novos pacotes para o Python precisamos fazê-lo via prompt de comando. O Anaconda nos fornece um prompt de comando próprio, o **Anaconda Prompt** que facilita a instalação desses pacotes através dos comandos `conda` ou `pip`. Vamos instalar como exemplo o pacote `tqdm`, que nos permite criar barras de progresso em atividades repetidas.

1. Abra o **Anaconda prompt**.
2. Veja se o pacote já não está instalado: digite `conda list`.
3. Você pode utilizar 2 comandos distintos para instalar o pacote: `conda install tqdm` ou `pip install tqdm`.

Note também que alguns dos principais pacotes sobre os quais falarei ao longo do curso, *NumPy*, *SciPy*, *Matplotlib* e *Pandas*, já vem instalados com o Anaconda, o que não é verdade no Python puro. Mais do que mostrar o que temos instalado na nossa máquina local, o comando `conda list` nos mostra as versões de cada uma das bibliotecas disponíveis e qual o ambiente utilizado em sua instalação.

Diferença entre `conda` e `pip`: falar mais sobre a diferença entre esses dois comandos populares para instalação de pacotes vai um pouco além do escopo desse curso. De forma bastante resumida, podemos dizer que o `pip` é um gerenciador de pacotes que nos permite instalar qualquer biblioteca escrita em Python e disponível no *Python Package Index* (PyPI), o principal repositório de pacotes para a linguagem. O `conda`, no entanto, é mais do que um simples gerenciador de pacotes já que é possível fazer diferentes instalações em diferentes ambientes de modo a reduzir problemas de incompatibilidade entre versões. Além disso, através do `conda` podemos instalar pacotes escritos em outras linguagens também, como R e C++. Por outro lado, é possível que você demore mais e tenha problemas para instalar alguns pacotes utilizando o `conda`. Aos que quiserem saber um pouco mais sobre a diferença entre esses dois métodos para instalação, esses links ([1](#) e [2](#)) podem ser um bom começo.

2.2.5.2 Como importar

Sempre que quisermos trazer algo “de fora”, devemos carregar as bibliotecas específicas, seja por completo ou apenas um subconjunto de suas funções. Fazemos isso por meio do comando `import`. Vamos utilizar como exemplo a biblioteca `NumPy`, uma das principais bibliotecas de comando do Python em se tratando de operações algébricas e matriciais.

Dedicaremos uma aula inteira para trabalhar com o `NumPy` daqui algumas semanas, mas por hora trabalharemos apenas com as funções geradoras de números aleatórios, em especial a função geradora de números aleatórios distribuídos de acordo com uma Distribuição Uniforme

padrão (você deve se lembrar das características dessa distribuição das aulas de estatística, mas caso ainda restem dúvidas sempre existe o [Wikipedia](#)).

Há 3 formas de trabalhar com essa função específica, contida no NumPy:

1. Podemos importar a biblioteca inteira

```
import numpy
```

2. Podemos importar a biblioteca inteira mas dando-lhe um “apelido”

```
import numpy as np
```

3. Podemos importar apenas a função que nos interessa nesse caso

```
from numpy.random import uniform
```

Qual a diferença entre esses métodos? No Python, sempre que formos utilizar uma função de algum pacote “de fora” é preciso dizer ao Python de onde que essa função está vindo. Se quisermos usar a função `uniform()` através do 1º método, por exemplo, é preciso chamá-la utilizando o nome do pacote, nesse caso `numpy.random.uniform()`. O 2º método encurta esse nome de tal forma que é possível chamar a mesma função usando `np.random.uniform()`. Por fim, no 3º caso basta chamar a função diretamente, isto é, `uniform()`.

Mas se o 3º caso é mais direto, porque não usá-lo sempre? Por duas razões bem simples: (i) usar a sintaxe dos 2 primeiros casos torna o código mais compreensível e depurável, já que no caso de algum problema de execução sabemos onde procurar a resposta, e (ii) é comum pacotes distintos usarem o mesmo nome para funções que fazem operações distintas, o que pode gerar problemas de incompatibilidade e/ou de executarmos algo diferente daquilo que gostaríamos de executar.

Na maioria das vezes, acabamos utilizando o 2º método, já que isso permite reduzir linhas desnecessárias de código ao mesmo tempo que facilita a replicabilidade e a atividade de depuração do código.

2.3 Exercícios

1. Inicialize o interpretador do Python e use-o como uma calculadora. Responda as seguintes perguntas:
 - a. Quantos segundos há em 42 minutos e 42 segundos?
 - b. Quantas milhas há em 10 quilômetros? Dica: uma milha equivale a 1,61 quilômetro.

- c. Se você correr 10 quilômetros em 42 minutos e 42 segundos, qual é o seu passo médio (tempo por milha em minutos e segundos)? Qual é a sua velocidade média em milhas por hora?
2. Crie uma conta no StackOverflow e monte seu perfil.

3 Programação e IA

3.1 Introdução

Por mais que se conheça a sintaxe e estrutura de uma linguagem de programação, construir um código eficiente pode não ser das tarefas mais fáceis. Além disso, traduzir um código de uma determinada linguagem para outra linguagem pode ser igualmente difícil, mesmo com sintaxes parecidas.

Utilizar a Inteligência Artificial para fazer isso pela gente pode nos economizar um tempo valioso. Antes de entender a aplicabilidade dos modelos de inteligência artificial no mundo da programação, é preciso primeiro definir inteligência artificial e, principalmente, sua vertente generativa.

3.1.1 Pré-requisitos

Embora as funções e estruturas necessárias para a realização de *loops* no Python sejam nativas na linguagem, para algumas das aplicações utilizaremos um dos pacotes externos que nos permitem trabalhar com números aleatórios.

- numpy

```
import numpy as np
```

3.2 O que é Inteligência Artificial?

Definir IA é uma tarefa difícil, já que se trata de uma área da ciência da computação bastante ampla que, a depender do contexto, envolve um conjunto de modelos e aplicações diferentes. Em todos os casos, porém, é uma área responsável por criar sistemas capazes de executar tarefas que requerem inteligência humana, como aprendizado, raciocínio, reconhecimento de padrões, processamento de áudio, vídeo e linguagem. Esses sistemas são construídos à partir de uma grande amostra de dados, nos quais diferentes modelos são treinados e à partir dos quais é possível construir previsões e análises sobre conjuntos de dados novos. Podemos citar alguns exemplos que, em geral, são aplicações de sistemas de IA: sistemas de recomendação, robótica, diagnóstico médico, sistema de previsão, automação.

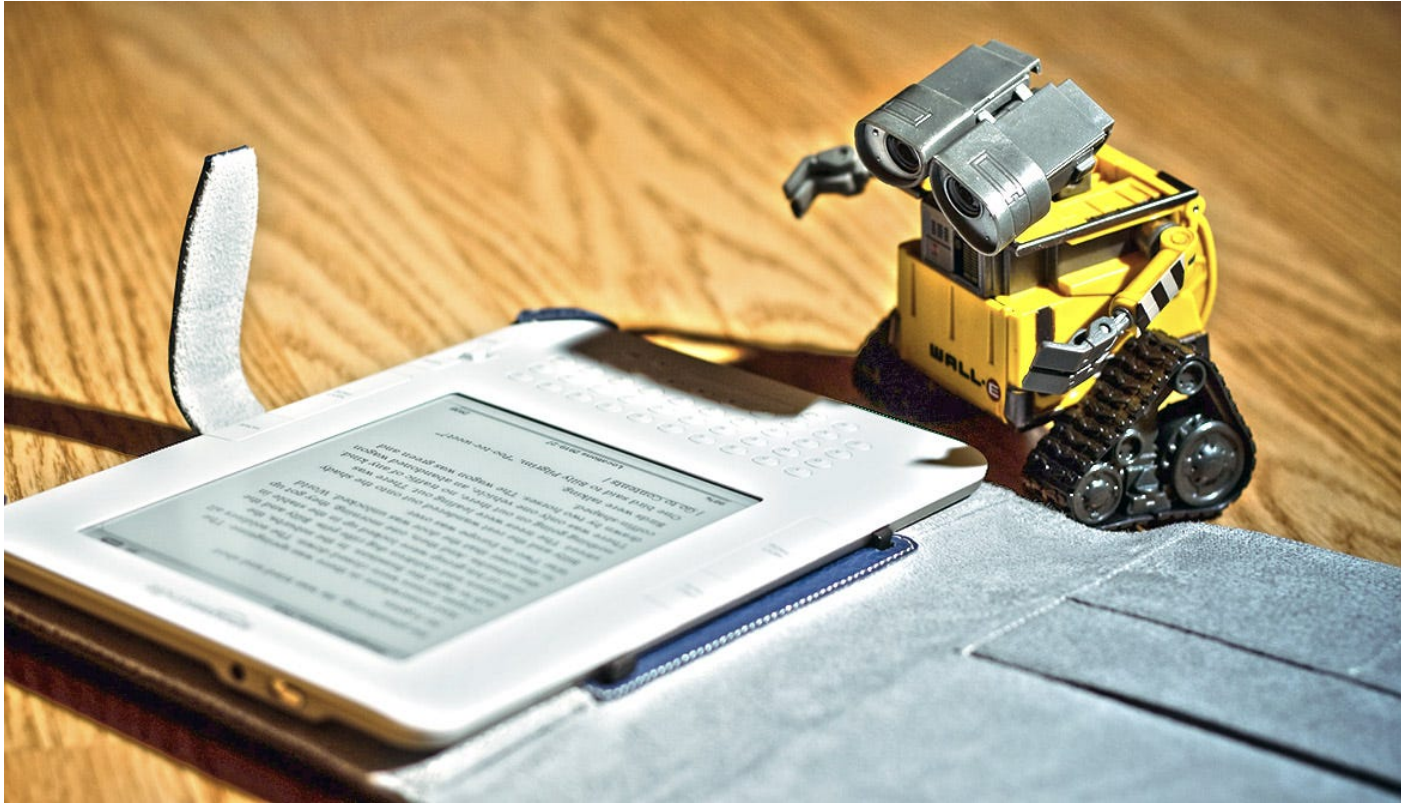


Figura 3.1: Meme 2

Embora se discuta Inteligência Artificial há algum tempo, apenas mais recentemente a chamada Inteligência Artificial Generativa (IAG) ganhou espaço no debate e nos modelos de uso comercial. Diferentemente dos algoritmos mais gerais de inteligência artificial, os modelos de IAG são projetados para a criação de algo novo, que não estava nos dados originais. Em essência, os sistemas de IAG são sistemas de IA com um escopo mais restrito e bem definido.

Dentro do grupo de modelos de IAG estão os famosos GPTs, como o [ChatGPT](#) e o [Gemini](#). Os GPTs são modelos da classe dos *Large Language Models* (LLM), que compreendem e geram conteúdo textual novo com base em uma etapa de pré-treino em grande volume de textos. Nesses modelos, o sistema recebe uma mensagem (prompt) de entrada, “raciocina” em cima do texto fornecido e dos dados nos quais foi pré-treinado, e retorna uma mensagem de saída em resposta. É como se o modelo tivesse por objetivo “prever” o conteúdo textual que melhor se encaixa ao conteúdo fornecido no prompt inicial.

i Curiosidade

Você sabe o que significa GPT? É a sigla para **Generative Pre-trained Transformer**, que resume as características dos modelos por trás de sistemas como o ChatGPT. *Generative* faz referência a modelos de IAG, que tem como objetivo principal criar conteúdo novo a partir de um dado conjunto de informações. *Pre-trained* faz referência ao fato do modelo ser pré-treinado em um grande volume de dados textuais genéricos, em geral de domínio público como o Wikipedia, até uma determinada data de corte. Por fim, *Transformer* é uma classe de modelos mais recentes, desenvolvidos inicialmente pelas equipes de pesquisa do Google, com certas particularidades que os tornam altamente eficientes e assertivos em tarefas que envolvem mensagens de entrada em formato de texto.

3.3 ChatGPT

3.3.1 Acesso à plataforma e primeiro uso

O acesso à ferramenta é feito pelo [site](#) ou pelo aplicativo móvel, disponível gratuitamente na Apple Store ou Google Store. A plataforma se parece com um chat qualquer, assim como um aplicativo de mensagens. A diferença é que quem responde é o próprio modelo.

Colar imagens

As requisições ao modelo são gratuitas, porém, com acesso limitado às versões mais recentes. A OpenAI, empresa por trás do ChatGPT, oferece alguns planos pagos que dão direito a uso ilimitado dessas novas versões e outras funcionalidades.

Colar imagens

Nem sempre as ferramentas de inteligência artificial, como o ChatGPT, são indicadas. Questões como acurácia, verificabilidade e uso de dados sensíveis são relevantes na hora de definir se o ChatGPT é a ferramenta mais indicada para uma aplicação específica.

Colar imagens

3.3.2 Conceitos básicos de engenharia de prompt

Prompt Engineering (ou engenharia de prompt) é o processo de estruturar uma mensagem de entrada de forma a auxiliar a compreensão dos modelos de linguagem e ampliar a assertividade e eficiência das respostas. Apesar de ser uma área em expansão, que conta com várias estratégias para a construção de prompts altamente personalizados para cada aplicação, são duas as características essenciais que devem estar presentes em todo prompt: clareza e especificidade.

- **Clareza:** para tornar o prompt claro aos olhos do modelo, é preciso fornecer elementos que objetivem que auxiliem o modelo a fornecer respostas mais direcionadas. Para isso, é importante (i) descrever o contexto relevante, (ii) adicionar detalhes específicos ao objeto e (iii) ser preciso nas instruções.
- **Especificidade:** para tornar o prompt menos genérico e melhor direcionar a resposta do modelo, é preciso que o foco seja apenas no que é mais relevante para o objetivo em questão. Prompts vagos e ambíguos promovem respostas menos acuradas e eficientes. Nesse caso, (i) seja descritivo, (ii) evite amiguidades e informações desnecessárias, (iii) evite dizer ao modelo o que não fazer; o foco deve ser em instruções do que queremos que seja feito.

No fim das contas, um bom prompt é aquele que oferece as instruções corretas ao modelo, que então devolve a resposta esperada. Alguns elementos ajudam nesse processo de construir um prompt mais claro e específico.

COLAR IMAGEM

3.3.3 Alguns exemplos

3.3.4 Integração com ferramentas e APIs externas

Nem sempre precisamos de um browser para utilizar o ChatGPT. O acesso às diferentes versões pode ser realizado através de uma Interface de Programação de Aplicações, ou API em inglês. Uma API é um conjunto de regras e protocolos que configuram uma forma simplificada de comunicação entre diferentes softwares. É possível, por exemplo, acessar as funcionalidades da ferramenta dentro de um código da linguagem de programação Python.

No caso do ChatGPT, para acessar a API é preciso se registrar nesse [link](#) e garantir uma chave de acesso. A OpenAI já forneceu limites gratuitos para teste da API. Por conta de sobrecarga dos servidores e uso inadequado, hoje é preciso pagar pelo uso da API.

Usuários que assinam a versão paga do ChatGPT não têm acesso direto à API. Para utilizar a API, o usuário cadastra um cartão de crédito e, ao final do mês, é cobrado pelo número e complexidade das demandas. É possível configurar limites de uso de modo a evitar gastos maiores do que o esperado. O preço é diferente para modelos diferentes. <https://openai.com/api/pricing/>

Exemplo

3.4 StackOverflow

[Stack Overflow](#) é um site de perguntas e respostas, um fórum, para programadores profissionais e entusiastas. É o site principal da Stack Exchange Network. Foi criado em 2008 por Jeff Atwood e Joel Spolsky. Ele apresenta perguntas e respostas sobre uma ampla gama de tópicos em programação de computadores. Ele foi criado para ser uma alternativa mais aberta aos sites anteriores de perguntas e respostas, como o Experts-Exchange. O Stack Overflow foi vendido para a Prosus, um conglomerado de internet para consumidores com sede na Holanda, em 2 de junho de 2021 por US\$ 1,8 bilhão.

O site serve como uma plataforma para os usuários fazerem e responderem perguntas e, por meio de associação e participação ativa, votarem em perguntas e respostas semelhantes ao Reddit e editarem perguntas e respostas de maneira semelhante a um wiki. Os usuários do Stack Overflow podem ganhar pontos de reputação e “emblemas”; por exemplo, uma pessoa recebe 10 pontos de reputação por receber um voto “para cima” em uma pergunta ou resposta a uma pergunta, e pode receber medalhas por suas valiosas contribuições, o que representa uma gamificação do tradicional Q&A local na internet. Os usuários desbloqueiam novos privilégios com um aumento na reputação, como a capacidade de votar, comentar e até editar as postagens de outras pessoas.

Em março de 2021, o Stack Overflow tinha mais de 14 milhões de usuários registrados e recebeu mais de 21 milhões de perguntas e 31 milhões de respostas. Esse fórum, juntamente com outros sites de perguntas e respostas de programação semelhantes, substituíram globalmente principalmente os livros de programação para referência de programação do dia-a-dia nos anos 2000, e hoje são uma parte importante da programação de computadores. Vocês lembram do nosso gráfico de participação de algumas das principais linguagens no total de perguntas do StackOverflow? Qual linguagem está em 1º lugar desde o fim de 2018?

Mas pera lá, o que significa esse monte de adjetivo? (fontes: computersciencewiki.org, geeksforgeeks.org, freecodecamp.org)

3.5 Exercícios

1. X
2. X

4 Tipos primitivos e objetos básicos

4.1 Introdução

4.1.1 Pré-requisitos

Nesse capítulo você utilizará essencialmente funções nativas do Python. No entanto, para trabalhar com padrões textuais de forma mais profunda utilizaremos pacotes externos:

- `re`

```
import re
```

4.2 Strings

Strings não são como números inteiros ou de ponto flutuante. No Python, uma string nada mais é do que uma sequência ordenada de caracteres unicode. Eles são delimitados sempre por aspas (simples ou duplas). Relembrando nosso primeiro “programa” da aula passada podemos atribuir a uma variável a string *Hello, World* usando aspas.

```
str1 = 'Hello, World'

print(str1)
print(type(str1))
```

```
Hello, World
<class 'str'>
```

Usando aspas duplas o resultado seria o mesmo

```
str2 = "Hello, World"

print(str2)
print(type(str2))
```

```
Hello, World
<class 'str'>
```

Note que podemos também usar uma sequência de três aspas duplas e escrever strings que percorrem várias linhas.

```
str3 = """Das Utopias

Se as coisas são inatingíveis...ora!
Não é motivo para não querê-las...
Que tristes os caminhos, se não fora
A presença distante das estrelas!

Mario Quintana
"""

print(str3)
print(type(str3))
```

Das Utopias

Se as coisas são inatingíveis...ora!
Não é motivo para não querê-las...
Que tristes os caminhos, se não fora
A presença distante das estrelas!

Mario Quintana

```
<class 'str'>
```

4.2.1 Operações básicas

Por ser uma sequência de caracteres e não um número, operações aritméticas (em geral) não são permitidas, mas outras operações, como *slicing*, o são. É possível acessar um caracter específico da sequência utilizando a posição desse caractere, utilizando o que chamamos de **índice**. No caso do string *'Hello, World'*, para acessar a segunda letra podemos utilizar colchetes após o string e dentro dele o índice referente à posição do 'e' no string:

```
print(str1)
```

Hello, World

```
str1[2]
```

```
'l'
```

Ué, mas porque que obtivemos como resposta o caractere 'l', que é o 3º na sequência, e não o 'e', que é o 2º elemento? Aqui vai uma particularidade da sintaxe do Python: **em se tratando de índices, o Python sempre começa a contagem em 0!**. Dessa forma, para acessar o primeiro caractere de 'Hello, World' devemos pedir `str1[0]`, para acessar o segundo é preciso pedir `str1[1]` e assim por diante.

```
str1[1]
```

```
'e'
```

Podemos acessar contando de trás para frente e usando um índice negativo

```
str1[-1]
```

```
'e'
```

É possível que o índice seja uma expressão, mas deve sempre ser um valor inteiro.

```
n=0  
str1[n+1]
```

```
'e'
```

```
str1[1.5]
```

```
TypeError: string indices must be integers, not 'float'
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[10], line 1  
----> 1 str1[1.5]  
TypeError: string indices must be integers, not 'float'
```

Para acessar uma **fatia** (*slice*) do string podemos utilizar um intervalo de índices (inclusive o índice inicial e exclusive o índice final). Por exemplo,

```
str1[0:5]
```

```
'Hello'
```

Além de ser possível calcular o número de caracteres em uma sequência string utilizando a função integrada `len()`, podemos “concatenar” ou “somar” strings utilizando apenas o sinal de +

```
len(str1)
```

```
12
```

```
str1 + ' --- ' + str2
```

```
'Hello, World --- Hello, World'
```

Note, porém, que strings são **imutáveis**, de modo que caso queira substituir um dos caracteres dentro de um string é preciso utilizar uma função específica para isso ou criar um novo string derivado do 1º. Apenas tentar substituir um dos caracteres de um string já definido não é permitido.

```
# Tentemos substituir o 'e' do str1 por 'a'
str1[1] = 'a'
```

```
TypeError: 'str' object does not support item assignment
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[14], line 2
      1 # Tentemos substituir o 'e' do str1 por 'a'
----> 2 str1[1] = 'a'
TypeError: 'str' object does not support item assignment
```

4.2.2 Métodos de strings

As strings oferecem métodos que executam várias operações úteis. Um método é em essência uma sequência de instruções encapsuladas dentro de um único comando que recebe argumentos e devolve um valor. Embora a sintaxe seja diferente, a ideia é a mesma quando falamos de funções, objeto do nosso estudo daqui algumas aulas.

No caso dos métodos, temos que passar o nome da string que foi definida anteriormente seguida de ‘.’ e depois do comando relacionado ao método específico. Dentre os principais métodos aplicáveis a strings e suas funcionalidades podemos citar:

1. **str_example.upper()**: devolve a string 'str_example' toda em letras maiúsculas.
2. **str_example.lower()**: devolve a string 'str_example' toda em letras minúsculas.
3. **str_example.strip()**: devolve a string 'str_example' retirando possíveis espaços em branco no início e no fim da string.
4. **str_example.startswith('xyz')**: testa se a string 'str_example' começa com a string 'xyz'.
5. **str_example.endswith('xyz')**: testa se a string 'str_example' termina com a string 'xyz'.
6. **str_example.find('xyz')**: procura a string 'xyz' dentro de 'str_example' e retorna o primeiro índice onde 'xyz' começa ou retorna -1 se nada for encontrado.
7. **str_example.replace('old','new')**: retorna uma string nova onde todas as ocorrências de 'old' encontradas em 'str_example' serão substituídas por 'new'.

Além desses principais, existem outros vários métodos para strings. Esse [link](#) é um bom ponto de partida para quem quiser conhecer outros exemplos.

```
str1 = ' Hello, World '  
  
print(str1.upper())  
print(str1.lower())
```

```
HELLO, WORLD  
hello, world
```

```
print(str1.strip())
```

```
Hello, World
```

```
print(str1.startswith(' '))  
print(str1.endswith('d'))
```

```
True  
False
```

```
print(str1.find(', '))
```

6

```
print(str1.replace('Hello', 'World'))
```

World, World

4.2.3 Formatação e a instrução print

Um método sobre o qual não falamos, mas que é bastante interessante quando queremos, por exemplo, printar bonitinho o resultado de determinada operação é `s.format()`. Com esse método podemos converter uma variável numérica para uma formato específico e printá-la dentro de um string maior. Imagine, por exemplo, que estejamos interessados em printar o valor de π arredondado para 2 casas decimais apenas dentro de um string que diz isso. Podemos implementar isso da seguinte forma

```
pi = 3.1415926535
print('O valor de pi arredondado para 2 casas decimais é {:.2f}. Interessante, não?'.format(pi))
```

O valor de pi arredondado para 2 casas decimais é 3.14. Interessante, não?

Os colchetes dentro do string mostram onde que o número deve aparecer. Mais do que isso, definimos dentro do colchete o formato do número. Após os `'.'`, o `'2'` significa que queremos 2 casas decimais enquanto `'f'` significa que queremos um formato de ponto fixo. Não vou entrar nos detalhes de todas as formatações possíveis, mas podemos ver um pouco mais disso [aqui](#).

Podemos também usar mais de um número dentro do mesmo método `format`. Por exemplo,

```
print('O valor de pi arredondado para 2 casas decimais é {:.2f}. Com 4 casas decimais, no entanto, o valor é {:.4f}'.format(pi, pi))
```

O valor de pi arredondado para 2 casas decimais é 3.14. Com 4 casas decimais, no entanto, o valor é 3.1416.

4.2.4 Introdução a expressões regulares

Por vezes queremos encontrar um padrão específico de texto (e.g., placas de carro, e-mails ou números de telefone) dentro de um texto maior, para realizar algum tipo de coleta, limpeza ou mesmo substituição que um simples `str.replace()` não dá conta. Para realizar tal ação podemos utilizar as famosas **Expressões Regulares**, também conhecidas como *Regular Expressions* no inglês, ou simplesmente *Regex*.

As expressões regulares são em essência uma potente linguagem para especificar padrões de texto. De forma mais detalhada, é uma composição dos chamados **metacaracteres**, caracteres com funções especiais, que, agrupados entre si e em conjunto com caracteres literais, formam

uma sequência, uma expressão. Essa expressão é interpretada como uma regra que indicará sucesso se uma entrada de dados qualquer casar com essa regra, ou seja, obedecer exatamente a todas as suas condições.

Imagine que você tenha o string abaixo, que mostra o texto de um trecho de uma notícia da CNN sobre o resultado da Pesquisa Datafolha para presidente divulgada em 18/08/2022 (link para a matéria completa [aqui](#)).

```
pesquisa = """
Pesquisa Datafolha divulgada nesta quinta-feira (18) mostra o ex-presidente Luiz Inácio Lula
com 47% das intenções de voto na corrida pelo Palácio do Planalto. O presidente Jair Bolsonaro
O primeiro turno das eleições acontece em 2 de outubro.

Na sequência, aparecem Ciro Gomes (PDT), com 7%; Simone Tebet (MDB), com 2%, e Vera Lúcia (P
"""

print(pesquisa)
```

Pesquisa Datafolha divulgada nesta quinta-feira (18) mostra o ex-presidente Luiz Inácio Lula com 47% das intenções de voto na corrida pelo Palácio do Planalto. O presidente Jair Bolsonaro O primeiro turno das eleições acontece em 2 de outubro.

Na sequência, aparecem Ciro Gomes (PDT), com 7%; Simone Tebet (MDB), com 2%, e Vera Lúcia (P

E se quiséssemos, por exemplo, substituir todas as porcentagens de intenção de voto por ‘ZZZ’? Note que as porcentagens são diferentes e não há uma repetição dos números que nos permita usar o `str.replace()` de uma vez só. No entanto, todas as porcentagens são representadas por 1 ou 2 números inteiros seguidos do símbolo %. Nesse caso, o mais indicado é utilizar as expressões regulares.

Os módulos e funções nativas do Python não nos trazem muito material para trabalhar com expressões regulares. Para operar com elas utilizaremos uma biblioteca de comandos chamada `re`. Falaremos mais sobre importação de bibliotecas de comandos e funções mais a frente, mas por hora tenha na cabeça que para utilizar um conjunto de instruções disponível em alguma biblioteca importada é preciso utilizar o nome da biblioteca seguido de ponto e do nome da função dessa biblioteca que você quer utilizar. No caso, para realizar a substituição das porcentagens no string *pesquisa* utilizaremos a função `sub()` de dentro da biblioteca `re`. Mas o que devemos colocar como input dessa função?

O mundo das expressões regulares é um mundo gigante e à parte, com conteúdo suficiente para preencher um outro curso. De forma geral, a combinação entre metacaracteres e caracteres literais é o que dá o padrão do texto pelo qual procuramos. Alguns dos metacaracteres-padrão

são . ? * + ^ | [] { } () \, cada um realizando uma função específica. Para o nosso caso utilizaremos basicamente a expressão regular dada por

```
[0-9]{1,2}%
```

Mas o que essa coisa bizarra diz de fato? A função buscará todo e qualquer elemento dentro dos colchetes (no caso os dígitos numéricos) que apareça uma ou duas vezes (código dentro dos colchetes) e que seja seguido pelo símbolo de porcentagem. Note que esse é o padrão de qualquer uma das porcentagens no nosso string *pesquisa*. Vamos ver o que acontece se usarmos isso dentro de `re.sub()`.

```
pesquisa2 = re.sub('[0-9]{1,2}%', 'ZZZ', pesquisa)
```

```
print(pesquisa)
```

Pesquisa Datafolha divulgada nesta quinta-feira (18) mostra o ex-presidente Luiz Inácio Lula com 47% das intenções de voto na corrida pelo Palácio do Planalto. O presidente Jair Bolsonaro venceu o primeiro turno das eleições acontece em 2 de outubro.

Na sequência, aparecem Ciro Gomes (PDT), com 7%; Simone Tebet (MDB), com 2%, e Vera Lúcia (PP)

```
print(pesquisa2)
```

Pesquisa Datafolha divulgada nesta quinta-feira (18) mostra o ex-presidente Luiz Inácio Lula com ZZZ das intenções de voto na corrida pelo Palácio do Planalto. O presidente Jair Bolsonaro venceu o primeiro turno das eleições acontece em 2 de outubro.

Na sequência, aparecem Ciro Gomes (PDT), com ZZZ; Simone Tebet (MDB), com ZZZ, e Vera Lúcia

Conseguimos exatamente o que a gente queria. Boa, time!

Pare um pouco e pense sobre a aplicabilidade dessa ferramenta dentro de um mundo repleto de dados não-estruturados, como tweets e notícias. **O potencial de uso é gigante!** Mas como dissemos, o mundo de *regex* é muito grande e existem cursos e livros que se dedicam integralmente a estudar essa linguagem de padrões textuais. Uma boa referência introdutória é o livro [Expressões Regulares: Uma Abordagem Divertida](#).

Se eu pudesse dar um conselho para o meu eu de 15 anos atrás seria: beba água e estude expressões regulares.

4.3 Listas

Como uma string, uma lista é uma sequência de valores. Em uma string, os valores são caracteres; em uma lista, eles podem ser de qualquer tipo. Podemos ter uma lista de strings, uma lista de valores numéricos ou mesmo uma lista de listas, combinando strings, números e mesmo outros tipos de objetos que ainda veremos, como tuplas, dicionários e dataframes.

Uma lista é delimitada por colchetes e os elementos, ou itens, pertencentes a ela são separados por vírgula. Para definir uma lista com 5 números inteiros, ordenados de forma sequencial e começando em 1 devemos escrever a seguinte linha de código:

```
lista1 = [1,2,3,4,5]

print(lista1)
print(type(lista1))
```

```
[1, 2, 3, 4, 5]
<class 'list'>
```

Podemos definir uma lista de strings, uma lista mista de strings e números, e uma lista composto por outras listas (lista aninhada):

```
lista2 = ['Danilo Souza', 'Claudio Lucinda']
lista3 = ['Turma 2024201', 46.0, 'Turmas 2024202', 49, 'Turmas 2024221', 81]
lista4 = [lista1, lista2]

print(lista2)
print(lista3)
print(lista4)
```

```
['Danilo Souza', 'Claudio Lucinda']
['Turma 2024201', 46.0, 'Turmas 2024202', 49, 'Turmas 2024221', 81]
[[1, 2, 3, 4, 5], ['Danilo Souza', 'Claudio Lucinda']]
```

De forma análoga à listas com elementos não vazios, é possível definir uma lista vazia utilizando apenas os colchetes:

```
lista_vazia = []

print(lista_vazia)
```

```
[]
```

4.3.1 Operações básicas

A sintaxe para acessar os elementos de uma lista é a mesma que para acessar os caracteres de uma string: o operador de colchete. A expressão dentro dos colchetes especifica o índice ou o intervalo de índices. **Lembrando que o índice no Python começa em ZERO e não em UM**

```
print(lista2[0])
print(lista2[1])
print(lista3[-1])
print(lista3[0:2])
```

```
Danilo Souza
Claudio Lucinda
81
['Turma 2024201', 46.0]
```

Diferente das strings, listas são mutáveis. Quando o operador de colchete aparece do lado esquerdo de uma atribuição, ele identifica o elemento da lista que será atribuído:

```
numbers = [42, 123]
numbers[1] = 5
numbers
```

```
[42, 5]
```

O segundo elemento de `numbers` (índice 1), que era 123, agora é 5.

Índices de listas funcionam da mesma forma que os índices de strings:

- Qualquer expressão de números inteiros pode ser usada como índice.
- Se tentar ler ou escrever um elemento que não existe, você recebe um `IndexError`.
- Se um índice tiver um valor negativo, ele conta de trás para a frente, a partir do final da lista.

O operador `in`, que serve ao propósito de testar a existência de determinado elemento dentro de um objeto específico, também funciona com listas:

```
cheeses = ['Cheddar', 'Gorgonzola', 'Gouda']
'Edam' in cheeses
```

False

```
'Brie' in cheeses
```

False

Assim como com strings, é possível calcular o número de elementos em uma lista utilizando a função integrada `len()` e “concatenar” ou “somar” listas utilizando apenas o sinal de +

```
print(len(cheeses))
```

3

```
lista_soma = lista2 + cheeses  
print(lista_soma)
```

```
['Danilo Souza', 'Claudio Lucinda', 'Cheddar', 'Gorgonzola', 'Gouda']
```

4.3.2 Métodos de listas

As listas também possuem métodos bastante úteis, que nos facilitam a vida em várias dimensões. Dentre os principais métodos aplicáveis a listas e suas funcionalidades podemos citar:

1. **lista_example.append()**: adiciona um novo elemento ao fim da lista “lista_example”.
2. **lista_example.extend(lista2)**: toma a lista “lista_example” como argumento e adiciona todos os elementos de lista2 como novos elementos da lista inicial.
3. **lista_example.sort()**: classifica os elementos de “lista_example” em ordem ascendente.
4. **lista_example.remove(x)**: exclui o elemento igual a “x” de “lista_example”. É um método bastante útil quando queremos excluir um elemento específico, mas não sabemos sua posição dentro da lista.

A maior parte dos métodos de listas são nulos; eles alteram a lista e retornam None. Se você escrever `t = t.sort()` por acidente, ficará desapontado com o resultado.

```
t = ['a', 'b', 'c']  
t.append('d')  
print(t)
```

```
['a', 'b', 'c', 'd']
```

```
t1 = ['a', 'b', 'c']  
t2 = ['d', 'e']  
t1.extend(t2)  
print(t1)
```

```
['a', 'b', 'c', 'd', 'e']
```

```
t = ['d', 'c', 'e', 'b', 'a']  
t.sort()  
print(t)
```

```
['a', 'b', 'c', 'd', 'e']
```

Assim como já explicamos, uma lista é uma sequência de valores e uma string é uma sequência de caracteres, mas uma lista de caracteres não é a mesma coisa que uma string. Para converter uma string em uma lista de caracteres, você pode usar o comando `list`:

```
s = 'spam'  
t = list(s)  
print(t)
```

```
['s', 'p', 'a', 'm']
```

A função `list` quebra uma string em letras individuais. Se você quiser quebrar uma string em palavras, você pode usar o método `split()`. Esse método admite um argumento adicional, chamado *delimiter*, que especifica quais caracteres podem ser usados para demonstrar os limites das palavras. Isso é muito útil, por exemplo, quando queremos separar um texto em palavras e fazer a contagem de palavras que mais se repetem. Qual caracter deveríamos passar como argumento em `split()` nesse caso?

```
s = 'pining for the fjords'  
t = s.split(' ')  
print(t)
```

```
['pining', 'for', 'the', 'fjords']
```

```
s = 'spam-spam-spam'
t = s.split('-')
t
```

```
['spam', 'spam', 'spam']
```

O método `join()` é o contrário de `split()`. Ele toma uma lista de strings e concatena os elementos. `join()`, no entanto, é um método de string, então é preciso invocá-lo no string delimitador (por exemplo, " - ") e passar a lista de strings como parâmetro:

```
t = ['spam', 'spam', 'spam']
s = ' - '.join(t)
print(s)
```

```
spam - spam - spam
```

Além desses principais, existem outros vários métodos para listas. Esse [link](#) é um bom ponto de partida para quem quiser conhecer outros exemplos.

4.3.3 Operadores lógicos e variáveis booleanas

Agora faremos um pequeno desvio, que será bem útil daqui para frente, para falar de expressões e variáveis **booleanas**. Em resumo, uma expressão booleana é uma expressão que pode ser verdadeira ou falsa e dessa forma assumir apenas dois valores como resultado: **True** e **False**. Os exemplos seguintes usam o operador de igual no Python (`==`) que compara dois operandos e produz `True` se forem iguais e `False` se não forem:

```
5 == 5
```

```
True
```

```
5 == 6
```

```
False
```

```
lista1 = [1,2,3,4,5]
lista2 = [5,4,3,2,1]

lista1 == lista2
```

False

True e *False* são valores especiais que pertencem ao tipo *bool*; não são strings. Além disso, é possível fazer operações aritméticas com variáveis do tipo *bool* já que o Python entende *True* como sendo equivalente ao número 1 e *False* como sendo equivalente ao número 0.

```
print(type(True))
print(type(False))
```

```
<class 'bool'>
<class 'bool'>
```

```
print(True + True)
print(True + False)
print(False + False)
```

```
2
1
0
```

O operador `==` é um dos operadores relacionais dentro do Python, os outros são:

1. `x != y`: x não é igual a y
2. `x > y`: x é maior que y
3. `x < y`: x é menor que y
4. `x >= y`: x é maior ou igual a y
5. `x <= y`: x é menor ou igual a y

Embora essas operações provavelmente sejam familiares para você, os símbolos do Python são diferentes dos símbolos matemáticos. Um erro comum é usar apenas um sinal de igual (`=`) em vez de um sinal duplo (`==`). Lembre-se de que `=` é um operador de atribuição e `==` é um operador relacional. Não existe `=>` ou `=<`.

Há três operadores lógicos: `and`, `or` e `not`. A semântica (significado) destes operadores é semelhante ao seu significado em inglês. Por exemplo, `x>=0 and x<=10` só é verdade se `x` for maior que 0 e menor que 10. `n%2 == 0 or n%3 == 0` é verdadeiro se uma ou as duas condição(ões) for(em) verdadeira(s), isto é, se o número `n` for divisível por 2 ou 3 (caso você não esteja familiarizado com a divisão pelo piso e o operador módulo, tente brincar um pouco com `//` e com `%`). Finalmente, o operador `not` nega uma expressão booleana, então `not (x > y)` é verdade se `x > y` for falso, isto é, se `x` for menor que ou igual a `y`.

Falando estritamente, os operandos dos operadores lógicos devem ser expressões booleanas, mas o Python não é muito estrito. Qualquer número que não seja zero é interpretado como *True*:

```
42 and True
```

```
True
```

Esta flexibilidade tem sua utilidade, mas há algumas sutilezas relativas a ela que podem ser confusas. Assim, pode ser uma boa ideia evitá-la (a menos que você tenha certeza absoluta do que está fazendo).

4.3.4 Execução condicional

Para escrever programas úteis, quase sempre precisamos da capacidade de verificar condições e mudar o comportamento do programa de acordo com elas. Instruções condicionais nos dão esta capacidade. A forma mais simples é a instrução `if`:

```
x=5

if x > 0:
    print('x é positivo')
```

```
x é positivo
```

A expressão booleana depois do `if` é chamada de condição. Se for verdadeira, a instrução indentada é executada. Se não, nada acontece. *Aqui vale mais um adendo*: uma característica muito importante da sintaxe do Python é justamente a **identação**. Diferentemente de outras linguagens de programação, a indentação exerce papel importante aqui, já que é através dela que se determina onde se inicia e onde termina um bloco de código, que pode ser uma expressão condicional ou mesmo uma função, como veremos mais a frente. Além de economizar várias chaves (“{” e “}”) e vários “end”, a indentação exerce um papel estético importante ao permitir uma melhor visualização do código como um todo.

Não há limite para o número de instruções que podem aparecer no corpo de uma instrução `if`, mas deve haver pelo menos uma. Ocasionalmente, é útil ter um corpo sem instruções (normalmente como um espaço reservado para código que ainda não foi escrito). Neste caso, você pode usar a instrução `pass`, que não faz nada.

```
if x < 0:
    pass
```

4.3.5 Execução alternativa

Uma segunda forma da instrução if é a “execução alternativa”, na qual há duas possibilidades e a condição determina qual será executada. A sintaxe pode ser algo assim:

```
x = 5

if x % 2 == 0:
    print('x é par')
else:
    print('x é ímpar')
```

x é ímpar

Se o resto quando x for dividido por 2 for 0, então sabemos que x é par e o programa exibe uma mensagem adequada. Se a condição for falsa, o segundo conjunto de instruções é executado. Como a condição deve ser verdadeira ou falsa, exatamente uma das alternativas será executada. As alternativas são chamadas de ramos (branches), porque são ramos no fluxo da execução.

Podemos usar a instrução if para testar o tamanho de uma lista também:

```
lista1 = [1,2,3,4,5]

if len(lista1) > 2:
    print('lista1 tem mais de dois elementos')
else:
    print('lista1 tem menos de dois elementos')
```

lista1 tem mais de dois elementos

4.3.6 Condicionais encadeadas

Às vezes, há mais de duas possibilidades e precisamos de mais que dois ramos. Esta forma de expressar uma operação de computação é uma condicional encadeada:

```
x = 5
y = 6

if x < y:
    print('x é menor do que y')
elif x > y:
    print('x é maior do que y')
else:
    print('x e y são iguais')
```

x é menor do que y

`elif` é uma abreviatura de “else if”. Novamente, exatamente um ramo será executado. Não há nenhum limite para o número de instruções `elif`. Se houver uma cláusula `else`, ela deve estar no fim, mas não é preciso haver uma. Cada condição é verificada em ordem. Se a primeira for falsa, a próxima é verificada, e assim por diante. Se uma delas for verdadeira, o ramo correspondente é executado e a instrução é encerrada. Mesmo se mais de uma condição for verdade, só o primeiro ramo verdadeiro é executado.

4.4 Tuplas

Agora falaremos de mais um tipo de objeto básico do Python, a tupla. Uma tupla é uma sequência de valores. Os valores podem ser de qualquer tipo, e podem ser indexados por números inteiros, portanto, nesse sentido, as tuplas são muito parecidas com as listas. A diferença importante é que as tuplas são **imutáveis**, assim como os strings.

Em resumo, uma tupla é uma lista de valores separados por vírgulas e em geral delimitado por parênteses (lembre que listas são delimitadas por colchetes):

```
t = ('a', 'b', 'c', 'd', 'e')

print(type(t))
```

<class 'tuple'>

Um único valor entre parênteses não é uma tupla. Para criar uma tupla com um único elemento, é preciso incluir uma vírgula final após o elemento, com ou sem os parênteses.

```
t1 = ('a')
t2 = 'a',
t3 = ('a',)

print(type(t1))
print(type(t2))
print(type(t3))
```

```
<class 'str'>
<class 'tuple'>
<class 'tuple'>
```

Outra forma de criar uma tupla é com a função integrada `tuple`. Sem argumentos, cria uma tupla vazia. Se os argumentos forem uma sequência (string, lista ou tupla), o resultado é uma tupla com os elementos da sequência:

```
t = tuple()
print(t)
```

```
()
```

```
t = tuple('lupins')
print(t)
```

```
('l', 'u', 'p', 'i', 'n', 's')
```

4.4.1 Operações básicas

A maior parte dos operadores de lista também funciona em tuplas. O operador de colchetes indexa um elemento e o operador de fatia seleciona vários elementos.

```
t = ('a', 'b', 'c', 'd', 'e')
print(t[0])
print(t[1:3])
```

```
a
('b', 'c')
```

Entretanto, se tentar alterar um dos elementos da tupla, vai receber um erro:

```
t[0] = 'A'
```

```
TypeError: 'tuple' object does not support item assignment
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[58], line 1  
----> 1 t[0] = 'A'  
TypeError: 'tuple' object does not support item assignment
```

Como tuplas são imutáveis, você não pode alterar os elementos, mas pode substituir uma tupla por outra.

```
t = ('A',) + t[1:]  
print(t)
```

```
('A', 'b', 'c', 'd', 'e')
```

Os operadores relacionais funcionam com tuplas e outras sequências. O Python começa comparando o primeiro elemento de cada sequência. Se forem iguais, vai para os próximos elementos, e assim por diante, até que encontre elementos que sejam diferentes. Os elementos subsequentes não são considerados (mesmo se forem muito grandes).

```
(0, 1, 2) < (0, 3, 4)
```

```
True
```

```
(0, 1, 2000000) < (0, 3, 4)
```

```
True
```

4.4.2 Atribuição de tuplas

Muitas vezes, é útil trocar os valores de duas variáveis. Com a atribuição convencional, é preciso usar uma variável temporária. Por exemplo, trocar a e b.

```
a=5
b=6

temp = a
a = b
b = temp

print(b)
```

5

Essa solução é trabalhosa; a atribuição de tuplas é mais elegante:

```
a=5
b=6

a, b = b, a

print(b)
```

5

O lado esquerdo é uma tupla de variáveis e o lado direito é uma tupla de expressões. Cada valor é atribuído à sua respectiva variável. Todas as expressões no lado direito são avaliadas antes de todas as atribuições.

O número de variáveis à esquerda e o número de valores à direita precisam ser iguais:

```
a, b = 1, 2, 3
```

ValueError: too many values to unpack (expected 2)

```
-----
ValueError                                Traceback (most recent call last)
Cell In[64], line 1
----> 1 a, b = 1, 2, 3
ValueError: too many values to unpack (expected 2)
```

De forma geral, o lado direito pode ter qualquer tipo de sequência (string, lista ou tupla). Por exemplo, para dividir um endereço de email em um nome de usuário e um domínio, você poderia escrever:

```
addr = 'monty@python.org'
uname, domain = addr.split('@')
```

O valor de retorno do `split()` é uma lista com dois elementos; o primeiro elemento é atribuído a `uname`, o segundo a `domain`:

```
print(uname)
print(domain)
```

```
monty
python.org
```

4.4.3 Tuplas como valores de retorno

Falando estritamente, uma função só pode retornar um valor, mas se o valor for uma tupla, o efeito é o mesmo que retornar valores múltiplos. Por exemplo, se você quiser dividir dois números inteiros e calcular o quociente e resto, não é eficiente calcular `x/y` e depois `x%y`. É melhor calcular ambos ao mesmo tempo. A função integrada `divmod` toma dois argumentos e devolve uma tupla de dois valores: o quociente e o resto da divisão do primeiro termo pelo segundo termo. Você pode guardar o resultado como uma tupla:

```
print(7//3)
print(7%3)
```

```
2
1
```

```
t = divmod(7, 3)

print(t)
print(type(t))
```

```
(2, 1)
<class 'tuple'>
```

4.4.4 Operações nativas com tuplas

`zip` é uma função integrada que recebe duas ou mais sequências e devolve uma lista de tuplas onde cada tupla contém um elemento de cada sequência. O nome da função tem a ver com o zíper, que se junta e encaixa duas carreiras de dentes.

Este exemplo encaixa uma string e uma lista:

```
s = 'abc'
t = [0, 1, 2]
zip(s,t)
```

```
<zip at 0x1d55eb65d80>
```

O resultado é um objeto `zip` que sabe como percorrer os pares. O uso mais comum de `zip` é em um loop `for` (falaremos sobre loops mais adiante no curso):

```
for pair in zip(s, t):
    print(pair)
```

```
('a', 0)
('b', 1)
('c', 2)
```

Um objeto `zip` é um tipo de iterador, ou seja, qualquer objeto que percorre ou itera sobre uma sequência. Iteradores são semelhantes a listas em alguns aspectos, mas, ao contrário de listas, não é possível usar um índice para selecionar um elemento de um iterador. Se quiser usar operadores e métodos de lista, você pode usar um objeto `zip` para fazer uma lista:

```
list(zip(s, t))
```

```
[('a', 0), ('b', 1), ('c', 2)]
```

O resultado é uma lista de tuplas. Neste exemplo, cada tupla contém um caractere da string e o elemento correspondente da lista. Se as sequências não forem do mesmo comprimento, o resultado tem o comprimento da mais curta:

```
list(zip('Anne', 'Elk'))
```

```
[('A', 'E'), ('n', 'l'), ('n', 'k')]
```

4.5 Dicionários

Dicionários são um outro tipo de objeto bastante importante e útil em nossas aplicações. Dicionários são um dos melhores recursos do Python, eles são os blocos de montar de muitos algoritmos eficientes e elegantes.

Um dicionário se parece com uma lista, mas é mais geral. Em uma lista os índices têm que ser números inteiros, em um dicionário eles podem ser de (quase) qualquer tipo. Um dicionário contém uma coleção de índices, que se chamam **chaves** e uma coleção de valores. **Cada chave é associada com um único valor.** A associação de uma chave e um valor chama-se par chave-valor ou item.

Em linguagem matemática, um dicionário representa um mapeamento de chaves a valores, para que você possa dizer que cada chave “mostra o mapa” a um valor. Como exemplo, vamos construir um dicionário que faz o mapa de palavras do inglês ao espanhol, portanto as chaves e os valores são todos strings.

A função `dict` cria um novo dicionário sem itens. Como `dict` é o nome de uma função integrada, você deve evitar usá-lo como nome de variável.

```
eng2sp = dict()
eng2sp
```

```
{}
```

As chaves `{}` representam um dicionário vazio. Para acrescentar itens ao dicionário, você pode usar colchetes:

```
eng2sp['one'] = 'uno'
```

Esta linha cria um item que mapeia da chave ‘one’ ao valor ‘uno’. Se imprimirmos o dicionário novamente, vemos um par chave-valor com dois pontos entre a chave e o valor:

```
eng2sp
```

```
{'one': 'uno'}
```

Este formato de saída também é um formato de entrada. Por exemplo, você pode criar um dicionário com três itens:

```
eng2sp = {'one': 'uno', 'two': 'dos', 'three': 'tres'}  
eng2sp
```

```
{'one': 'uno', 'two': 'dos', 'three': 'tres'}
```

Ué, mas a ordem parece diferente daquela que definimos, não? Se você digitar o mesmo exemplo no seu computador, pode receber um resultado diferente. Em geral, a ordem dos itens em um dicionário é imprevisível. No entanto, isso não é um problema porque os elementos de um dicionário nunca são indexados com índices de números inteiros. Em vez disso, você usa as chaves para procurar os valores correspondentes. A chave 'two', por exemplo sempre mapeia ao valor 'dos', assim a ordem dos itens não importa.

```
eng2sp['two']
```

```
'dos'
```

Se a chave não estiver no dicionário, você recebe uma exceção:

```
eng2sp['four']
```

```
KeyError: 'four'
```

```
-----  
KeyError                                Traceback (most recent call last)  
Cell In[78], line 1  
----> 1 eng2sp['four']  
KeyError: 'four'
```

Assim como em strings e listas, a função `len` também é compatível com dicionários. Nesse caso ela devolve o número de pares chave-valor.

```
len(eng2sp)
```

```
3
```

O operador `in` funciona em dicionários também. Ele acusa se algo aparece como chave no dicionário (aparecer como valor não é o suficiente).

```
'one' in eng2sp
```

```
True
```

```
'uno' in eng2sp
```

False

Para ver se algo aparece como um valor em um dicionário, você pode usar o método `values()`, que devolve uma coleção de valores, e então usar o operador `in`:

```
vals = eng2sp.values()  
'uno' in vals
```

True

4.5.1 Operações básicas

Podemos adicionar um novo par de chave e valor ao dicionário usando colchetes, mas caso a chave já exista no dicionário, podemos atualizar o valor.

```
y = {}  
y['one'] = 1  
y['two'] = 2  
print(y)
```

```
{'one': 1, 'two': 2}
```

```
y['two'] = 'dos'  
print(y)
```

```
{'one': 1, 'two': 'dos'}
```

O valor-chave deve ser um tipo de dados imutável, por essa razão, se você tentar definir um dicionário com um tipo de dados mutável, o Python levantará um erro de exceção.

A instrução `del` pode ser usada para remover uma entrada (par de valor de chave) de um dicionário.

```
y = {'one': 1, 'two': 2}  
print(y)
```

```
{'one': 1, 'two': 2}
```

```
del y['two']
print(y)
```

```
{'one': 1}
```

Mais uma vez, podemos utilizar a função nativa `len` para retornar o número de entradas (pares de valor de chave) em um dicionário.

```
x = {'one': 0, 'two': 2}
print(len(x))
```

```
2
```

Tente acessar uma chave que não está em um dicionário e você receberá um erro de exceção do Python. Para lidar com essa exceção, você pode usar mais uma vez o operador `in` que testa se existe uma chave em um dicionário. Esse operador retorna *True* se o dicionário tiver um valor armazenado sob a chave fornecida e *Falso* caso contrário.

```
y = {'one': 1, 'two': 2}
del y['three']
```

```
KeyError: 'three'
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[88], line 2
      1 y = {'one': 1, 'two': 2}
----> 2 del y['three']
KeyError: 'three'
```

```
'three' in y
```

```
False
```

4.5.2 Métodos de dicionários

Dentre os principais métodos aplicáveis a dicionários e suas funcionalidades podemos citar:

1. **dict_example.update(x)**: atualiza o dicionário “dict_example” com todos os pares de valor-chave de um segundo dicionário “x”. Os valores de chaves, que são comuns a ambos os dicionários, do segundo dicionário irão se sobrepor aos valores das chaves do primeiro dicionário.
2. **dict_example.keys(x)**: permite que você obtenha todas as chaves no dicionário. Muitas vezes é usado dentro de uma série de instruções repetida várias vezes para iterar sobre o conteúdo de um dicionário.
3. **dict_example.items()**: retorna todas as chaves do dicionário “dict_example” e seus valores associados como uma sequência de tuplas.
4. **dict_example.get(x,y)**: devolve o valor associado a uma chave “x” se o dicionário contiver essa chave. Caso o dicionário não contenha a chave, você pode especificar um segundo argumento opcional “y” para retornar um valor padrão (se o argumento não estiver incluído o método retornará *None*).
5. **dict_example.setdefault(x,y)**: é semelhante ao método **get()**: ele retorna o valor associado a uma chave “x” se o dicionário contiver essa chave, mas caso o dicionário não contenha a chave, este método criará um novo elemento no dicionário (par de valor de chave), onde o primeiro argumento neste método é a chave, e o segundo argumento (“y”) é o valor. O segundo argumento é opcional, mas se isso não for incluído, o valor atrelado a essa nova chave será *None*.

Além desses principais, existem outros vários métodos para dicionários. Esse [link](#) é um bom ponto de partida para quem quiser conhecer outros exemplos.

```
x = {'one': 0, 'two': 2}
y = {'one': 1, 'three': 3}
print(x)

x.update(y)
print(x)
```

```
{'one': 0, 'two': 2}
{'one': 1, 'two': 2, 'three': 3}
```

```
x = {'one': 1, 'two': 2}
print(x.keys())
```

```
dict_keys(['one', 'two'])
```

```
x = {'one': 1, 'two': 2}
print(x.items())
```

```
dict_items([('one', 1), ('two', 2)])
```

```
y = {'one': 1, 'two': 2}
print(y.get('one'))
print(y.get('three'))
print(y.get('three', 'The key does not exist.'))
```

```
1
None
The key does not exist.
```

```
y = {'one': 1, 'two': 2}
print(y.setdefault('three', '3'))
print(y.setdefault('two', 'dos'))
print(y)
```

```
3
2
{'one': 1, 'two': 2, 'three': '3'}
```

```
print(y.setdefault('four'))
print(y)
```

```
None
{'one': 1, 'two': 2, 'three': '3', 'four': None}
```

4.6 Exercícios

1. Considere a seguinte lista: `fruit = ['pear', 'orange', 'apple', 'grapefruit', 'apple', 'pear']`. Use uma função de lista para dizer o índice da primeira ocorrência de `apple`
2. Usando a lista do exercício anterior, use uma função de lista para retornar o número de vezes que `apple` ocorre.

3. Existem duas listas abaixo. Escreva um programa que as converta em um dicionário em que o item de **keys** é a chave e o item de **values** é o valor

```
keys = ['Ten', 'Twenty', 'Thirty']  
values = [10, 20, 30]  
  
### Output esperado  
{'Ten': 10, 'Twenty': 20, 'Thirty': 30}
```

5 Controle de fluxo e iteração

5.1 Introdução

5.1.1 Pré-requisitos

Embora as funções e estruturas necessárias para a realização de *loops* no Python sejam nativas na linguagem, para algumas das aplicações utilizaremos um dos pacotes externos que nos permitem trabalhar com números aleatórios.

- numpy

```
import numpy as np
```

5.2 Repetição simples e a instrução *for*

Os computadores muitas vezes são usados para automatizar tarefas repetitivas. A repetição de tarefas idênticas ou semelhantes sem cometer erros é algo que os computadores fazem muito bem e as pessoas não. Em um programa de computador, a repetição também é chamada de iteração.

De maneira um pouco mais formal, iteração significa executar o mesmo bloco de código repetidamente, potencialmente muitas vezes. Uma estrutura de programação que implementa a iteração é chamada de *loop*. A forma mais simples de iteração é a chamada *iteração definida*, em que o número de vezes que o bloco designado será executado é especificado explicitamente no momento em que o *loop* é iniciado.

Para atingir o objetivo proposto pela *iteração definida* utilizaremos no Python a instrução **for**. Um loop **for** tem duas partes: um cabeçalho especificando a iteração, que termina em dois pontos, e um corpo **identado** que é executado uma vez por iteração. O corpo pode conter qualquer número de instruções, mas o Python só reconhecerá como parte do corpo o código que estiver identado em relação ao cabeçalho.

No Python o **for** tem a cara abaixo

```
for <elemento> in <objeto>:  
    <instruções>
```

Note que <objeto> em geral se refere à uma sequência, seja ela uma lista, uma tupla, um string ou qualquer outro objeto pelo qual seja possível percorrer. Para cada <elemento> dentro da sequência <objeto> o conjunto de instruções em <instruções> será executado. Após percorrer todos os elementos pertencentes à sequência <objeto>, o Python passa a execução do *loop*.

Imagine agora que estejamos interessados em criar um `for` loop que printa 10 números aleatórios sorteados de uma distribuição uniforme que vai de 0 a 1. Utilizando o objeto `range()` e a lógica de listas:

```
print(range(0,10))
```

```
range(0, 10)
```

```
print(list(range(0,10)))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
for i in list(range(0,10)):  
    u = np.random.uniform()  
    print(u)
```

```
0.08012596003685812  
0.8370154377503121  
0.12453779187343217  
0.5460742264238596  
0.302570527251779  
0.0128901448416181  
0.12141008948709997  
0.48242897095710413  
0.7315825528467169  
0.8916534297537082
```

No *loop* acima não utilizamos o elemento `i` em nenhum momento, de modo que a lista nos serviu apenas para ditar o número de vezes que gostaríamos de rodar o bloco de instruções abaixo do cabeçalho, no caso 10. Podemos ir um pouco além.

```
for i in list(range(0,10)):

    u = np.random.uniform()
    print('Essa é a iteração '+str(i+1)+' e o número sorteado foi: {:.2f}'.format(u))
```

```
Essa é a iteração 1 e o número sorteado foi: 0.40
Essa é a iteração 2 e o número sorteado foi: 0.87
Essa é a iteração 3 e o número sorteado foi: 0.44
Essa é a iteração 4 e o número sorteado foi: 0.66
Essa é a iteração 5 e o número sorteado foi: 0.79
Essa é a iteração 6 e o número sorteado foi: 0.94
Essa é a iteração 7 e o número sorteado foi: 0.56
Essa é a iteração 8 e o número sorteado foi: 0.80
Essa é a iteração 9 e o número sorteado foi: 0.32
Essa é a iteração 10 e o número sorteado foi: 0.05
```

Note que nesse caso utilizamos tanto o elemento *iterável* *i*, quanto o resultado de cada sorteio individual *u*.

Como dito anteriormente, o loop *for* funciona com qualquer sequência ou objeto pelo qual seja possível percorrer. Podemos trabalhar, inclusive, com strings e listas como nos exemplos abaixo:

```
str1 = 'Vou nadaaaaa'

for c in str1:
    print(c)
```

```
V
o
u

n
a
d
a
a
a
a
a
```

```
lista1 = ['Minha terra tem palmeiras', 'onde canta o sabiá', 'seno A cosseno B', 'seno B cosseno A']

for f in lista1:
    print(f)
```

```
Minha terra tem palmeiras
onde canta o sabiá
seno A cosseno B
seno B cosseno A
```

5.2.1 *for* loop e instruções condicionais

Podemos utilizar instruções condicionais dentro de um loop `for` também. Mais do que isso, é possível utilizar esse tipo de instrução para interromper a repetição do código, mesmo antes do Python percorrer todos os elementos da sequência alvo.

```
for i in ['foo', 'bar', 'baz', 'qux']:
    if 'b' in i:
        break
    print(i)
```

```
foo
```

Note que no caso acima, o segundo elemento da lista `['foo', 'bar', 'baz', 'qux']` contém a letra `'b'` mas o primeiro não. A instrução `if`, portanto, retorna `False` na primeira iteração e `True` na segunda. No momento em que ela retorna `True` o código indentado `break` é executado e o loop para aí! Dessa forma, o único termo que a instrução `print` vai imprimir será o primeiro da lista, isto é, `'foo'`.

5.2.2 Usando *enumerate* e *zip*

Às vezes pode ser útil, dentro de um loop `for`, ter uma variável que muda em cada iteração do loop que possibilite fazer um controle mais direto sobre qual iteração está sendo executada em determinado momento. Em vez de criar e incrementar uma variável você mesmo, você pode usar a função nativa `enumerate()` para obter um contador e o valor do objeto iterável ao mesmo tempo! Para deixar claro como funciona a sintaxe nesse caso, vamos trabalhar em cima de uma lista de strings.

```

str1 = '0 Tata é foota, o Tata é foota'

# Algumas linhas de código adicionais para excluir pontuação e transformar o string em uma l
str1 = str1.replace('.', '')
str1 = str1.replace(',', '')
str1 = str1.replace('?', '')
lista1 = str1.split()

for c in lista1:
    print(c)

```

```

0
Tata
é
foota
o
Tata
é
foota

```

Utilizando o `enumerate()` podemos, em uma mesma linha de código, printar não apenas cada string individual da lista, mas também a posição desse string dentro da lista!

```

for count,value in enumerate(lista1):

    print('Elemento '+str(count)+' = '+value)

```

```

Elemento 0 = 0
Elemento 1 = Tata
Elemento 2 = é
Elemento 3 = foota
Elemento 4 = o
Elemento 5 = Tata
Elemento 6 = é
Elemento 7 = foota

```

Uma outra forma de criar esses mesmos resultados seria utilizando a função nativa `zip()`. Essa função nos permite iterar ao longo de duas ou mais sequências ao mesmo tempo, contanto que as sequências possuam o mesmo comprimento.

```
print(lista1)
```

```
['0', 'Tata', 'é', 'foota', 'o', 'Tata', 'é', 'foota']
```

```
lista2 = list(range(0,len(lista1)))  
print(lista2)
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
len(lista1) == len(lista2)
```

```
True
```

```
for count,value in zip(lista2, lista1):  
    print('Elemento '+str(count)+' = '+value)
```

```
Elemento 0 = 0  
Elemento 1 = Tata  
Elemento 2 = é  
Elemento 3 = foota  
Elemento 4 = o  
Elemento 5 = Tata  
Elemento 6 = é  
Elemento 7 = foota
```

Embora tenhamos utilizado a função `zip()` para chegar ao mesmo resultado de `enumerate()`, seu escopo de ação é muito mais geral já que nos permite percorrer dois objetos distintos ao mesmo tempo, sejam quais forem esses objetos. A única obrigatoriedade é que esses objetos tenham o mesmo tamanho, nada mais!

```
professores = ['Danilo Souza','Danilo Souza','Claudio Lucinda']  
turmas = ['2024201','2024202','2024221']  
  
for p,t in zip(professores,turmas):  
    print('Nesse curso, a turma '+t+' é de responsabilidade do professor '+p)
```

Nesse curso, a turma 2024201 é de responsabilidade do professor Danilo Souza
Nesse curso, a turma 2024202 é de responsabilidade do professor Danilo Souza
Nesse curso, a turma 2024221 é de responsabilidade do professor Claudio Lucinda

5.2.3 For e list comprehensions

List comprehensions é uma das várias formas que o Python nos oferece para de criar, alterar e manipular listas. Sua sintaxe é bastante concisa e nos permite olhar para um exercício de iteração através de algo parecido com uma fórmula.

Imagine que estejamos interessados em criar uma lista que contenha a parte inteira da raiz quadrada de todo número de uma outra lista original. Usualmente faríamos isso utilizando um loop for:

```
lista1 = [1,4,9,16,25,36,49,64,81,100,121,144,169,196,225]
lista2 = []

for x in lista1:
    lista2.append(int(x**0.5))

print(lista2)
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

Alternativamente, podemos utilizar *list comprehensions* através da seguinte sintaxe:

```
lista2 = [int(x**0.5) for x in lista1]

print(lista2)
```

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]

Em termos práticos isso funciona como se fosse um loop definido dentro de uma única linha de código. Mas porque aprender as duas formas? Será que alguma delas é mais eficiente? Vamos fazer um teste para esse exercício da raiz quadrada, porém para uma lista maior. Utilizaremos também a função `time` do pacote `time` para calcular o tempo utilizado por cada um dos métodos.

```
import time
lista1 = list(range(1,1000000))

# list comprehension
start_comp = time.time()

list_comp = [int(x**0.5) for x in lista1]
```

```

end_comp    = time.time()

# loop
start_loop = time.time()

list_loop = []
for x in lista1:
    list_loop.append(int(x**0.5))

end_loop    = time.time()

ratio = (start_loop - end_loop) / (start_comp - end_comp)

```

Qual abordagem será que levou menos tempo?

```

print('Tempo necessário para a realização dos cálculos utilizando list comprehensions: {:.4f}'.format(ratio))
print('Tempo necessário para a realização dos cálculos utilizando loop: {:.4f} segundos'.format(end_loop - start_loop))
print('\nA abordagem de loop demorou {:.1%} mais tempo! Dê uma chance para list comprehensions ;)')

```

Tempo necessário para a realização dos cálculos utilizando list comprehensions: 0.0921 segundos

Tempo necessário para a realização dos cálculos utilizando loop: 0.1264 segundos

A abordagem de loop demorou 37.3% mais tempo! Dê uma chance para list comprehensions ;)

5.3 Reatribuição e atualização de variáveis

Pode ser que você já tenha descoberto que é permitido fazer mais de uma atribuição para a mesma variável. Uma nova atribuição faz uma variável existente referir-se a um novo valor (e deixar de referir-se ao valor anterior).

```

x = 5
print(x)

```

5

```

x = 7
print(7)

```

A primeira vez que exibimos `x`, seu valor é 5. Na segunda vez, seu valor é 7.

Neste ponto quero tratar de uma fonte comum de confusão. Como o Python usa o sinal de igual (`=`) para atribuição, é tentador interpretar uma afirmação como `a = b` como uma proposição matemática de igualdade, isto é, a declaração de que `a` e `b` são iguais. Mas esta é uma interpretação equivocada.

Em primeiro lugar, a igualdade é uma relação simétrica e a atribuição não é. Por exemplo, na matemática, se `a=7` então `7=a`. Mas no Python, a instrução `a = 7` é legal e `7 = a` não é. Além disso, na matemática, uma proposição de igualdade é verdadeira ou falsa para sempre. Se `a=b` agora, então `a` sempre será igual a `b`. No Python, uma instrução de atribuição pode tornar duas variáveis iguais, mas elas não precisam se manter assim:

```
a = 5
b = a # a e b agora são iguais

print('Valor de a: '+str(a)+'\nValor de b: '+str(b))

a = 3 # a e b não são mais iguais

print('\nValor de a: '+str(a)+'\nValor de b: '+str(b))
```

Valor de a: 5

Valor de b: 5

Valor de a: 3

Valor de b: 5

Quando redefinimos o valor de `a` para 3, não mudamos o valor de `b`, de modo que as variáveis deixam de ser iguais. A reatribuição de variáveis muitas vezes é útil, mas você deve usá-la com prudência. Se os valores das variáveis mudarem frequentemente, isso pode dificultar a leitura e depuração do código.

5.4 Repetição condicional e a instrução *while*

Em programação existem dois tipos de iteração, indefinidas e definidas:

- Com **iteração definida**, o número de vezes que o bloco designado será executado é especificado explicitamente no momento em que o loop é iniciado. É o exemplo do `for` loop com o qual trabalhamos anteriormente.

- Com **iteração indefinida**, o número de vezes que o loop é executado não é especificado explicitamente com antecedência. Em vez disso, o bloco designado é executado repetidamente enquanto alguma condição for atendida.

Para interação indefinida, a construção típica do Python é o **while**. A forma mais simples desse loop **while** é a seguinte:

```
while <expr>:  
    <instruções>
```

Em que **instruções** representam uma ou mais linhas de código a serem executadas. Da mesma forma que nas outras estruturas, a **indentação** determina quais linhas vão ser executadas.

A expressão de controle, **<expr>**, normalmente envolve uma ou mais variáveis que são inicializadas antes de iniciar o loop e depois modificadas em algum lugar no corpo do loop. **Você precisa dessa expressão bem pensada, porque é ela que vai determinar o momento em que o loop para**

Quando um loop **while** é encontrado, **<expr>** é avaliado primeiro no contexto booleano. Se for verdadeiro, o corpo do loop é executado. Em seguida **<expr>**, é verificado novamente e, se ainda for verdadeiro, o corpo é executado novamente. Isso continua até **<expr>** se tornar falso, momento em que a execução do programa prossegue para a primeira instrução além do corpo do loop. **Dentro de <instruções> você precisa ter algo que modifique algum elemento de <expr>. Senão ou o loop não vai rodar ou vai rodar para sempre.** Uma das formas de interromper **à força** a execução de um loop é por meio das teclas **Ctrl+C**.

Considere o caso abaixo:

```
n = 5  
  
while n > 0:  
    n = n-1  
    print(n)  
  
print('\nÀ partir de agora as instruções fora do loop serão executadas.')
```

4
3
2
1
0

À partir de agora as instruções fora do loop serão executadas.

Veja o que está acontecendo neste exemplo:

O valor inicial de `n` é 5. A expressão no cabeçalho da instrução `while` na linha 2 é `n > 0`, o que é `True`, então o corpo do loop é executado. Dentro do corpo do loop redefinimos o valor de `n` para que seja igual ao valor anterior de `n` menos 1, e então o novo valor de `n` é impresso.

Quando o corpo do loop termina, a execução do programa retorna ao topo do loop na linha 2 e a expressão é avaliada novamente. Ainda é verdade, então o corpo é executado novamente e 3 é impresso. Isso continua até `n` se tornar 0. Nesse ponto, quando a expressão é testada, ela é falsa (0 não é maior do que 0) e o loop termina. A execução é retomada na primeira instrução após o corpo do loop, no caso a instrução `print`. Observe que a expressão de controle do loop `while` é testada primeiro, antes que qualquer outra coisa aconteça.

A seguir temos um loop `while` que usa uma lista:

```
a = ['foo', 'bar', 'baz']
print(a)

while a:

    a.pop(-1)
    print(a)
```

```
['foo', 'bar', 'baz']
['foo', 'bar']
['foo']
[]
```

Quando uma lista é avaliada em contexto booleano, ela é verdadeira se tiver elementos nela e falsa se estiver vazia. Neste exemplo, `a` é verdadeiro desde que tenha elementos nele. Quando todos os itens forem removidos com o método `.pop()` e a lista estiver vazia, `a` será falsa e o loop terminará.

5.4.1 Formas de Interromper um Loop

Em cada exemplo que você viu até agora, todo o corpo do loop `while` é executado em cada iteração. Python fornece duas palavras-chave que encerram uma iteração de loop prematuramente:

- A instrução Python `break` imediatamente encerra um loop inteiramente. Já vimos como ela funciona no caso do `for` loop e instruções condicionais.

- A instrução **continue**, por outro lado, encerra imediatamente a iteração do loop atual. A execução salta para o topo do loop e a expressão de controle é reavaliada para determinar se o loop será executado novamente ou terminará.

Um exemplo de **break**:

```
n = 5
print(n)

while n > 0:
    n = n - 1
    if n == 2:
        break
    print(n)

print('Loop ended.')
```

```
5
4
3
Loop ended.
```

Um exemplo de **continue**:

```
n = 5

while n > 0:
    n = n - 1
    if n == 2:
        continue
    print(n)
print('Loop ended.')
```

```
4
3
1
0
Loop ended.
```

5.5 Aplicação: Teorema Central do Limite

Obs: esse exercício é o mais complexo que fizemos até agora. Nele utilizamos grande parte dos conceitos que estudamos até aqui nesse curso, desde listas a loops e também a importação de bibliotecas de comandos. Você pode pular essa parte por enquanto, mas aconselho que aos poucos você tente replicá-lo, de forma a entender todas suas etapas com o tempo.

O **Teorema Central do Limite** é um resultado super importante em estatística e com aplicações nas mais diversas áreas. Em sua formulação mais simples, o teorema diz que:

A distribuição da média amostral da variável aleatória X aproxima-se cada vez mais de uma distribuição normal conforme aumenta o tamanho da amostra, independentemente da distribuição original de X .

Suponha que estejamos amostrando de uma variável aleatória X com média finita e igual a μ e desvio padrão finito e igual a σ . Então média e desvio padrão da distribuição amostral são representados por:

$$\mu_{\bar{X}} = \mu$$

$$\sigma_{\bar{X}} = \frac{\sigma}{\sqrt{N}}$$

Vamos apresentar uma implementação desse Teorema usando Python, em várias partes. A primeira parte mostra como tirar 10 números aleatórios entre -40 e 40, usando o NumPy. Mais uma vez utilizaremos a função `uniform()` já que elas sorteia números reais com igual probabilidade dentro de um intervalo especificado.

```
import numpy as np

x = np.random.uniform(-40, 40, 10)
print(x)
```

```
[ 36.54990223   0.74060922 -15.07737902 -38.07333374   9.3933279
 19.2484132    6.89403689   9.68213611  27.76959224  -4.17873714]
```

Note que se sortearmos 10 números novamente, o resultado será diferente

```
x = np.random.uniform(-40, 40, 10)
print(x)
```

```
[ 38.38276571 -28.26750616 -14.0773458  -30.81170843 -12.01404468
 37.36352155 -34.85748942   2.4632147   16.35091636  11.831912  ]
```

Para tornar o nosso exemplo mais previsível, vamos usar uma função dentro do NumPy que faz com o que o sorteio dos números parta do mesmo lugar e, portanto, o resultado seja o mesmo.

```
np.random.seed(1)

x = np.random.uniform(-40, 40, 10)
print(x)
```

```
[ -6.63823962  17.62595948 -39.99085001 -15.81339419 -28.25952873
 -32.61291242 -25.09918309 -12.35514184  -8.25860206   3.10533872]
```

```
np.random.seed(1)

x = np.random.uniform(-40, 40, 10)
print(x)
```

```
[ -6.63823962  17.62595948 -39.99085001 -15.81339419 -28.25952873
 -32.61291242 -25.09918309 -12.35514184  -8.25860206   3.10533872]
```

Feito o sorteio de números aleatórios, o próximo passo é tirar a média desse conjunto de números:

```
np.mean(x)
```

```
np.float64(-14.829655377323416)
```

Como vocês podem notar, a média não deu igual a zero, embora a média da distribuição uniforme com números inteiros que vai de -40 a 40 seja igual a 0. E aí surge a mágica do Teorema Central do Limite. Ainda que uma amostra não tenha a média igual à média populacional, a **média das médias** vai convergindo para zero à medida em que extraímos novas amostras e calculamos novas médias.

Agora vou fazer isso em um loop, sorteando 50 números aleatórios por amostra e primeiro com 10 amostras:

```

# number of sample
n = 10
means = []

np.random.seed(1)

for j in list(range(0,n)):

    lista_sorteio = np.random.uniform(-40, 40, 50)
    x = np.mean(lista_sorteio)
    means.append(x)

means = [np.round(elem,2) for elem in means]
print(means)
np.mean(means)

```

```

[np.float64(-2.64), np.float64(0.38), np.float64(-2.3), np.float64(-0.13), np.float64(3.95),
2.02), np.float64(2.95)]

```

```

np.float64(0.6619999999999999)

```

Agora com 100 amostras

```

# number of sample
n = 100
means = []

np.random.seed(1)

for j in list(range(0,n)):

    lista_sorteio = np.random.uniform(-40, 40, 50)
    x = np.mean(lista_sorteio)
    means.append(x)

means = [np.round(elem,2) for elem in means]
print(means)
np.mean(means)

```

```
[np.float64(-2.64), np.float64(0.38), np.float64(-2.3), np.float64(-0.13), np.float64(3.95),
2.02), np.float64(2.95), np.float64(-2.02), np.float64(-1.95), np.float64(-
1.69), np.float64(1.3), np.float64(2.67), np.float64(1.64), np.float64(-
2.17), np.float64(0.71), np.float64(-3.31), np.float64(-0.82), np.float64(-
1.1), np.float64(2.47), np.float64(-0.34), np.float64(1.6), np.float64(0.52), np.float64(-
0.78), np.float64(2.61), np.float64(2.68), np.float64(0.98), np.float64(-
2.24), np.float64(-4.57), np.float64(5.56), np.float64(2.17), np.float64(3.94), np.float64(4
7.24), np.float64(0.77), np.float64(-5.73), np.float64(0.17), np.float64(5.3), np.float64(3.
8.14), np.float64(-4.17), np.float64(-2.82), np.float64(-1.41), np.float64(-
7.05), np.float64(1.03), np.float64(1.15), np.float64(2.35), np.float64(-
2.7), np.float64(-3.12), np.float64(0.67), np.float64(-1.7), np.float64(-
0.95), np.float64(3.93), np.float64(1.95), np.float64(2.65), np.float64(-
2.48), np.float64(-5.42), np.float64(4.43), np.float64(1.88), np.float64(-
0.04), np.float64(-1.46), np.float64(1.04), np.float64(-1.54), np.float64(-
2.82), np.float64(-2.62), np.float64(-3.85), np.float64(-0.56), np.float64(1.08), np.float64
3.13), np.float64(-1.05), np.float64(3.14), np.float64(-5.01), np.float64(-
1.15), np.float64(-4.29), np.float64(5.22), np.float64(-5.29), np.float64(1.37), np.float64(
8.62), np.float64(-7.99), np.float64(-3.2), np.float64(3.04), np.float64(0.36), np.float64(3
1.4), np.float64(-3.84), np.float64(0.81), np.float64(7.0)]
```

```
np.float64(0.0246000000000000007)
```

Vamos deixar essa tentativa de aproximar a média das médias de μ mais automatizada:

```
expoentes = [1,2,3,4,5,6,7,8,9,10]
N = [2**exp for exp in expoentes]

means = []

np.random.seed(1)
for n in N:

    means_atual = []
    for j in list(range(0,n)):

        lista_sorteio = np.random.uniform(-40, 40, 100)
        x = np.mean(lista_sorteio)
        means_atual.append(x)

    means.append(np.mean(means_atual))
```

```
for m in means:

    print(np.round(m,2))
```

```
-1.17
0.92
0.37
-0.25
0.31
-0.32
0.23
-0.15
-0.06
0.03
```

Olha só a convergência aí, minha gente!

Note que até aqui a gente só mostrou que a **média das médias converge para a média populacional**. O Teorema, no entanto, é mais completo do que isso. Ele diz que a distribuição da média amostral é uma Normal e, além disso, possui aquelas características em relação à distribuição original. Ainda precisamos de mais algumas aulas para ir além e olhar essas outras questões do Teorema, mas logo logo chegamos lá.

5.6 Exercícios

1. O último teorema de *Fermat* diz que não há nenhum número inteiro positivo a, b, c tal que

$$a^n + b^n = c^n$$

para quaisquer valores de n maiores do que 2.

Use o que você aprendeu com condicionais, operações aritméticas e instruções `print` para testar se o teorema se mantém, dada a lista de números inteiros abaixo. Note que ao fim de cada iteração, o programa deve exibir “Holy smokes! Fermat was wrong” caso o teorema não valha e “Fermat was right” caso você não tenha refutado um gênio dos tempos modernos.

- `a = [1,2,3,4,5,6,7,8,9,10]`
- `b = [1,2,3,4,5,6,7,8,9,10]`
- `n = [3,4,5,6,7,8,9,10,37,52,89,100]`

2. X

6 Funções

6.1 Introdução

6.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- `numpy`
- `time`

```
import numpy as np
import time
```

6.2 O que é uma função?

No contexto das linguagens de programação, uma função é uma sequência nomeada de instruções, que executa algum tipo de operação específica mas não necessariamente numérica, como é o caso das funções matemáticas. A ideia essencial por trás de uma função é a de juntar algumas tarefas comuns ou repetidas e criar uma função para que, em vez de escrever o mesmo código várias vezes, possamos chama-la pelo nome e reutilizar o conjunto de instruções nela contida sempre que necessário.

Caso o objetivo de dividir um programa em funções ainda não tenha ficado claro, saiba que:

- Criar uma nova função dá a oportunidade de nomear um grupo de instruções, o que deixa o seu programa mais fácil de ler e de depurar
- As funções podem tornar um programa menor, eliminando o código repetitivo. Depois, caso precise fazer alguma alteração, basta fazê-la em um lugar só.
- Dividir um programa longo em funções permite depurar as partes uma de cada vez e então reuni-las em um conjunto funcional.
- As funções bem projetadas muitas vezes são úteis para muitos programas. Uma vez escritas e depuradas, você pode reutilizar as funções em programas fora daquele para o qual elas foram originalmente construídas.

Existem várias funções nativas no Python (e.g., `type()` para encontrar o tipo de um objeto) e mesmo dentro de bibliotecas com as quais já trabalhamos um pouco (e.g., `numpy.random.uniform()` para sortear números aleatórios de acordo com uma distribuição uniforme). À partir de agora veremos como podemos criar nossas próprias funções dentro da linguagem!

No Python, a sintaxe de uma função é dada por:

```
def nome_da_funcao(argumentos):  
    <instruções>
```

Assim como nos *loops*, para definir uma nova função no Python é preciso começar com uma palavra-chave, nesse caso `def`. O nome que daremos à função vem logo em seguida. É através desse nome, `nome_da_funcao`, que chamaremos essa função em outras partes do nosso código. Entre parênteses definimos os **argumentos** que a função recebe para realizar o conjunto de instruções em `<instruções>`.

Note, mais uma vez, que todo o bloco de código que estiver **identado** e abaixo da linha de cabeçalho da função fará parte da função. Assim como nos *loops*, a função termina quando passarmos a primeira linha de código não-identada.

6.3 Nossa primeira função

Vamos começar criando uma função simples, que tem por objetivo printar uma das frases mais conhecidas da história do cinema:

```
def frase_cinema():  
    print('Que a força esteja com você!')
```

Note que nesse caso, o parênteses logo após o nome da função está vazio. Isso quer dizer que essa função não recebe argumentos, apenas printa a frase entre aspas sempre que for chamada. Para chamá-la, basta usar o nome `frase_cinema` seguido dos parênteses vazios:

```
frase_cinema()
```

Que a força esteja com você!

6.4 Argumentos de uma função

Na função anterior, não tínhamos nenhum argumento. Ou seja, toda vez que você chamar a função `frase_cinema`, ela vai fazer a mesma coisa. Pode até ser que seja o seu objetivo fazer exatamente isso – é uma forma de economizar código.

O ponto é que a ideia de função é muito mais poderosa do que simplesmente uma “abreviação” de um monte de linhas de código. A abstração implícita no conceito de função é poderosa o suficiente para garantir que a função retorne coisas diferentes caso você altere algum **argumento**. Vamos falar disso a seguir.

6.4.1 *Keyword arguments e default values*

Vamos então criar uma função que tenha argumentos.

```
def my_func(name,place):  
    print(f"Olá {name}! Você é de {place}?")  
  
my_func("Emily","Paris")
```

Olá Emily! Você é de Paris?

O que acontece se você especificar o `place` primeiro e depois o `name`? Vamos descobrir.

```
my_func("Hawaii","Robert")
```

Olá Hawaii! Você é de Robert?

Meio bizarro, não? A razão é que aqui temos os chamados **argumentos posicionais**. Ou seja, a função vai assumir que o primeiro argumento é o `name` e o segundo é o `place` não importa o que tenhamos passado como argumento. Para lidar com isso, a gente pode atribuir um nome a cada um dos argumentos, ou **palavra-chave**, que aí vai ser a palavra-chave, e não a posição, que vai determinar qual o valor atribuído a cada argumento.

```
my_func(place="Hawaii",name="Robert")
```

Olá Robert! Você é de Hawaii?

Vimos aqui que a posição passou a ser irrelevante porque colocamos os nomes de cada um dos argumentos. E se quiséssemos dar mais flexibilidade ainda à função, só especificando um subconjunto dos argumentos? Para que isso funcione, nós precisamos especificar os **valores-padrão** dos argumentos caso eles não sejam fornecidos.

Aqui tem uma função que faz isso.

```
def total_calc(bill_amount,tip_perc=10):  
  
    total = bill_amount*(1 + tip_perc/100)  
    total = round(total,2)  
    print(f"Please pay ${total}")
```

Nesse caso, o argumento `bill_amount` é obrigatório. Por outro lado, o argumento `tip_perc` vai assumir o valor de 10 toda vez que ele não for explicitamente fornecido na chamada da função.

6.4.2 Argumentos arbitrários

Vamos começar fazendo algumas perguntas: * E se não soubermos o número exato de argumentos de antemão? * Podemos criar funções que funcionem com um número variável de argumentos?

A resposta é *sim*! E vamos criar essa função imediatamente. Vamos criar uma função simples `my_var_sum()` que retorna a soma de todos os números passados como argumento. No entanto, o número de argumentos pode ser potencialmente diferente cada vez que chamamos a função.

```
def my_var_sum(*args):  
  
    sum = 0  
    for arg in args:  
        sum += arg  
  
    print(f"The numbers that you have add up to {sum}")
```

Observe como a definição da função agora tem `*args` em vez de apenas o nome do parâmetro. No corpo da função, fazemos um loop em `args` até usarmos todos os argumentos. A função `my_var_sum` retorna a soma de todos os números passados como argumentos. Olha só o que acontece quando chamamos a função com diferentes números de argumentos:

```
my_var_sum(99,10,54,23)  
my_var_sum(9,87)  
my_var_sum(5,21,36,79,45,65)  
my_var_sum(1)
```

```
The numbers that you have add up to 186
The numbers that you have add up to 96
The numbers that you have add up to 251
The numbers that you have add up to 1
```

Mas e se eu quiser passar não apenas uma sequência de valores, mas uma sequência de valores *com nomes*? Não se preocupe, existe uma possibilidade de fazer isso, usando o ****kwargs** na definição dos argumentos. Qual é a diferença entre ***args** e ****kwargs**? A diferença é que você vai passar uma sequência de tamanho arbitrário de parâmetros, cada um deles nomeado.

O Python vai entender o ****kwargs** como um dicionário. Cada elemento passado é um par chave-valor, e dentro da função você tem que desempacotar o valor da chave. Vamos seguir alguns exemplos.

```
def myFun(**kwargs):

    for key, value in kwargs.items():
        print("%s == %s" % (key, value))

myFun(first='Geeks', mid='for', last='Geeks')
```

```
first == Geeks
mid == for
last == Geeks
```

Podemos misturar os tipos de argumentos (posicionais versus ***args**/****kwargs**)? Sim, como podemos ver no exemplo seguinte:

```
def myFun(arg1, **kwargs):

    for key, value in kwargs.items():
        print(arg1+" %s == %s" % (key, value))

myFun("Hi - ", first='Geeks', mid='for', last='Geeks')
```

```
Hi - first == Geeks
Hi - mid == for
Hi - last == Geeks
```

6.4.3 Elementos dentro da função: execução condicional

As funções podem retornar objetos booleanos também, o que pode ser conveniente para esconder testes complicados dentro de funções. Por exemplo:

```
def is_divisible(x, y):  
  
    if x % y == 0:  
        print(True)  
    else:  
        print(False)  
  
is_divisible(6, 4)
```

False

6.4.4 Aplicação: Teorema Central do Limite - Parte 1

O primeiro passo para replicarmos, na forma de uma função, a aplicação do Teorema Central do Limite da aula anterior é definir uma função que recebe como argumentos a distribuição da variável aleatória X , o intervalo no qual a variável está definida e o número de sorteios que faremos em cada amostra. Com o que aprendemos até agora podemos definir a função `func_tcl` abaixo:

```
def func_tcl(dist=None, intervalo=(0,1), n=100):  
  
    if dist == None:  
        print('Você esqueceu de carregar uma função que defina a distribuição de X! Volte 2 c  
    else:  
        x = dist(intervalo[0], intervalo[1], n)  
        print(x)
```

Para todos os 3 argumentos definimos valores *default* que, caso não sejam alterados a função vai utilizá-los em suas operações. Vamos rodar a função com todos os valores *default* e ver o que acontece:

```
func_tcl()
```

Você esqueceu de carregar uma função que defina a distribuição de X ! Volte 2 casas.

Nesse caso, a execução condicional `if` dentro da função acende já que não passamos nenhuma distribuição como default, apenas o valor `None`. Vamos então repetir o que fizemos na aula passada e utilizar a distribuição uniforme, no intervalo $[-40, 40]$ e sorteando apenas 10 números desta distribuição:

```
func_tcl(dist=np.random.uniform,intervalo=(-40,40),n=10)
```

```
[-39.65806441 -30.96541965 -19.08863921  30.54649373   1.84760604
 -18.95243566 -25.48647421  38.03515039 -21.06328405 -12.58741168]
```

6.4.5 Elementos dentro da função: iteração

Da mesma forma que podemos ter condições lógicas dentro da função, podemos ter blocos de repetição (`for` e `while`) dentro da função, assim como no caso de `*args`. Vamos olhar para um outro exemplo em que a função recebe uma lista de strings como argumento e faz operações em cada um dos elementos, um a um:

```
states = [' Alabama ', 'Georgia!', 'Georgia', 'georgia', 'FlOrIda', 'south carolina##', 'West virgini']

def clean_strings(lista_strings):
    result=[]

    for value in lista_strings:
        value = value.strip()
        value = value.title()
        value = value.replace('#', '')
        value = value.replace('?', '')
        value = value.replace('!', '')
        result.append(value)

    print(result)

print(states)
print()
clean_strings(states)
```

```
[' Alabama ', 'Georgia!', 'Georgia', 'georgia', 'FlOrIda', 'south carolina##', 'West virginia']
```

```
['Alabama', 'Georgia', 'Georgia', 'Georgia', 'Florida', 'South Carolina', 'West Virginia']
```

6.4.6 Aplicação: Teorema Central do Limite - Parte 2

Agora podemos dar mais um passo e incorporar o loop dentro da função, passando como novo argumento da função o número de amostras com o qual queremos trabalhar:

```
def func_tcl(dist=None, intervalo=(0,1), n=100, samples=10):

    if dist == None:
        print('Você esqueceu de carregar uma função que defina a distribuição de X!')
    else:
        means = []
        for j in range(0, samples):
            x_func_tcl = dist(intervalo[0], intervalo[1], n)
            mean_x = sum(x_func_tcl)/len(x_func_tcl)

            means.append(mean_x)

        print('Essa é a lista de médias:')
        print(means)
        print('\nE essa é a média das médias:')
        print(sum(means)/len(means))

# utilizamos a função seed mais uma vez para tornar os resultados previsíveis
np.random.seed(1)

func_tcl(dist=np.random.uniform, intervalo=(-40,40), n=50, samples=10)
```

Essa é a lista de médias:

```
[np.float64(-2.6419520291841945), np.float64(0.382420445207494), np.float64(-2.2971869003899186), np.float64(-0.12856511296217676), np.float64(3.945331018819081), np.float64(2.0177969126899638), np.float64(2.952952578448611)]
```

E essa é a média das médias:

```
0.662169619365357
```

Que bacana!!!

Mas note que o valor da média que nos interessa nesse caso só aparece printado quando chamamos a função. O que podemos fazer para conseguir guardá-lo como uma nova variável, por exemplo?

6.5 Valor de retorno de uma função

O comando `return` é usado para a saída de uma função de volta ao lugar no código de onde ela foi chamada. A sintaxe desse comando é simples:

```
return <expression_list>
```

Essa declaração pode ter um comando que é executado e o valor resultante devolvido. Se esse comando não for anexado ou não tiver nada além do `return`, a função é devolvida com o valor de `None`. Por exemplo:

```
def my_func(name,place):  
  
    print(f"Olá {name}! Você é de {place}?")  
    return  
  
print(my_func("Jane","Paris"))
```

Olá Jane! Você é de Paris?
None

Funções que não contenham o comando `return` sempre retornarão um valor vazio. Quando queremos utilizar uma função com o objetivo de realizar algum tipo de operação e **guardar** o valor dessa operação, não podemos esquecer do `return`. Vejamos o exemplo da função que calcula a raiz quadrada de um número:

```
def raiz(x):  
  
    fx = x**0.5  
  
y = raiz(100)  
print(y)
```

None

```
def raiz(x):  
  
    fx = x**0.5  
    return fx  
  
y = raiz(100)  
print(y)
```

6.5.1 Expectativa de vida de variáveis dentro da função

Quando você cria uma variável **dentro** de uma função ela é local, ou seja, só existe dentro da própria função. Por exemplo:

```
def concat_strings(str1,str2):

    texto_concatenado = str1 + ' ' + str2
    print(texto_concatenado)

texto1 = 'Que a força esteja com você,'
texto2 = 'jovem Padawan.'

concat_strings(texto1,texto2)
```

Que a força esteja com você, jovem Padawan.

Essa função recebe dois argumentos, concatena-os e exibe o resultado em uma única linha de texto. No entanto, assim que a função é encerrada, a variável `texto_concatenado` é deletada. O que acontece se tentarmos acessá-la?

```
print(texto_concatenado)
```

```
NameError: name 'texto_concatenado' is not defined
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[23], line 1
----> 1 print(texto_concatenado)
NameError: name 'texto_concatenado' is not defined
```

6.5.2 Aplicação: Teorema Central do Limite - Parte 3

Retomando a função da parte 2:

```
def func_tcl(dist=None, intervalo=(0,1), n=100, samples=10):

    if dist == None:
        print('Você esqueceu de carregar uma função que defina a distribuição de X!')
    else:
        means = []
        for j in range(0, samples):
            x_func_tcl = dist(intervalo[0], intervalo[1], n)
            mean_x = sum(x_func_tcl)/len(x_func_tcl)

            means.append(mean_x)
            mean_of_means = sum(means)/len(means)

        print('Essa é a lista de médias:')
        print(means)
        print('\nE essa é a média das médias:')
        print(mean_of_means)
```

```
np.random.seed(1)
```

```
func_tcl(dist=np.random.uniform, intervalo=(-40,40), n=50, samples=10)
```

Essa é a lista de médias:

```
[np.float64(-2.6419520291841945), np.float64(0.382420445207494), np.float64(-
2.2971869003899186), np.float64(-0.12856511296217676), np.float64(3.945331018819081), np.float64(
2.0177969126899638), np.float64(2.952952578448611)]
```

E essa é a média das médias:

```
0.662169619365357
```

Como acabamos de ver, se quisermos acessar a variável `mean_of_means` o Python retornará um erro. As variáveis definidas dentro de uma função tem vida curta: elas existem apenas dentro da função!

```
print(mean_of_means)
```

```
NameError: name 'mean_of_means' is not defined
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[26], line 1
----> 1 print(mean_of_means)
NameError: name 'mean_of_means' is not defined
```

No entanto, agora já sabemos o que a palavra-chave `return` faz dentro de uma função. Vamos utilizá-la!

```
def func_tcl(dist=None, intervalo=(0,1), n=100, samples=10):

    if dist == None:
        print('Você esqueceu de carregar uma função que defina a distribuição de X!')
    else:
        means = []
        for j in range(0, samples):
            x_func_tcl = dist(intervalo[0], intervalo[1], n)
            mean_x = sum(x_func_tcl)/len(x_func_tcl)

            means.append(mean_x)
            mean_of_means = sum(means)/len(means)

        return mean_of_means
```

```
np.random.seed(1)
```

```
func_tcl(dist=np.random.uniform, intervalo=(-40,40), n=50, samples=10)
```

```
np.float64(0.662169619365357)
```

Maravilha, agora a função nos retorna apenas o que é do nosso interesse: a média das médias. Para chegar, finalmente, no mesmo resultado da aula passada basta colocar essa função dentro de um loop e tacar-lhe pau nesse carrinho, Marcos!

```
expoentes = [1,2,3,4,5,6,7,8,9,10]
Y = [2**exp for exp in expoentes]

means_of_means = []

np.random.seed(1)
for y in Y:

    mean_of_means = func_tcl(dist=np.random.uniform, intervalo=(-40,40), n=100, samples=y)
    means_of_means.append(mean_of_means)
```

```
for m in means_of_means:

    print(np.round(m,2))
```

```
-1.17
0.92
0.37
-0.25
0.31
-0.32
0.23
-0.15
-0.06
0.03
```

Voilà!

Como exercício para casa, tente fazer alterações nessa função de modo que ela receba não um valor com o número de amostras, mas uma lista de números de amostras. A função deve cuspir como resultado a lista `means_of_means` e não apenas o valor `mean_of_means`. Pratique!

6.6 Documentação

Uma **docstring** é uma string no início de uma função, definida pelo usuário, que serve como documentação do que a função faz. A docstring vem logo depois da primeira linha que define a função e é delimitada por aspas triplas, o que permite que a string se estenda por várias linhas, como vocês devem se lembrar.

Vamos criar como exemplo uma função que printa o tipo do objeto que é passado como argumento:

```
def imprime_tipo(x):

    """
    Função criada para a matéria EAE1106 - Métodos Computacionais para Economia
    Objetivo: função simples que imprime o tipo do objeto recebido como argumento.
    """

    print(type(x))
```

Embora opcional, a documentação é uma **boa prática de programação**. A menos que você consiga se lembrar qual foi o cardápio do bandejão na semana passada, sempre documente seu código. Podemos acessar a documentação de determinada função utilizando o atributo `__doc__`.

```
print(imprime_tipo.__doc__)
```

Função criada para a matéria EAE1106 - Métodos Computacionais para Economia

Objetivo: função simples que imprime o tipo do objeto recebido como argumento.

Isso vale também para funções nativas e definidas em outros pacotes do Python. Por exemplo,

```
# documentação da função nativa len()
print(len.__doc__)
```

Return the number of items in a container.

```
# documentação da função time() dentro do pacote time
print(time.time.__doc__)
```

time() -> floating-point number

Return the current time in seconds since the Epoch.

Fractions of a second may be present if the system clock provides them.

```
# documentação da função uniform() dentro do pacote NumPy
print(np.random.uniform.__doc__)
```

```
uniform(low=0.0, high=1.0, size=None)
```

Draw samples from a uniform distribution.

Samples are uniformly distributed over the half-open interval `[low, high)` (includes low, but excludes high). In other words, any value within the given interval is equally likely to be drawn by `uniform`.

.. note::
New code should use the `~numpy.random.Generator.uniform` method of a `~numpy.random.Generator` instance instead; please see the :ref:`random-quick-start`.

Parameters

`low` : float or array_like of floats, optional
Lower boundary of the output interval. All values generated will be greater than or equal to `low`. The default value is 0.

`high` : float or array_like of floats
Upper boundary of the output interval. All values generated will be less than or equal to `high`. The high limit may be included in the returned array of floats due to floating-point rounding in the equation `low + (high-low) * random_sample()`. The default value is 1.0.

`size` : int or tuple of ints, optional
Output shape. If the given shape is, e.g., `(m, n, k)`, then `m * n * k` samples are drawn. If `size` is `None` (default), a single value is returned if `low` and `high` are both scalars. Otherwise, `np.broadcast(low, high).size` samples are drawn.

Returns

`out` : ndarray or scalar
Drawn samples from the parameterized uniform distribution.

See Also

`randint` : Discrete uniform distribution, yielding integers.
`random_integers` : Discrete uniform distribution over the closed interval `[low, high]`.
`random_sample` : Floats uniformly distributed over `[0, 1)`.
`random` : Alias for `random_sample`.
`rand` : Convenience function that accepts dimensions as input, e.g., `rand(2,2)` would generate a 2-by-2 array of floats, uniformly distributed over `[0, 1)`.
`random.Generator.uniform`: which should be used for new code.

Notes

The probability density function of the uniform distribution is

```
.. math:: p(x) = \frac{1}{b - a}
```

anywhere within the interval ``[a, b)``, and zero elsewhere.

When ``high`` == ``low``, values of ``low`` will be returned. If ``high`` < ``low``, the results are officially undefined and may eventually raise an error, i.e. do not rely on this function to behave when passed arguments satisfying that inequality condition. The ``high`` limit may be included in the returned array of floats due to floating-point rounding in the equation ``low + (high-low) \* random\_sample()``. For example:

```
>>> x = np.float32(5*0.99999999)
>>> x
np.float32(5.0)
```

Examples

Draw samples from the distribution:

```
>>> s = np.random.uniform(-1,0,1000)
```

All values are within the given interval:

```
>>> np.all(s >= -1)
True
>>> np.all(s < 0)
True
```

Display the histogram of the samples, along with the probability density function:

```
>>> import matplotlib.pyplot as plt
>>> count, bins, ignored = plt.hist(s, 15, density=True)
>>> plt.plot(bins, np.ones_like(bins), linewidth=2, color='r')
>>> plt.show()
```

Note do exemplo acima que uma docstring pode conter uma descrição detalhada do funcionamento de uma função, inclusive com exemplos de aplicação. Legal, né? Esse tipo de

documentação também está disponível para as bibliotecas como um todo. Isso pode nos ajudar, por exemplo, a conhecer o conteúdo de uma determinada biblioteca.

```
# documentação do NumPy
print(np.__doc__)
```

NumPy
=====

Provides

1. An array object of arbitrary homogeneous items
2. Fast mathematical operations over arrays
3. Linear Algebra, Fourier Transforms, Random Number Generation

How to use the documentation

Documentation is available in two forms: docstrings provided with the code, and a loose standing reference guide, available from the NumPy homepage <<https://numpy.org>>`\_`.

We recommend exploring the docstrings using IPython <<https://ipython.org>>`, an advanced Python shell with TAB-completion and introspection capabilities. See below for further instructions.

The docstring examples assume that `numpy` has been imported as ``np``::

```
>>> import numpy as np
```

Code snippets are indicated by three greater-than signs::

```
>>> x = 42
>>> x = x + 1
```

Use the built-in ``help`` function to view a function's docstring::

```
>>> help(np.sort)
... # doctest: +SKIP
```

For some objects, ``np.info(obj)`` may provide additional help. This is particularly true if you see the line "Help on ufunc object:" at the top

of the `help()` page. Ufuncs are implemented in C, not Python, for speed. The native Python `help()` does not know how to view their help, but our `np.info()` function does.

Available subpackages

lib

Basic functions used by several sub-packages.

random

Core Random Tools

linalg

Core Linear Algebra Tools

fft

Core FFT routines

polynomial

Polynomial tools

testing

NumPy testing tools

distutils

Enhancements to distutils with support for

Fortran compilers support and more (for Python <= 3.11)

Utilities

test

Run numpy unittests

show_config

Show numpy build configuration

__version__

NumPy version string

Viewing documentation using IPython

Start IPython and import ``numpy`` usually under the alias ``np``: ``import numpy as np``. Then, directly past or use the ``%cpaste`` magic to paste examples into the shell. To see which functions are available in ``numpy``, type ``np.<TAB>`` (where ``<TAB>`` refers to the TAB key), or use ``np.*cos*<ENTER>`` (where ``<ENTER>`` refers to the ENTER key) to narrow down the list. To view the docstring for a function, use ``np.cos?<ENTER>`` (to view the docstring) and ``np.cos??<ENTER>`` (to view the source code).

Copies vs. in-place operation

Most of the functions in `numpy` return a copy of the array argument (e.g., `np.sort`). In-place versions of these functions are often available as array methods, i.e. ``x = np.array([1,2,3]); x.sort()``. Exceptions to this rule are documented.

Podemos usar sempre esse atalho caso desejemos conhecer as funcionalidades que uma biblioteca guarda por trás de suas cortinas!

6.7 Funções anônimas

Em Python, uma função anônima é uma função definida sem nome. Quem poderia imaginar, não é mesmo?

Enquanto as funções normais são definidas usando a palavra-chave **def** em Python, as funções anônimas são definidas usando a palavra-chave **lambda**. Portanto, funções anônimas também são chamadas de **funções lambda**. A estrutura usual de uma função lambda é a seguinte:

```
lambda <argumentos>: <expressão>
```

Usualmente a gente precisa de uma função lambda porque precisamos de uma função rápida por um período de tempo e/ou quando a gente usa técnicas mais poderosas que possuem funções como argumento, como **filter** e **map**. Vou fazer um exemplo com cada uma delas.

Usando filter

A função **filter()** em Python recebe uma função e uma lista como argumentos. A função é chamada com todos os itens da lista e uma nova lista é retornada contendo itens para os quais a função avalia **True**. Aqui está um exemplo de uso da função para filtrar apenas números pares de uma lista.

```
my_list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(filter(lambda x: (x%2 == 0) , my_list))

print(new_list)
```

```
[4, 6, 8, 12]
```

Usando map

A função `map()` em Python recebe uma função e uma lista. A função é chamada com todos os itens da lista e uma nova lista é retornada contendo os itens retornados por essa função para cada item. Aqui está um exemplo de uso da função `map()` para dobrar todos os itens em uma lista.

```
my_list = [1, 5, 4, 6, 8, 11, 3, 12]

new_list = list(map(lambda x: x * 2 , my_list))

print(new_list)
```

[2, 10, 8, 12, 16, 22, 6, 24]

6.8 Exercícios

1. X
2. X

Parte II

Computação numérica, análise de dados e visualização

Após a introdução à lógica de programação e aos principais conceitos da linguagem Python, esta parte do curso concentra-se na aplicação prática dessas ideias em tarefas típicas de análise empírica. O objetivo é apresentar as principais bibliotecas utilizadas para computação numérica, manipulação de dados e visualização, e mostrar como elas se articulam em um fluxo de trabalho único e coerente.

Nesta etapa, trabalharemos com três bibliotecas centrais do ecossistema Python: **NumPy**, **Pandas** e **Matplotlib**, além de outras ferramentas complementares. O objetivo é fornecer uma base sólida para trabalhar com dados reais e organizar análises de forma reproduzível. Listo abaixo alguns dos principais conceitos trabalhados nos capítulos que compõe esta parte do curso:

- **Capítulo 7 – Arrays, matrizes e álgebra linear.** Introduz estruturas numéricas fundamentais, como vetores e matrizes, e operações básicas e avançadas de álgebra linear, que formam a base da computação numérica em Python. A principal biblioteca utilizada aqui é o **NumPy**.
- **Capítulo 8 – Gestão e análise de dados.** Apresenta ferramentas para organizar, transformar e analisar bases de dados estruturadas, incluindo importação de dados, manipulação de colunas, agregações, junções e reorganização de dataframes. A principal biblioteca utilizada aqui é o **Pandas**.
- **Capítulo 9 — Visualização de dados.** Discute a construção de gráficos utilizando o **Matplotlib** e bibliotecas complementares, explorando diferentes tipos de visualizações e princípios básicos para comunicar informações de forma clara e informativa. A principal biblioteca utilizada aqui é o **Matplotlib**, mas falaremos também de outros ecossistemas importantes no mundo da visualização, como **Seaborn** e **Plotly**.
- **Capítulo 10 — Análise empírica integrada.** Integra os conteúdos anteriores em um fluxo completo de análise, abordando organização de projetos, limpeza de dados, análises descritivas, visualização e interpretação de resultados empíricos. A ideia principal deste capítulo é implementar um projeto de análise de dados do início ao fim, incorporando as bibliotecas, funções e conceitos aprendidos nos três capítulos anteriores.

Juntos, esses quatro capítulos fornecem uma visão integrada das principais ferramentas e práticas utilizadas em análise de dados com Python. Ao final desta parte do curso, mais do que dominar bibliotecas específicas, o aluno deverá ser capaz de estruturar e executar uma análise empírica completa – desde a organização dos dados e a realização de operações numéricas até a visualização e interpretação dos resultados – utilizando um fluxo de trabalho coerente, reproduzível e adequado a aplicações empíricas em economia.

7 Arrays, matrizes e álgebra linear

7.1 Introdução

7.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- `numpy`
- `time`
- `sympy`

```
import numpy as np
import sympy
import time
```

7.2 O que é o NumPy?

NumPy (**N**umerical **P**ython) é o pacote fundamental para computação científica em Python. Essa biblioteca é peça fundamental em outras bibliotecas igualmente importantes, como o Pandas e o Matplotlib. É uma biblioteca Python que tem como principal objeto o `ndarray`, um array multidimensional que guarda bastante semelhança com a ideia de vetores e matrizes, embora seja um objeto específico dentro da linguagem, com suas características e métodos próprios. O pacote contém também uma variedade de rotinas para operações rápidas em arrays, incluindo matemática, lógica, álgebra linear básica, operações estatísticas básicas e muito mais.

Mas o que é de fato um `ndarray`? É um objeto multidimensional que nos permite armazenar dados de forma sequencial e que podem ser acessados via indexação. Ué, mas isso é muito parecido com uma lista (ou um conjunto de listas). Qual a diferença então?

- NumPy arrays têm um tamanho fixo na criação, ao contrário das listas, que podem crescer. Alterar o tamanho de um `ndarray` criará um novo array e excluirá o original.

- Todos os elementos em um array devem ser do mesmo tipo de dados, diferentemente de listas, que são objetos mais genéricos. Isso facilita a gestão de memória e torna operações com esse tipo de objeto ordens de magnitude mais rápidas do que se utilizássemos listas.
- A maior velocidade e eficiência de armazenamento fazem do NumPy uma das bibliotecas mais utilizadas em aplicações matemáticas e científicas. Saber apenas as ferramentas nativas do Python, como listas, hoje já não é mais suficiente.

São muitas as qualidades do NumPy que fazem dele a melhor escolha quanto o assunto é lidar com objetos sequenciais, multidimensionais, e com os quais queremos operar tal qual vetores e matrizes. Mas chega de lenga lenga, vamos ao trabalho!

7.3 Elementos básicos do NumPy

7.3.1 Arrays unidimensionais

Começemos criando um `numpy.ndarray` do zero, contendo os números 1, 2 e 3. Podemos fazê-lo da seguinte forma:

```
a = np.array([1, 2, 3])
print(a)
print(type(a))
```

```
[1 2 3]
<class 'numpy.ndarray'>
```

A sintaxe é essa mesma: um par de colchetes dentro dos parênteses. Se tentarmos passar sem os colchetes, o Python retornará um erro.

```
a = np.array(1, 2, 3)
```

```
TypeError: array() takes from 1 to 2 positional arguments but 3 were given
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 a = np.array(1, 2, 3)
TypeError: array() takes from 1 to 2 positional arguments but 3 were given
```

Da mesma forma que uma sequência de 3 números inteiros, podemos criar um `numpy.ndarray` que repete 3 vezes o número 0 ou 3 vezes o número 1.

```
a = np.zeros(3)
b = np.ones(3)

print(a)
print(b)
```

```
[0. 0. 0.]
[1. 1. 1.]
```

Note que em ambos os casos os números aparecem com o ponto da casa decimal, o que é um indicativo de que estão armazenados como valores do tipo `float`. E se quiséssemos criar esses mesmos arrays, mas especificando que os números são inteiros, i.e., do tipo `int`?

```
a = np.zeros(3, dtype=int)
b = np.ones(3, dtype=int)

print(a)
print(b)
```

```
[0 0 0]
[1 1 1]
```

A função `numpy.linspace(x,y,z)` nos permite criar um array que vai de `x` até `y`, com `z` elementos igualmente espaçados.

```
a = np.linspace(0,8,5, dtype=int)

print(a)
```

```
[0 2 4 6 8]
```

Podemos acessar os elementos de um array qualquer utilizando a mesma ideia de indexação de listas:

```
print('Array a =',a)
print('\nPrimeiro elemento de a = ',a[0])
print('Segundo elemento de a = ',a[1])
print('Último elemento de a = ',a[-1])
print('Dois primeiros elementos de a = ',a[0:2])
```

```
Array a = [0 2 4 6 8]
```

```
Primeiro elemento de a = 0
```

```
Segundo elemento de a = 2
```

```
Último elemento de a = 8
```

```
Dois primeiros elementos de a = [0 2]
```

7.3.2 Arrays multidimensionais

Como falamos anteriormente, um objeto do tipo `numpy.ndarray` é um array *n-dimensional* (por isso o **nd** em **ndarray**). Até agora trabalhamos apenas com uma dimensão, mas para as nossas aplicações é particularmente interessante o caso em que o número de dimensões é igual a 2, i.e., para o caso em que o array assume a forma de uma **matriz**.

Podemos criar esse tipo de array usando a mesma lógica de antes, com pequenas alterações na sintaxe:

```
A_22 = np.array([[1, 2], [3, 4]], dtype=int)

print('Matriz A =\n',A_22)
```

```
Matriz A =
[[1 2]
 [3 4]]
```

O NumPy nos oferece algumas funções interessantes para criarmos matrizes específicas:

```
print('Matriz identidade 2x2 =\n',np.eye(2,dtype=int))
print('\nMatriz diagonal 3x3 =\n',np.diag([1,2,3]))
```

```
Matriz identidade 2x2 =
[[1 0]
 [0 1]]
```

```
Matriz diagonal 3x3 =
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

Podemos criar a matriz **transposta** de uma dada matriz utilizando a função `np.transpose()` (ou apenas o método `.T`):

```
print('Matriz A =\n',A_22)
print('\nTransposta da matriz A =\n',np.transpose(A_22))
print('\nTransposta da matriz A =\n',A_22.T)
```

```
Matriz A =
[[1 2]
 [3 4]]
```

```
Transposta da matriz A =
[[1 3]
 [2 4]]
```

```
Transposta da matriz A =
[[1 3]
 [2 4]]
```

Assim como em arrays unidimensionais, podemos acessar os elementos de uma matriz utilizando indexação, mas agora em 2 dimensões:

```
A = np.array([[1,2,3], [4,5,6], [7,8,9]])

print('Matriz A =\n',A)
print('\nElemento 11 de A = ',A[0,0])
print('Elemento 23 de A = ',A[1,2])
print('Primeira linha de A = ',A[0,:])
print('Segunda coluna de A = ',A[:,1])
print('\nSubmatriz de A delimitada pelos elementos 22 e 33 =\n',A[1:,1:])
```

```
Matriz A =
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

```
Elemento 11 de A = 1
Elemento 23 de A = 6
Primeira linha de A = [1 2 3]
Segunda coluna de A = [2 5 8]
```

```
Submatriz de A delimitada pelos elementos 22 e 33 =
[[5 6]
 [8 9]]
```

7.3.3 Propriedades de arrays

O NumPy fornece alguns métodos úteis que, apesar de não receberem nenhum argumento, nos permitem acessar algumas das características dos arrays que criamos.

- `ndim` retorna o número de dimensões do array;
- `shape` retorna o tamanho do array em cada uma de suas dimensões;
- `dtype` retorna o tipo de dado contido no array;
- `size` retorna o número total de elementos contidos no array.

```
X1 = np.array([[1,2,3], [4,5,6], [7,8,9]])
X2 = X1.flatten() # O método flatten reduz um array de n-dimensões em um array de uma única d

print('Array X1 =\n',X1)
print('\nDimensões de X1 = ', X1.ndim)
print('Shape de X1 = ', X1.shape)
print('Tipo de dado em X1 = ', X1.dtype)
print('Número de elementos em X1 = ', X1.size)

print('\n\nArray X2 =\n',X2)
print('\nDimensões de X2 = ', X2.ndim)
print('Shape de X2 = ', X2.shape)
print('Tipo de dado em X2 = ', X2.dtype)
print('Número de elementos em X2 = ', X2.size)
```

Array X1 =

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Dimensões de X1 = 2

Shape de X1 = (3, 3)

Tipo de dado em X1 = int64

Número de elementos em X1 = 9

Array X2 =

```
[1 2 3 4 5 6 7 8 9]
```

Dimensões de X2 = 1

Shape de X2 = (9,)

Tipo de dado em X2 = int64

Número de elementos em X2 = 9

Um método interessante de arrays é o `reshape(x,y)` que reorganiza um array existente de acordo com os argumentos `x` e `y`:

```
X = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]])

print('Array X1 4x4 =\n',X)
print('\n X1 reorganizado em 2x8 =\n',X.reshape(2,8))
print('\n X1 reorganizado em 8x2 =\n',X.reshape(8,2))
```

```
Array X1 4x4 =
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
X1 reorganizado em 2x8 =
[[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]]
```

```
X1 reorganizado em 8x2 =
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]
 [13 14]
 [15 16]]
```

7.4 Operações básicas com arrays

7.4.1 Operações aritméticas simples

Dois tipos de operações que serão úteis para arrays de qualquer dimensão são:

1. Operações entre um array e um único número.
2. Operações entre dois arrays da mesma forma.

Quando realizamos operações em um array usando um único número, simplesmente aplicamos essa operação a cada elemento do array. Isso vale tanto para arrays unidimensionais (vetores) quanto multidimensionais (por ex., matrizes).

```
x = np.array([1,2,3], dtype=int)
print("x =\n", x)
print("\n2 + x =\n", 2 + x)
print("\n2 - x =\n", 2 - x)
print("\n2 * x =\n", 2 * x)
print("\nx / 2 =\n", x / 2)
```

```
x =
[1 2 3]
```

```
2 + x =
[3 4 5]
```

```
2 - x =
[ 1  0 -1]
```

```
2 * x =
[2 4 6]
```

```
x / 2 =
[0.5 1.  1.5]
```

```
X = np.ones((2, 2), dtype=int)

print("X =\n", X)
print("\n2 + X =\n", 2 + X)
print("\n2 - X =\n", 2 - X)
print("\n2 * X =\n", 2 * X)
print("\nX / 2 =\n", X / 2)
```

```
X =
[[1 1]
 [1 1]]
```

```
2 + X =
[[3 3]
 [3 3]]
```

```
2 - X =
[[1 1]
 [1 1]]
```

```
2 * X =  
[[2 2]  
 [2 2]]
```

```
X / 2 =  
[[0.5 0.5]  
 [0.5 0.5]]
```

Para operações entre dois arrays de mesmo tamanho, basta aplicar a operação elemento a elemento (*elementwise*, em inglês) entre os arrays.

```
x = np.array([2, 4, 6], dtype=int)  
y = np.array([2, 2, 1], dtype=int)  
  
print("x =\n", x)  
print("\ny =\n", y)  
print("\nx + y =\n", x + y)  
print("\nx - y =\n", x - y)
```

```
x =  
[2 4 6]
```

```
y =  
[2 2 1]
```

```
x + y =  
[4 6 7]
```

```
x - y =  
[0 2 5]
```

```
X = np.array([[2, 4], [6, 8]], dtype=int)  
Y = np.array([[2, 2], [2, 2]], dtype=int)  
  
print("X =\n", X)  
print("\nY =\n", Y)  
print("\nX + Y =\n", X + Y)  
print("\nX - Y =\n", X - Y)
```

```
X =
```

```
[[2 4]
 [6 8]]
```

```
Y =
[[2 2]
 [2 2]]
```

```
X + Y =
[[ 4  6]
 [ 8 10]]
```

```
X - Y =
[[0 2]
 [4 6]]
```

As operações de multiplicação e divisão entre arrays são um pouco diferentes. São duas as possibilidades:

- Multiplicação e divisão elemento a elemento.
- Multiplicação entre arrays usando a lógica de matriz e “divisão” usando a lógica de matriz inversa.

Para realizar as operações *elementwise* basta utilizar os sinais usuais de `*` e `/`, independentemente do número de dimensões do array.

```
print('Vetores x e y:')
print('x =',x)
print('y =',y)
print("\n x * y =\n", x * y)
print("\n x / y =\n", x / y)

print('\nMatrizes X e Y:')
print('X =\n',X)
print('Y =\n',Y)
print("\n X * Y =\n", X * Y)
print("\n X / Y =\n", X / Y)
```

```
Vetores x e y:
x = [2 4 6]
y = [2 2 1]
```

```
x * y =
```

```
[4 8 6]
```

```
x / y =  
[1. 2. 6.]
```

Matrizes X e Y:

```
X =  
[[2 4]  
 [6 8]]
```

```
Y =  
[[2 2]  
 [2 2]]
```

```
X * Y =  
[[ 4  8]  
 [12 16]]
```

```
X / Y =  
[[1. 2.]  
 [3. 4.]]
```

A **multiplicação** de matriz com matriz, do jeito que a gente conhece do Ensino Médio é feita usando o símbolo @ (ou através da função `np.dot()`):

```
X= np.array([[1, 2], [3, 4]], dtype=int)  
Y= np.array([[10, 20], [30, 40]], dtype=int)  
  
print('X =\n',X)  
print('Y =\n',Y)  
print('\nX * Y =\n', X @ Y)  
print('\nX * Y =\n', np.dot(X,Y))
```

```
X =  
[[1 2]  
 [3 4]]  
Y =  
[[10 20]  
 [30 40]]
```

```
X * Y =  
[[ 70 100]  
 [150 220]]
```

```
X * Y =  
[[ 70 100]  
 [150 220]]
```

Para calcular a inversa de uma matriz utilizando o NumPy é preciso um pouco mais de estrutura e de conhecimento acerca do subpacote `linalg` que nos traz funções para realizar operações de álgebra linear. Falaremos disso já já!

7.4.2 Funções universais

As funções universais no Numpy são funções matemáticas simples. É apenas um termo que demos às funções matemáticas na biblioteca Numpy, que cobrem uma ampla variedade de operações. Essas funções incluem funções trigonométricas padrão, funções para operações aritméticas, manipulação de números complexos, funções estatísticas, etc. Essas funções possuem como características principais:

- Elas executam operações de array elemento por elemento.
- Elas suportam vários recursos, como conversão de tipos.
- As funções universais são objetos que pertencem à classe `numpy.ufunc`.
- As funções do Python também podem ser criadas como uma função universal usando a função da biblioteca `frompyfunc`.
- Algumas funções universais são chamadas automaticamente quando o operador aritmético correspondente é usado em arrays. Por exemplo, quando a adição de dois arrays é executada em elementos usando o operador '+', então `np.add()` é chamado internamente.

Dentre as principais funções universais matemáticas estão:

- `sin`, `cos`, `tan`: calcular seno, cosseno e tangente de ângulos.
- `hypot`: calcule a hipotenusa do triângulo retângulo dado.
- `arcsinh`, `arcosh`, `arctanh`: calcular seno hiperbólico inverso, cosseno e tangente.
- `deg2rad`: converter grau em radianos.
- `rad2deg`: converter radianos em graus.

Dentre as principais funções universais estatísticas estão:

- `amin`, `amax`: retorna o mínimo ou máximo de um array ou ao longo de um eixo.
- `ptp`: retorna o intervalo de valores (máximo-mínimo) de um array ou ao longo de um eixo.
- `sum`: retorna a soma de valores de um array ao longo de um eixo.
- `percentile(a, p, eixo)`: calcular o p-ésimo percentil da matriz ou ao longo do eixo especificado.
- `median`: calcular a mediana dos dados ao longo do eixo especificado.

- **mean**: calcular a média dos dados ao longo do eixo especificado.
- **var**: calcular a variância de dados ao longo do eixo especificado.
- **log**: calcular o log dos dados ao longo do eixo especificado.

Alguns exemplos:

```
angulos_notaveis_deg = np.array([30,45,60])
angulos_notaveis_rad = np.deg2rad(angulos_notaveis_deg)

seno_notaveis = [np.round(elem,2) for elem in np.sin(angulos_notaveis_rad)]
cosseno_notaveis = [np.round(elem,2) for elem in np.cos(angulos_notaveis_rad)]
tangente_notaveis = [np.round(elem,2) for elem in np.tan(angulos_notaveis_rad)]

print('Seno, cosseno e tangente de 30 graus: ', seno_notaveis[0], ', ', cosseno_notaveis[0], ' e '
print('Seno, cosseno e tangente de 45 graus: ', seno_notaveis[1], ', ', cosseno_notaveis[1], ' e '
print('Seno, cosseno e tangente de 60 graus: ', seno_notaveis[2], ', ', cosseno_notaveis[2], ' e ')
```

```
Seno, cosseno e tangente de 30 graus:  0.5 ,  0.87 e  0.58
Seno, cosseno e tangente de 45 graus:  0.71 ,  0.71 e  1.0
Seno, cosseno e tangente de 60 graus:  0.87 ,  0.5 e  1.73
```

```
x = np.array([1,2,3,4,5])

print('Array x = ',x)
print('\nMínimo de x = ',np.amin(x))
print('Máximo de x = ',np.amax(x))
print('Intervalo de x = ',np.ptp(x))
print('Soma de x = ',np.sum(x))
print('Média de x = ',np.mean(x))
print('Log de x = ',[np.round(elem,2) for elem in np.log(x)])
```

```
Array x =  [1 2 3 4 5]
```

```
Mínimo de x =  1
Máximo de x =  5
Intervalo de x =  4
Soma de x =  15
Média de x =  3.0
Log de x =  [np.float64(0.0), np.float64(0.69), np.float64(1.1), np.float64(1.39), np.float64(1.61)]
```

7.4.3 Arrays e listas

Agora que já vimos um pouco de operações básicas, podemos entender um pouco melhor a diferença de desempenho entre arrays e listas. Considere um array de dez milhões de números inteiros e uma lista equivalente:

```
my_array = np.arange(10000000)
my_list = list(range(10000000))
```

Agora vamos multiplicar, elemento a elemento, todos os números por 2 e salvar o resultado correspondente. Utilizaremos a função `time` do pacote `time` para fazer a medição do tempo utilizado pelos dois métodos.

```
import time

# arrays
start_array = time.time()
my_array2 = my_array * 2
end_array = time.time()

# listas
start_lista = time.time()
my_list2 = [x * 2 for x in my_list]
end_lista = time.time()

ratio = (end_lista - start_lista) / (end_array - start_array)
```

Qual abordagem será que levou menos tempo?

```
print('Tempo necessário para a realização dos cálculos utilizando arrays: {:.4f} segundos'.format(end_array - start_array))
print('Tempo necessário para a realização dos cálculos utilizando listas: {:.4f} segundos'.format(end_lista - start_lista))
print('\nA abordagem de listas demorou {:.0f}x mais tempo! Esqueça listas e use arrays ;)'.format(ratio))
```

```
Tempo necessário para a realização dos cálculos utilizando arrays: 0.0124 segundos
Tempo necessário para a realização dos cálculos utilizando listas: 0.2891 segundos
```

```
A abordagem de listas demorou 22x mais tempo! Esqueça listas e use arrays ;)
```

7.5 Aplicação: solução de sistema de equações lineares

Chega de exemplos vazios, vamos usar o NumPy para resolver um problema concreto e que nos é muito familiar: a solução de sistemas de equações lineares. Considere o sistema linear de equações dado por

$$A \cdot \vec{x} = \vec{b}$$

tal que A é a matriz de coeficientes, $\vec{x}$ é o vetor de incógnitas e $\vec{b}$ o vetor de constantes.

7.5.1 Algoritmo de eliminação de Gauss-Jordan

Segundo [Hubbard e Hubbard \(2015\)](#), uma matriz de coeficientes A é representada em sua forma escalonada reduzida por linhas (*reduced row echelon form*, em inglês) se

- Em toda e qualquer linha, a primeira entrada diferente de zero é igual a 1 (1 *pivotal*).
- O 1 *pivotal* de uma linha mais abaixo está sempre à direita de um outro 1 *pivotal* de alguma linha acima.
- Em toda e qualquer coluna que contém um 1 *pivotal*, todas as outras entradas são iguais a zero.
- Toda linha contendo apenas zeros está no fim da matriz.

À partir dessa definição, é possível mostrar que para qualquer matriz A , existe uma matriz $\tilde{A}$ na forma escalonada reduzida por linhas que pode ser obtida à partir de operações elementares nas linhas de A . Além disso, é possível mostrar também que $\tilde{A}$ é única. Ao algoritmo utilizado para encontrar $\tilde{A}$ é dado o nome de **Algoritmo de Eliminação de Gauss-Jordan**.

ALGORITMO DE ELIMINAÇÃO DE GAUSS-JORDAN

Para levar uma matriz A a sua forma escalonada reduzida por linhas $\tilde{A}$ devemos seguir os seguintes passos:

1. Encontre a primeira coluna que não é composta apenas de zeros, chame isso de primeira coluna pivotal e chame sua primeira entrada diferente de zero de pivô. Se o pivô não for na primeira linha, mova a linha que a contém para o topo da matriz.
2. Divida a primeira linha inteira pelo pivô, de modo que a primeira entrada da primeira coluna pivotal seja igual a 1.
3. Adicione múltiplos apropriados da primeira linha às outras linhas para garantir que todas as outras entradas da primeira coluna pivotal sejam iguais a 0. O 1 na primeira coluna é agora um pivô 1.

- Escolha a próxima coluna que contém pelo menos uma entrada diferente de zero abaixo da primeira linha e coloque a linha que contém o novo pivô na posição da segunda linha. Faça do pivô um pivô 1: divida a linha inteira pelo pivô e adicione múltiplos apropriados desta linha às outras linhas abaixo, para tornar todas as outras entradas desta coluna iguais a 0.

- Repita o processo até que a matriz esteja em sua forma escalonada reduzida por linhas.

Assuma o caso em que a matriz de coeficiente A e o vetor de constantes b são tais que a matriz ampliada é dada por:

$$\left[\begin{array}{ccc|c} 2 & 2 & 1 & 1 \\ 1 & 3 & 1 & 2 \\ 1 & 2 & 2 & -1 \end{array} \right]$$

- Passo 1: Defina a matriz M

```
M = np.array([(2, 2, 1, 1),
              (1, 3, 1, 2),
              (1, 2, 2, -1)], dtype=float)
print('Matriz ampliada =\n', M)
```

```
Matriz ampliada =
[[ 2.  2.  1.  1.]
 [ 1.  3.  1.  2.]
 [ 1.  2.  2. -1.]]
```

- Passo 2: Divida a linha 1 pelo primeiro elemento da primeira linha

```
M[0,:] = M[0,:] / M[0,0]
print(M)
```

```
[[ 1.  1.  0.5  0.5]
 [ 1.  3.  1.  2. ]
 [ 1.  2.  2. -1. ]]
```

- Passo 3: Subtraia a linha 1 da linha 2 e da linha 3

```
M[1,:] = M[1,:] - M[0,:]
M[2,:] = M[2,:] - M[0,:]
print(M)
```

```
[[ 1.  1.  0.5  0.5]
 [ 0.  2.  0.5  1.5]
 [ 0.  1.  1.5 -1.5]]
```

- Passo 4: Divida a linha 2 pelo segundo elemento da segunda linha

```
M[1,:] = M[1,:] / M[1,1]

print(M)
```

```
[[ 1.  1.  0.5  0.5 ]
 [ 0.  1.  0.25 0.75]
 [ 0.  1.  1.5 -1.5 ]]
```

- Passo 5: Subtraia a linha 2 da linha 3

```
M[2,:] = M[2,:] - M[1,:]

print(M)
```

```
[[ 1.  1.  0.5  0.5 ]
 [ 0.  1.  0.25 0.75]
 [ 0.  0.  1.25 -2.25]]
```

- Passo 6: Divida a linha 3 pelo terceiro elemento da terceira linha

```
M[2,:] = M[2,:] / M[2,2]

print(M)
```

```
[[ 1.  1.  0.5  0.5 ]
 [ 0.  1.  0.25 0.75]
 [ 0.  0.  1.  -1.8 ]]
```

- Passo 7: Multiplique a linha 3 pelo terceiro elemento da segunda linha e subtraia da linha 2

```
M[1,:] = M[1,:] - M[1,2] * M[2,:]

print(M)
```

```
[[ 1.  1.  0.5  0.5]
 [ 0.  1.  0.  1.2]
 [ 0.  0.  1. -1.8]]
```

- Passo 8: Multiplique a linha 3 pelo terceiro elemento da primeira linha e subtraia da linha 1

```
M[0,:] = M[0,:] - M[0,2] * M[2,:]
print(M)
```

```
[[ 1.  1.  0.  1.4]
 [ 0.  1.  0.  1.2]
 [ 0.  0.  1. -1.8]]
```

- Passo 9: Subtraia a linha 2 da linha 1

```
M[0,:] = M[0,:] - M[1,:]
print(M)
```

```
[[ 1.  0.  0.  0.2]
 [ 0.  1.  0.  1.2]
 [ 0.  0.  1. -1.8]]
```

Temos a nossa matriz em sua forma escalonada reduzida por linhas! A solução do sistema é tal que:

```
print('A solução de x1 é: {:.2f}'.format(M[0,3]))
print('A solução de x2 é: {:.2f}'.format(M[1,3]))
print('A solução de x3 é: {:.2f}'.format(M[2,3]))
```

```
A solução de x1 é: 0.20
A solução de x2 é: 1.20
A solução de x3 é: -1.80
```

Uma forma mais simples de chegar na forma escalonada reduzida por linhas é através do pacote SymPy e das funções `Matrix` e `rref`:

```
import sympy

M_sympy = sympy.Matrix([(2, 2, 1, 1),
                        (1, 3, 1, 2),
                        (1, 2, 2, -1)])

M_sympy.rref()[0]
```

$$\begin{bmatrix} 1 & 0 & 0 & \frac{1}{5} \\ 0 & 1 & 0 & \frac{1}{5} \\ 0 & 0 & 1 & -\frac{9}{5} \end{bmatrix}$$

7.5.2 Solução de sistemas exatamente identificados

Uma outra forma de resolver sistemas de equações exatamente identificados, quando o número de incógnitas é igual ao número de equações (i.e., matriz A é quadrada) é através da matriz inversa de A . Para o caso em que A^{-1} existe, o sistema é tal que

$$\vec{x} = A^{-1} \cdot \vec{b}$$

Para que seja possível fazer dessa forma, A deve ser uma matriz quadrada e seu determinante ser diferente de 0. Mais uma vez assumo o caso em que A e $\vec{b}$ são dados por:

$$A = \begin{bmatrix} 2 & 2 & 1 \\ 1 & 3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$$

O primeiro passo é conferir se a matrix A é quadrada. Podemos fazer isso usando a função `numpy.shape()`

```
b = np.array([1, 2, -1])
A = np.array([(2, 2, 1), (1, 3, 1), (1, 2, 2)])

np.shape(A)[0] == np.shape(A)[1]
```

True

O próximo passo é ver se o determinante da matriz é igual a 0 e para isso usamos um função do subpacote de álgebra linera do numpy, `numpy.linalg.det()`:

```
print('A =')
print(A)

print('\nDeterminante = ', np.round(np.linalg.det(A)))
```

```
A =
[[2 2 1]
 [1 3 1]
 [1 2 2]]
```

```
Determinante = 5.0
```

Como ambas as condições são satisfeitas, por fim basta calcular a inversa da matriz A com `numpy.linalg.inv()` e multiplicar por $\vec{b}$ para chegar nos valores de x , y e z que solucionam o sistema de equações.

```
print('A =')
print(A)

print("\nInversa de A = ")
print(np.linalg.inv(A))
```

```
A =
[[2 2 1]
 [1 3 1]
 [1 2 2]]
```

```
Inversa de A =
[[ 0.8 -0.4 -0.2]
 [-0.2  0.6 -0.2]
 [-0.2 -0.4  0.8]]
```

```
solution = np.linalg.inv(A) @ b
solution
```

```
array([ 0.2,  1.2, -1.8])
```

```
print('A solução de x1 é: {:.2f}'.format(solution[0]))
print('A solução de x2 é: {:.2f}'.format(solution[1]))
print('A solução de x3 é: {:.2f}'.format(solution[2]))
```

A solução de x1 é: 0.20
A solução de x2 é: 1.20
A solução de x3 é: -1.80

Uma última alternativa para resolver o sistema $A \cdot \vec{x} = \vec{b}$, é utilizar a função `solve` do subpacote de álgebra linear do NumPy. Essa função faz o processo de resolução do sistema de forma direta e nos cospe o vetor de resultado.

```
np.linalg.solve(A,b)
```

```
array([ 0.2,  1.2, -1.8])
```

7.6 Exercícios

1. X
2. X

8 Gestão e análise de dados

8.1 Introdução

8.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- `numpy`
- `pandas`
- `os`

```
import numpy as np
import pandas as pd
import os
```

8.2 O que é o Pandas?

Pandas é uma biblioteca que contém objetos específicos e ferramentas próprias a esses objetos que permitem realizar limpeza, organização e análise de dados de forma bastante eficiente no Python. Não à toa, é uma das bibliotecas mais utilizadas na linguagem hoje em dia, um pré-requisito para a maior parte das aplicações de ciência dos dados e aprendizado de máquina.

Embora o Pandas se utilize em grande medida da estrutura e lógica do NumPy e seus arrays, a biblioteca é projetada para trabalhar com dados tabulares e heterogêneos. Como já vimos, o NumPy é mais adequado para trabalhar com dados homogêneos e em sua maior parte numéricos.

São dois os objetos particulares ao Pandas: **Series** e **DataFrame**.

Você pode pensar em uma série como uma “coluna” de dados, como uma sequência de observações em uma única variável. Por outro lado, um DataFrame é um objeto bidimensional para armazenar colunas de dados relacionadas, assim como uma tabela, com linhas e colunas, no Microsoft Excel.

8.3 Objetos no Pandas

Vamos começar criando duas listas: uma contendo o número de pessoas ocupadas (em mil pessoas) no Brasil de 2012 a 2021 e outra contendo exatamente o intervalo de anos.

```
personas_ocupadas = [90593,92170,92962,92366,90174,92228,93534,95515,87225,95747]
anos = list(range(2012,2021+1))

print(personas_ocupadas)
print(anos)
```

```
[90593, 92170, 92962, 92366, 90174, 92228, 93534, 95515, 87225, 95747]
[2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021]
```

Para criar uma série basta utilizar a função `pandas.Series`, que recebe como argumento os valores da série e os índices da série, que serão justamente os valores que identificam cada linha.

```
series_personas_ocup = pd.Series(data=personas_ocupadas,index=anos)

print(type(series_personas_ocup),'\n')
print(series_personas_ocup)
```

```
<class 'pandas.core.series.Series'>
```

```
2012    90593
2013    92170
2014    92962
2015    92366
2016    90174
2017    92228
2018    93534
2019    95515
2020    87225
2021    95747
dtype: int64
```

Já para o caso de um `DataFrame`, podemos criá-lo através da função `pandas.DataFrame`. Essa função recebe os dados que irão compor as colunas de dados de diversas formas, mas uma das mais intuitivas é através da utilização de dicionários. Seguindo o exemplo anterior, vamos criar

um DataFrame com as informações de pessoas ocupadas, desocupadas, dentro e fora da força de trabalho.

Primeiro passo é definir o dicionário com esses dados.

```
dados = {
    'ocupadas': [90593,92170,92962,92366,90174,92228,93534,95515,87225,95747],
    'desocupadas': [6730,6151,6555,9222,12476,12453,12413,11903,14412,12011],
    'na forca': [97322,98321,99516,101588,102650,104682,105947,107418,101637,107758],
    'fora da forca': [58007,59244,60162,60092,60953,60777,61299,61579,69042,64525]
}

print(dados)
```

```
{'ocupadas': [90593, 92170, 92962, 92366, 90174, 92228, 93534, 95515, 87225, 95747], 'desocupadas': [6730, 6151, 6555, 9222, 12476, 12453, 12413, 11903, 14412, 12011], 'na forca': [97322, 98321, 99516, 101588, 102650, 104682, 105947, 107418, 101637, 107758], 'fora da forca': [58007, 59244, 60162, 60092, 60953, 60777, 61299, 61579, 69042, 64525]}
```

Agora basta definir o DataFrame usando como argumento esse dicionários `dados`.

```
dados_emprego = pd.DataFrame(data=dados,index=anos)

print(type(dados_emprego),'\n')
dados_emprego
```

```
<class 'pandas.core.frame.DataFrame'>
```

	ocupadas	desocupadas	na forca	fora da forca
2012	90593	6730	97322	58007
2013	92170	6151	98321	59244
2014	92962	6555	99516	60162
2015	92366	9222	101588	60092
2016	90174	12476	102650	60953
2017	92228	12453	104682	60777
2018	93534	12413	105947	61299
2019	95515	11903	107418	61579
2020	87225	14412	101637	69042
2021	95747	12011	107758	64525

8.4 *DataFrames*: o coração do Pandas

8.4.1 Importação de dados externos

Agora que já apresentamos os dois tipos de objetos e como criá-los na mão, vamos avançar e trabalhar com dados trazidos de algum arquivo externo. É comum nos depararmos com arquivos do tipo `.csv` e é com ele que trabalharemos no Pandas, embora o Pandas seja flexível e nos permita trazer para dentro do Python arquivos de várias terminações como `.txt`, `.xlsx`, `.json`, `.dta`, etc.

A ideia daqui para frente é trabalhar com os dados do site [Our World in Data](#) sobre número de casos de covid-19, mortes e excesso de mortes atrelados à doença, e número de doses de vacina aplicadas ao redor do mundo desde o início da pandemia, em fevereiro de 2020. Esses dados são atualizados semanalmente e disponibilizados na plataforma Kaggle. O arquivo específico com o qual trabalharemos está disponível no Moodle e deve ser baixado para a sua máquina local.

Mas antes de aprendermos como trabalhar com um dataframe e as funcionalidades que o Pandas nos traz precisamos ler o arquivo com os nossos dados. Começemos pela função `os.chdir` do pacote `os` para mudar o diretório base do Python para o diretório no qual o nosso arquivo se encontra:

```
import os

# Qual o diretório no qual o Python está trabalhando? A função getcwd() nos diz isso
print('Diretório inicial: {}'.format(os.getcwd()))

# Vamos mudar para o nosso diretório atual
os.chdir('D:/Dropbox/Projetos e trabalhos/FEAUSP/Cursos/EAE1106 - Metodos Computacionais para')
print('Diretório final: {}'.format(os.getcwd()))
```

```
Diretório inicial: D:\Dropbox\Projetos e trabalhos\FEAUSP\Cursos\EAE1106 - Metodos Computacionais para
Diretório final: D:\Dropbox\Projetos e trabalhos\FEAUSP\Cursos\EAE1106 - Metodos Computacionais para
```

Quais são os arquivos disponíveis nesse diretório? A função `os.listdir()` lista as pastas e arquivos disponíveis no diretório que estamos trabalhando.

```
os.listdir()

['df_resultado_aula_pandas.csv',
 'EAE1106 - Aula Fundamentos.html',
 'EAE1106 - Aula Fundamentos.pdf',
 'EAE1106 - Aula Funções.html',
```

```

'EAE1106 - Aula Funções.pdf',
'EAE1106 - Aula Instalação.html',
'EAE1106 - Aula Instalação.pdf',
'EAE1106 - Aula Iteração.html',
'EAE1106 - Aula Iteração.pdf',
'EAE1106 - Aula Matplotlib.html',
'EAE1106 - Aula Matplotlib.pdf',
'EAE1106 - Aula NumPy.html',
'EAE1106 - Aula NumPy.pdf',
'EAE1106 - Aula Objetos.html',
'EAE1106 - Aula Objetos.pdf',
'EAE1106 - Aula Pandas.html',
'EAE1106 - Aula Pandas.pdf',
'EAE1106_Fundamentos_slides.aux',
'EAE1106_Fundamentos_slides.log',
'EAE1106_Fundamentos_slides.nav',
'EAE1106_Fundamentos_slides.out',
'EAE1106_Fundamentos_slides.pdf',
'EAE1106_Fundamentos_slides.snm',
'EAE1106_Fundamentos_slides.synctex.gz',
'EAE1106_Fundamentos_slides.tex',
'EAE1106_Fundamentos_slides.toc',
'EAE1106_Introducao_slides.aux',
'EAE1106_Introducao_slides.log',
'EAE1106_Introducao_slides.nav',
'EAE1106_Introducao_slides.out',
'EAE1106_Introducao_slides.pdf',
'EAE1106_Introducao_slides.snm',
'EAE1106_Introducao_slides.synctex.gz',
'EAE1106_Introducao_slides.tex',
'EAE1106_Introducao_slides.toc',
'notebook_images',
'owid-covid-data_top10.csv']

```

Legal, Agora utilizaremos a função `pandas.read_csv` para ler o arquivo `owid-covid-data_top10.csv`. Esse arquivo contém as informações para os 10 países com o maior número reportado de casos de covid-19.

```

df_covid = pd.read_csv('owid-covid-data_top10.csv', sep=',', encoding='utf8')
type(df_covid)

```

`pandas.core.frame.DataFrame`

8.4.2 Características básicas

Alguns métodos específicos ao objeto DataFrame nos são incrivelmente úteis quando queremos ter uma leitura rápida do que acabamos de gerar.

- `df.info()` nos retorna algumas informações técnicas do dataframe `df`, como o formato dos dados presentes em cada coluna.
- `df.head(n)` e `df.tail(n)` nos retornam as `n` primeiras e `n` últimas linhas, respectivamente.
- `df.describe()` nos retorna uma tabela de estatísticas descritivas das colunas numéricas.

Vejamos o que esses métodos nos retornam no nosso caso:

```
df_covid.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7663 entries, 0 to 7662
```

```
Data columns (total 67 columns):
```

#	Column	Non-Null Count	Dtype
0	iso_code	7663 non-null	object
1	continent	7663 non-null	object
2	location	7663 non-null	object
3	date	7663 non-null	object
4	total_cases	7663 non-null	float64
5	new_cases	7641 non-null	float64
6	new_cases_smoothed	7484 non-null	float64
7	total_deaths	7345 non-null	float64
8	new_deaths	7327 non-null	float64
9	new_deaths_smoothed	7173 non-null	float64
10	total_cases_per_million	7663 non-null	float64
11	new_cases_per_million	7641 non-null	float64
12	new_cases_smoothed_per_million	7484 non-null	float64
13	total_deaths_per_million	7345 non-null	float64
14	new_deaths_per_million	7327 non-null	float64
15	new_deaths_smoothed_per_million	7173 non-null	float64
16	reproduction_rate	7292 non-null	float64
17	icu_patients	4278 non-null	float64
18	icu_patients_per_million	4278 non-null	float64
19	hosp_patients	2861 non-null	float64
20	hosp_patients_per_million	2861 non-null	float64
21	weekly_icu_admissions	1377 non-null	float64
22	weekly_icu_admissions_per_million	1377 non-null	float64

23	weekly_hosp_admissions	3603	non-null	float64
24	weekly_hosp_admissions_per_million	3603	non-null	float64
25	new_tests	5755	non-null	float64
26	total_tests	6056	non-null	float64
27	total_tests_per_thousand	6056	non-null	float64
28	new_tests_per_thousand	5755	non-null	float64
29	new_tests_smoothed	7116	non-null	float64
30	new_tests_smoothed_per_thousand	7116	non-null	float64
31	positive_rate	5812	non-null	float64
32	tests_per_case	5812	non-null	float64
33	tests_units	7186	non-null	object
34	total_vaccinations	3950	non-null	float64
35	people_vaccinated	3938	non-null	float64
36	people_fully_vaccinated	3860	non-null	float64
37	total_boosters	2070	non-null	float64
38	new_vaccinations	3858	non-null	float64
39	new_vaccinations_smoothed	4181	non-null	float64
40	total_vaccinations_per_hundred	3950	non-null	float64
41	people_vaccinated_per_hundred	3938	non-null	float64
42	people_fully_vaccinated_per_hundred	3860	non-null	float64
43	total_boosters_per_hundred	2070	non-null	float64
44	new_vaccinations_smoothed_per_million	4181	non-null	float64
45	new_people_vaccinated_smoothed	4181	non-null	float64
46	new_people_vaccinated_smoothed_per_hundred	4181	non-null	float64
47	stringency_index	7580	non-null	float64
48	population	7663	non-null	float64
49	population_density	7663	non-null	float64
50	median_age	7663	non-null	float64
51	aged_65_older	7663	non-null	float64
52	aged_70_older	7663	non-null	float64
53	gdp_per_capita	7663	non-null	float64
54	extreme_poverty	5348	non-null	float64
55	cardiovasc_death_rate	7663	non-null	float64
56	diabetes_prevalence	7663	non-null	float64
57	female_smokers	7663	non-null	float64
58	male_smokers	7663	non-null	float64
59	handwashing_facilities	766	non-null	float64
60	hospital_beds_per_thousand	7663	non-null	float64
61	life_expectancy	7663	non-null	float64
62	human_development_index	7663	non-null	float64
63	excess_mortality_cumulative_absolute	692	non-null	float64
64	excess_mortality_cumulative	692	non-null	float64
65	excess_mortality	692	non-null	float64

```
66 excess_mortality_cumulative_per_million      692 non-null      float64
dtypes: float64(62), object(5)
memory usage: 5.7 MB
```

```
df_covid.head(2) # método para apresentar apenas as primeiras duas linhas do dataframe
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	total_deaths
0	BRA	South America	Brazil	2020-02-26	1.0	1.0	NaN	NaN
1	BRA	South America	Brazil	2020-02-27	1.0	0.0	NaN	NaN

Ué, mas não são 67 colunas? Onde estão todas elas? Por padrão o pandas mostra apenas uma parcela do total, para alterar essas opções basta utilizar a função `pd.set_option` e mudar o argumento relacionado ao número máximo de colunas.

```
pd.set_option('display.max_columns', 100)
df_covid.head(2)
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	total_deaths
0	BRA	South America	Brazil	2020-02-26	1.0	1.0	NaN	NaN
1	BRA	South America	Brazil	2020-02-27	1.0	0.0	NaN	NaN

```
df_covid.describe()
```

	total_cases	new_cases	new_cases_smoothed	total_deaths	new_deaths	new_deaths_smoothed
count	7.663000e+03	7.641000e+03	7484.000000	7345.000000	7327.000000	7173.000000
mean	7.413453e+06	3.248569e+04	32590.349244	143683.851464	420.451071	423.776663
std	1.200763e+07	6.990790e+04	66130.926542	193260.956591	656.855964	611.463015
min	1.000000e+00	0.000000e+00	0.000000	1.000000	0.000000	0.000000
25%	2.297820e+05	1.333000e+03	1440.107250	10145.000000	23.000000	27.857000
50%	2.702681e+06	9.921000e+03	11149.928500	83976.000000	161.000000	177.714000
75%	7.906961e+06	3.357100e+04	35266.357250	149512.000000	535.500000	567.143000
max	7.926573e+07	1.368167e+06	802518.429000	958437.000000	4529.000000	4190.000000

Uma outra coisa que pode ser bastante interessante é saber quantas vezes os valores de determinada variável se repetem. Quantas vezes cada país da base de dados aparece no DataFrame?

```
df_covid['location'].value_counts()
```

```
location
Japan          774
South Korea    774
United States  774
France         772
Germany        769
India          766
Italy          765
Russia         765
United Kingdom 765
Brazil         739
Name: count, dtype: int64
```

```
df_covid['location'].value_counts(normalize=True, dropna=False)
```

```
location
Japan          0.101005
South Korea    0.101005
United States  0.101005
France         0.100744
Germany        0.100352
India          0.099961
Italy          0.099830
Russia         0.099830
United Kingdom 0.099830
Brazil         0.096437
Name: proportion, dtype: float64
```

Por fim, podemos ordenar o DataFrame de acordo com uma coluna ou um conjunto de colunas. Qual eram os países com o maior número de novos casos de covid reportados no dia 1 de março de 2022?

```
df_covid[df_covid['date']=='2022-03-01'].sort_values(by=['new_cases'], ascending=False)
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_sm
6119	KOR	Asia	South Korea	2022-03-01	3492686.0	219237.0	166214.857
6884	GBR	Europe	United Kingdom	2022-03-01	19036570.0	99485.0	41712.714

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed
5345	RUS	Europe	Russia	2022-03-01	16257688.0	96092.0	118164.000
1506	FRA	Europe	France	2022-03-01	22835320.0	79794.0	54468.000
4580	JPN	Asia	Japan	2022-03-01	5071249.0	65368.0	65592.143
2275	DEU	Europe	Germany	2022-03-01	14974722.0	62096.0	135435.286
3806	ITA	Europe	Italy	2022-03-01	12829972.0	47136.0	39339.429
7658	USA	North America	United States	2022-03-01	79091361.0	47031.0	62331.429
734	BRA	South America	Brazil	2022-03-01	28818850.0	22279.0	65271.286
3041	IND	Asia	India	2022-03-01	42938599.0	7554.0	10224.000

Note que no código acima utilizamos uma forma de filtragem dos dados com base em uma condição, que é a data ser igual a 01 de março de 2022. Vamos ir com calma e mostrar todos os passos para realizar indexação, seleção e filtragem em dataframes agora.

8.4.3 Indexação, seleção e filtragem

Podemos selecionar um intervalo particular de linhas do DataFrame usando a lógica de indexação de sempre.

```
df_covid[0:2]
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	total_deaths
0	BRA	South America	Brazil	2020-02-26	1.0	1.0	NaN	NaN
1	BRA	South America	Brazil	2020-02-27	1.0	0.0	NaN	NaN

Utilizando a função `.iloc` podemos selecionar linhas e colunas com os índices numéricos. E se quisermos, por exemplo, comparar o número total de casos de todos os países ao longo do tempo, mantendo apenas as variáveis `location`, `date` e `total_cases`?

```
df_covid.iloc[:,2:5]
```

	location	date	total_cases
0	Brazil	2020-02-26	1.0
1	Brazil	2020-02-27	1.0
2	Brazil	2020-02-28	1.0
3	Brazil	2020-02-29	2.0
4	Brazil	2020-03-01	2.0

	location	date	total_cases
...	...	...	...
7658	United States	2022-03-01	79091361.0
7659	United States	2022-03-02	79143716.0
7660	United States	2022-03-03	79196008.0
7661	United States	2022-03-04	79250509.0
7662	United States	2022-03-05	79265726.0

Podemos fazer exatamente a mesma coisa com a função `.loc` e utilizando uma mistura de índices numéricos para as linhas e seleção das colunas através de seus nomes.

```
df_covid.loc[:, ['location', 'date', 'total_cases']]
```

	location	date	total_cases
0	Brazil	2020-02-26	1.0
1	Brazil	2020-02-27	1.0
2	Brazil	2020-02-28	1.0
3	Brazil	2020-02-29	2.0
4	Brazil	2020-03-01	2.0
...	...	...	...
7658	United States	2022-03-01	79091361.0
7659	United States	2022-03-02	79143716.0
7660	United States	2022-03-03	79196008.0
7661	United States	2022-03-04	79250509.0
7662	United States	2022-03-05	79265726.0

Uma terceira forma de fazer esse exercício de seleção de uma parte de um DataFrame já criado é através da criação de um novo DataFrame que mantenha apenas as colunas `location`, `date` e `total_cases`.

```
columns_to_keep = ['location', 'date', 'total_cases']
df_covid2 = df_covid[columns_to_keep]

df_covid2.head(2)
```

	location	date	total_cases
0	Brazil	2020-02-26	1.0
1	Brazil	2020-02-27	1.0

location	date	total_cases
----------	------	-------------

Podemos inclusive alterar o nome das colunas desse novo DataFrame de forma bem direta.

```
df_covid2.columns = ['localizacao', 'data', 'casos']
df_covid2.head()
```

	localizacao	data	casos
0	Brazil	2020-02-26	1.0
1	Brazil	2020-02-27	1.0
2	Brazil	2020-02-28	1.0
3	Brazil	2020-02-29	2.0
4	Brazil	2020-03-01	2.0

É possível redefinir o índice desse novo DataFrame para ser igual ao país.

```
df_covid2.set_index('localizacao', inplace=True)
df_covid2.head(2)
```

	data	casos
localizacao		
Brazil	2020-02-26	1.0
Brazil	2020-02-27	1.0

Por fim, podemos selecionar parte específica de um DataFrame utilizando testes booleanos sobre valores de linhas e colunas.

- Selecionar apenas os dados brasileiros

```
df_covid[df_covid['location']=='Brazil']
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	t
0	BRA	South America	Brazil	2020-02-26	1.0	1.0	NaN	N
1	BRA	South America	Brazil	2020-02-27	1.0	0.0	NaN	N
2	BRA	South America	Brazil	2020-02-28	1.0	0.0	NaN	N
3	BRA	South America	Brazil	2020-02-29	2.0	1.0	NaN	N

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	t
4	BRA	South America	Brazil	2020-03-01	2.0	0.0	NaN	N
...	...	...	...	...	...	...	...	...
734	BRA	South America	Brazil	2022-03-01	28818850.0	22279.0	65271.286	6
735	BRA	South America	Brazil	2022-03-02	28846495.0	27645.0	50451.286	6
736	BRA	South America	Brazil	2022-03-03	28906672.0	60177.0	45348.143	6
737	BRA	South America	Brazil	2022-03-04	28978052.0	71380.0	42625.857	6
738	BRA	South America	Brazil	2022-03-05	29040800.0	62748.0	41606.857	6

- Selecionar os dados brasileiros e apenas à partir do momento em que o número de casos se tornou maior do que 10 milhões

```
df_covid[(df_covid['location']=='Brazil') & (df_covid['total_cases']>=10000000)]
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	t
358	BRA	South America	Brazil	2021-02-18	10039804.0	53479.0	45188.143	2
359	BRA	South America	Brazil	2021-02-19	10091961.0	52157.0	45647.429	2
360	BRA	South America	Brazil	2021-02-20	10144332.0	52371.0	46721.000	2
361	BRA	South America	Brazil	2021-02-21	10173817.0	29485.0	46544.143	2
362	BRA	South America	Brazil	2021-02-22	10205663.0	31846.0	47406.714	2
...	...	...	...	...	...	...	...	...
734	BRA	South America	Brazil	2022-03-01	28818850.0	22279.0	65271.286	6
735	BRA	South America	Brazil	2022-03-02	28846495.0	27645.0	50451.286	6
736	BRA	South America	Brazil	2022-03-03	28906672.0	60177.0	45348.143	6
737	BRA	South America	Brazil	2022-03-04	28978052.0	71380.0	42625.857	6
738	BRA	South America	Brazil	2022-03-05	29040800.0	62748.0	41606.857	6

- Selecionar os dados do Brasil e Estados Unidos

```
df_covid[df_covid['iso_code'].isin(['BRA','USA'])]
```

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed	t
0	BRA	South America	Brazil	2020-02-26	1.0	1.0	NaN	
1	BRA	South America	Brazil	2020-02-27	1.0	0.0	NaN	
2	BRA	South America	Brazil	2020-02-28	1.0	0.0	NaN	
3	BRA	South America	Brazil	2020-02-29	2.0	1.0	NaN	
4	BRA	South America	Brazil	2020-03-01	2.0	0.0	NaN	
...	...	...	...	...	...	...	...	
7658	USA	North America	United States	2022-03-01	79091361.0	47031.0	62331.429	

	iso_code	continent	location	date	total_cases	new_cases	new_cases_smoothed
7659	USA	North America	United States	2022-03-02	79143716.0	52355.0	57824.857
7660	USA	North America	United States	2022-03-03	79196008.0	52292.0	55673.857
7661	USA	North America	United States	2022-03-04	79250509.0	54501.0	52764.286
7662	USA	North America	United States	2022-03-05	79265726.0	15217.0	47989.857

8.4.4 Operações com colunas numéricas

Criar novas colunas a partir de operações sobre aquelas que já existem é bastante simples. Além disso, vamos manter apenas os dados para o Brasil para melhor visualizar os resultados.

```
df_covid_br = df_covid[df_covid['iso_code']=='BRA'].reset_index()
```

Agora já podemos brincar com as colunas numéricas e mostrar o resultado dos cálculos.

- Coluna com o aumento percentual de casos a cada dia

```
df_covid_br['new_cases_percentage'] = df_covid_br['new_cases'] / df_covid_br['total_cases']

columns_to_show = ['date', 'total_cases', 'new_cases', 'new_cases_percentage']
df_covid_br[columns_to_show]
```

	date	total_cases	new_cases	new_cases_percentage
0	2020-02-26	1.0	1.0	1.000000
1	2020-02-27	1.0	0.0	0.000000
2	2020-02-28	1.0	0.0	0.000000
3	2020-02-29	2.0	1.0	0.500000
4	2020-03-01	2.0	0.0	0.000000
...	...	...	...	...
734	2022-03-01	28818850.0	22279.0	0.000773
735	2022-03-02	28846495.0	27645.0	0.000958
736	2022-03-03	28906672.0	60177.0	0.002082
737	2022-03-04	28978052.0	71380.0	0.002463
738	2022-03-05	29040800.0	62748.0	0.002161

- Coluna com o log do total de casos

```
import numpy as np
df_covid_br['log_total_cases'] = np.log(df_covid_br['total_cases'])

columns_to_show = ['date', 'total_cases', 'log_total_cases']
df_covid_br[columns_to_show]
```

	date	total_cases	log_total_cases
0	2020-02-26	1.0	0.000000
1	2020-02-27	1.0	0.000000
2	2020-02-28	1.0	0.000000
3	2020-02-29	2.0	0.693147
4	2020-03-01	2.0	0.693147
...	...	...	...
734	2022-03-01	28818850.0	17.176540
735	2022-03-02	28846495.0	17.177499
736	2022-03-03	28906672.0	17.179583
737	2022-03-04	28978052.0	17.182049
738	2022-03-05	29040800.0	17.184212

- Coluna com o total de casos em milhares

```
df_covid_br['total_cases_th'] = df_covid_br['total_cases'] / 1000

columns_to_show = ['date', 'total_cases', 'total_cases_th']
df_covid_br[columns_to_show]
```

	date	total_cases	total_cases_th
0	2020-02-26	1.0	0.001
1	2020-02-27	1.0	0.001
2	2020-02-28	1.0	0.001
3	2020-02-29	2.0	0.002
4	2020-03-01	2.0	0.002
...	...	...	...
734	2022-03-01	28818850.0	28818.850
735	2022-03-02	28846495.0	28846.495
736	2022-03-03	28906672.0	28906.672
737	2022-03-04	28978052.0	28978.052
738	2022-03-05	29040800.0	29040.800

- Subtrair a média do número de novos casos da coluna de novos casos para encontrar o desvio em relação à média.

```
df_covid_br['new_cases_demean'] = df_covid_br['new_cases'] - df_covid_br['new_cases'].mean()

columns_to_show = ['date', 'new_cases', 'new_cases_demean']
df_covid_br[columns_to_show]
```

	date	new_cases	new_cases_demean
0	2020-02-26	1.0	-39260.489824
1	2020-02-27	0.0	-39261.489824
2	2020-02-28	0.0	-39261.489824
3	2020-02-29	1.0	-39260.489824
4	2020-03-01	0.0	-39261.489824
...	...	...	...
734	2022-03-01	22279.0	-16982.489824
735	2022-03-02	27645.0	-11616.489824
736	2022-03-03	60177.0	20915.510176
737	2022-03-04	71380.0	32118.510176
738	2022-03-05	62748.0	23486.510176

Para realizar outras operações simples de soma, subtração, multiplicação e divisão a lógica é a mesma, independente de ser uma operação entre colunas ou de uma coluna com um escalar. Para operações mais complexas, no entanto, o método `.apply` em conjunto com funções anônimas do tipo `lambda` constituem uma ferramenta poderosíssima.

- Elevar o número de novos casos ao quadrado e dividir o resultado por 1 milhão.

```
df_covid_br['total_cases_sq'] = df_covid_br['total_cases'].apply(lambda x: (x**2)/ 1e6)

columns_to_show = ['date', 'total_cases', 'total_cases_sq']
df_covid_br[columns_to_show]
```

	date	total_cases	total_cases_sq
0	2020-02-26	1.0	1.000000e-06
1	2020-02-27	1.0	1.000000e-06
2	2020-02-28	1.0	1.000000e-06
3	2020-02-29	2.0	4.000000e-06
4	2020-03-01	2.0	4.000000e-06

	date	total_cases	total_cases_sq
...	...	...	...
734	2022-03-01	28818850.0	8.305261e+08
735	2022-03-02	28846495.0	8.321203e+08
736	2022-03-03	28906672.0	8.355957e+08
737	2022-03-04	28978052.0	8.397275e+08
738	2022-03-05	29040800.0	8.433681e+08

Para visualizar melhor, vamos focar no último valor, representado pelo índice 738.

```
print(df_covid_br.loc[738,'total_cases'])
print(df_covid_br.loc[738,'total_cases_sq'])
print((df_covid_br.loc[738,'total_cases'] ** 2) / 1000000)
```

```
29040800.0
843368064.64
843368064.64
```

Dei apenas exemplos super simples, mas note a força que o método **apply** tem quando trabalhado em conjunto com as funções **lambda**. O céu é o limite.

8.4.5 Operações com colunas contendo strings

O Pandas também nos fornece métodos e funções bastante interessantes para trabalhar com dados do tipo **string**. Vamos começar com um exemplo simples: transformando a nossa coluna com o nome dos países para uma coluna com letras minúsculas apenas. Para trabalhar com essas funções/métodos precisamos primeiro dizer ao Pandas que a coluna de interesse receberá operações específicas a strings.

```
df_covid_br['lower_location'] = df_covid_br['location'].str.lower()

columns_to_show = ['iso_code','continent','location','lower_location','date']
df_covid_br[columns_to_show]
```

	iso_code	continent	location	lower_location	date
0	BRA	South America	Brazil	brazil	2020-02-26
1	BRA	South America	Brazil	brazil	2020-02-27
2	BRA	South America	Brazil	brazil	2020-02-28

	iso_code	continent	location	lower_location	date
3	BRA	South America	Brazil	brazil	2020-02-29
4	BRA	South America	Brazil	brazil	2020-03-01
...	...	...	...	...	...
734	BRA	South America	Brazil	brazil	2022-03-01
735	BRA	South America	Brazil	brazil	2022-03-02
736	BRA	South America	Brazil	brazil	2022-03-03
737	BRA	South America	Brazil	brazil	2022-03-04
738	BRA	South America	Brazil	brazil	2022-03-05

Podemos aplicar a maior parte dos métodos de strings que já vimos (`replace`, `startswith`, `strip`, `extract`, etc.) utilizando essa sintaxe. Uma outra operação interessante, para, por exemplo, separar em colunas distintas o primeiro e o último nome de uma pessoa é realizada através do método `split`. Vamos tentar fazer isso com a coluna do continente.

```
df_covid_br[['continent_1', 'continent_2']] = df_covid_br['continent'].str.split(pat=' ',n=1)

columns_to_show = ['iso_code','continent','continent_1','continent_2','location','lower_location']
df_covid_br[columns_to_show]
```

	iso_code	continent	continent_1	continent_2	location	lower_location	date
0	BRA	South America	South	America	Brazil	brazil	2020-02-26
1	BRA	South America	South	America	Brazil	brazil	2020-02-27
2	BRA	South America	South	America	Brazil	brazil	2020-02-28
3	BRA	South America	South	America	Brazil	brazil	2020-02-29
4	BRA	South America	South	America	Brazil	brazil	2020-03-01
...	...	...	...	...	...	...	...
734	BRA	South America	South	America	Brazil	brazil	2022-03-01
735	BRA	South America	South	America	Brazil	brazil	2022-03-02
736	BRA	South America	South	America	Brazil	brazil	2022-03-03
737	BRA	South America	South	America	Brazil	brazil	2022-03-04
738	BRA	South America	South	America	Brazil	brazil	2022-03-05

Vamos olhar com calma esse resultado e os argumentos que utilizamos dentro do método `split`.

- *pat*: determina os caracteres que serão utilizados para fazer a divisão do string.
- *n*: limita o número de divisões a serem feitas no string.
- *expand*: caso o valor desse argumento seja igual a `True`, a divisão do string irá gerar *n+1* novas colunas no dataframe.

- *regex*: determina se o padrão *pat*, utilizado para a divisão do string, deve ser entendido como uma expressão regular ou não.

Bacana né? Esse método `split` nos permite fazer muita coisa legal com colunas de texto!

Por fim, uma outra funcionalidade interessante é a possibilidade de criação de novas colunas de strings utilizando dicionários. Podemos, por exemplo, utilizar essa funcionalidade para criar uma nova coluna que mostre se o número de novos casos por milhão está no quartil superior naquele dia ou não.

```
df_covid_br['new_cases_per_million'].describe()
```

```
count      737.000000
mean       183.470531
std        173.332413
min         0.000000
25%        62.684000
50%       148.126000
75%       254.279000
max       1341.859000
Name: new_cases_per_million, dtype: float64
```

```
# criando coluna a partir de list comprehension
df_covid_br['quartil_superior_bool'] = [True if elem>=254.279 else False for elem in df_covid_br['new_cases_per_million']]

# criando coluna a partir de um dicionario
quartil_superior = {
    True: 'Acima do 3º quartil de novos casos por milhão',
    False: 'Abaixo'
}

df_covid_br['quartil_superior_cat'] = df_covid_br['quartil_superior_bool'].apply(lambda x: quartil_superior[x])
```

E qual é o resultado final?

```
df_covid_br[['date', 'new_cases_per_million', 'quartil_superior_bool', 'quartil_superior_cat']]
```

	date	new_cases_per_million	quartil_superior_bool	quartil_superior_cat
0	2020-02-26	0.005	False	Abaixo

	date	new_cases_per_million	quartil_superior_bool	quartil_superior_cat
1	2020-02-27	0.000	False	Abaixo
2	2020-02-28	0.000	False	Abaixo
3	2020-02-29	0.005	False	Abaixo
4	2020-03-01	0.000	False	Abaixo
...	...	...	...	...
734	2022-03-01	104.111	False	Abaixo
735	2022-03-02	129.186	False	Abaixo
736	2022-03-03	281.210	True	Acima do 3º quartil de novos casos por
737	2022-03-04	333.562	True	Acima do 3º quartil de novos casos por
738	2022-03-05	293.224	True	Acima do 3º quartil de novos casos por

8.4.6 Agrupamento e junção de DataFrames

Suponha que queiramos calcular médias mensais de novos casos e novos casos por milhão e salvar isso em um novo dataframe. Para essa operação podemos utilizar o famoso **groupby**. Vamos fazer isso para os dados brasileiros e norte-americanos de forma separada.

```
df_covid['ano_mes'] = df_covid['date'].str[0:7]
columns_to_keep = ['location', 'ano_mes', 'new_cases', 'new_cases_per_million']

df_covid_br = df_covid[df_covid['iso_code']=='BRA'][columns_to_keep]
df_covid_us = df_covid[df_covid['iso_code']=='USA'][columns_to_keep]

df_covid_br.head(2)
```

	location	ano_mes	new_cases	new_cases_per_million
0	Brazil	2020-02	1.0	0.005
1	Brazil	2020-02	0.0	0.000

```
df_covid_br_mensal = df_covid_br.groupby(by=['location', 'ano_mes'], as_index=False).mean()
df_covid_us_mensal = df_covid_us.groupby(by=['location', 'ano_mes'], as_index=False).mean()

df_covid_br_mensal
```

	location	ano_mes	new_cases	new_cases_per_million
0	Brazil	2020-02	0.500000	0.002500

	location	ano_mes	new_cases	new_cases_per_million
1	Brazil	2020-03	184.354839	0.861452
2	Brazil	2020-04	2715.666667	12.690400
3	Brazil	2020-05	13837.096774	64.661387
4	Brazil	2020-06	31597.517241	147.656517
5	Brazil	2020-07	40573.612903	189.602194
6	Brazil	2020-08	40141.225806	187.581516
7	Brazil	2020-09	30084.533333	140.586233
8	Brazil	2020-10	23238.645161	108.595065
9	Brazil	2020-11	26806.733333	125.268900
10	Brazil	2020-12	43195.709677	201.855258
11	Brazil	2021-01	49273.838710	230.258677
12	Brazil	2021-02	48198.928571	225.235607
13	Brazil	2021-03	71138.741935	332.434258
14	Brazil	2021-04	63723.733333	297.783600
15	Brazil	2021-05	60734.967742	283.817000
16	Brazil	2021-06	67080.266667	313.468767
17	Brazil	2021-07	43608.064516	203.782323
18	Brazil	2021-08	27840.322581	130.098935
19	Brazil	2021-09	18009.931034	84.161034
20	Brazil	2021-10	12331.064516	57.623548
21	Brazil	2021-11	9517.266667	44.474533
22	Brazil	2021-12	6183.741935	28.896806
23	Brazil	2022-01	102312.612903	478.111000
24	Brazil	2022-02	119037.178571	556.265607
25	Brazil	2022-03	48845.800000	228.258600

Note que fizemos um dataframe para o Brasil e outro para os Estados Unidos. Podíamos ter feito tudo junto? Podíamos. Mas fizemos dessa forma justamente para mostrar outra funcionalidade bastante importante quando trabalhamos com bases de dados: a junção de duas bases diferentes. Essa junção de bases distintas pode ser realizada utilizando duas funções distintas: `concat` e `merge`.

A função `concat` “soma” duas bases, criando uma nova base que apenas junta as linhas dos diferentes dataframes. Isso é muito útil, por exemplo, quando geramos dataframes dentro de um loop e queremos ir juntando cada novo dataframe a um grande e único.

```
df_covid_concat = pd.concat(objs=[df_covid_br_mensal, df_covid_us_mensal]).reset_index(drop=True)
df_covid_concat
```

	location	ano_mes	new_cases	new_cases_per_million
0	Brazil	2020-02	0.500000	0.002500
1	Brazil	2020-03	184.354839	0.861452
2	Brazil	2020-04	2715.666667	12.690400
3	Brazil	2020-05	13837.096774	64.661387
4	Brazil	2020-06	31597.517241	147.656517
5	Brazil	2020-07	40573.612903	189.602194
6	Brazil	2020-08	40141.225806	187.581516
7	Brazil	2020-09	30084.533333	140.586233
8	Brazil	2020-10	23238.645161	108.595065
9	Brazil	2020-11	26806.733333	125.268900
10	Brazil	2020-12	43195.709677	201.855258
11	Brazil	2021-01	49273.838710	230.258677
12	Brazil	2021-02	48198.928571	225.235607
13	Brazil	2021-03	71138.741935	332.434258
14	Brazil	2021-04	63723.733333	297.783600
15	Brazil	2021-05	60734.967742	283.817000
16	Brazil	2021-06	67080.266667	313.468767
17	Brazil	2021-07	43608.064516	203.782323
18	Brazil	2021-08	27840.322581	130.098935
19	Brazil	2021-09	18009.931034	84.161034
20	Brazil	2021-10	12331.064516	57.623548
21	Brazil	2021-11	9517.266667	44.474533
22	Brazil	2021-12	6183.741935	28.896806
23	Brazil	2022-01	102312.612903	478.111000
24	Brazil	2022-02	119037.178571	556.265607
25	Brazil	2022-03	48845.800000	228.258600
26	United States	2020-01	0.777778	0.002333
27	United States	2020-02	0.586207	0.001759
28	United States	2020-03	6195.290323	18.609226
29	United States	2020-04	29444.100000	88.443333
30	United States	2020-05	22969.967742	68.996452
31	United States	2020-06	28578.766667	85.844033
32	United States	2020-07	61294.612903	184.114871
33	United States	2020-08	48365.322581	145.278355
34	United States	2020-09	39903.133333	119.859833
35	United States	2020-10	61870.806452	185.845677
36	United States	2020-11	149008.733333	447.587733
37	United States	2020-12	211725.580645	635.974742
38	United States	2021-01	198457.129032	596.119355
39	United States	2021-02	86034.107143	258.426643
40	United States	2021-03	58529.064516	175.807742

	location	ano_mes	new_cases	new_cases_per_million
41	United States	2021-04	62970.333333	189.148333
42	United States	2021-05	29748.451613	89.357484
43	United States	2021-06	13276.700000	39.880100
44	United States	2021-07	42530.483871	127.751742
45	United States	2021-08	138009.000000	414.547258
46	United States	2021-09	138317.100000	415.472700
47	United States	2021-10	80986.354839	243.264323
48	United States	2021-11	84910.433333	255.051300
49	United States	2021-12	200890.870968	603.429839
50	United States	2022-01	650778.903226	1954.789548
51	United States	2022-02	145005.857143	435.564107
52	United States	2022-03	44279.200000	133.004200

Já a função `merge` une duas bases de acordo com colunas identificadoras. Vamos juntar as bases do Brasil e EUA utilizando a coluna de ano-mês como identificadora.

```
df_covid_merge = pd.merge(df_covid_br_mensal, df_covid_us_mensal, how='inner', on=['ano_mes'])
df_covid_merge
```

	location_x	ano_mes	new_cases_x	new_cases_per_million_x	location_y	new_cases_y	new_cases_per_million_y
0	Brazil	2020-02	0.500000	0.002500	United States	0.586207	0.002500
1	Brazil	2020-03	184.354839	0.861452	United States	6195.290323	18.435484
2	Brazil	2020-04	2715.666667	12.690400	United States	29444.100000	89.357484
3	Brazil	2020-05	13837.096774	64.661387	United States	22969.967742	64.661387
4	Brazil	2020-06	31597.517241	147.656517	United States	28578.766667	89.357484
5	Brazil	2020-07	40573.612903	189.602194	United States	61294.612903	189.602194
6	Brazil	2020-08	40141.225806	187.581516	United States	48365.322581	147.656517
7	Brazil	2020-09	30084.533333	140.586233	United States	39903.133333	140.586233
8	Brazil	2020-10	23238.645161	108.595065	United States	61870.806452	108.595065
9	Brazil	2020-11	26806.733333	125.268900	United States	149008.733333	425.304839
10	Brazil	2020-12	43195.709677	201.855258	United States	211725.580645	603.429839
11	Brazil	2021-01	49273.838710	230.258677	United States	198457.129032	586.207000
12	Brazil	2021-02	48198.928571	225.235607	United States	86034.107143	243.264323
13	Brazil	2021-03	71138.741935	332.434258	United States	58529.064516	177.156667
14	Brazil	2021-04	63723.733333	297.783600	United States	62970.333333	189.148333
15	Brazil	2021-05	60734.967742	283.817000	United States	29748.451613	89.357484
16	Brazil	2021-06	67080.266667	313.468767	United States	13276.700000	39.880100
17	Brazil	2021-07	43608.064516	203.782323	United States	42530.483871	127.751742

	location_x	ano_mes	new_cases_x	new_cases_per_million_x	location_y	new_cases_y	n
18	Brazil	2021-08	27840.322581	130.098935	United States	138009.000000	4
19	Brazil	2021-09	18009.931034	84.161034	United States	138317.100000	4
20	Brazil	2021-10	12331.064516	57.623548	United States	80986.354839	2
21	Brazil	2021-11	9517.266667	44.474533	United States	84910.433333	2
22	Brazil	2021-12	6183.741935	28.896806	United States	200890.870968	6
23	Brazil	2022-01	102312.612903	478.111000	United States	650778.903226	1
24	Brazil	2022-02	119037.178571	556.265607	United States	145005.857143	4
25	Brazil	2022-03	48845.800000	228.258600	United States	44279.200000	1

Que legal! Tome um tempo em casa para entender as particularidades dessa função `merge`, elas são muitas.

Mas note que existiam colunas com o mesmo nome nas bases. Por padrão o pandas renomeia essas colunas com `_x` e `_y`. Vamos renomear essas colunas utilizando dicionários, uma forma alternativa ao que fizemos anteriormente.

```
df_covid_merge.drop(columns=['location_x','location_y'], inplace=True)

dict_rename = {
    'new_cases_x': 'nc_br',
    'new_cases_per_million_x': 'ncpm_br',
    'new_cases_y': 'nc_us',
    'new_cases_per_million_y': 'ncpm_us'
}

df_covid_merge.rename(columns=dict_rename, inplace=True)

df_covid_merge
```

	ano_mes	nc_br	ncpm_br	nc_us	ncpm_us
0	2020-02	0.500000	0.002500	0.586207	0.001759
1	2020-03	184.354839	0.861452	6195.290323	18.609226
2	2020-04	2715.666667	12.690400	29444.100000	88.443333
3	2020-05	13837.096774	64.661387	22969.967742	68.996452
4	2020-06	31597.517241	147.656517	28578.766667	85.844033
5	2020-07	40573.612903	189.602194	61294.612903	184.114871
6	2020-08	40141.225806	187.581516	48365.322581	145.278355
7	2020-09	30084.533333	140.586233	39903.133333	119.859833
8	2020-10	23238.645161	108.595065	61870.806452	185.845677

	ano_mes	nc_br	ncpm_br	nc_us	ncpm_us
9	2020-11	26806.733333	125.268900	149008.733333	447.587733
10	2020-12	43195.709677	201.855258	211725.580645	635.974742
11	2021-01	49273.838710	230.258677	198457.129032	596.119355
12	2021-02	48198.928571	225.235607	86034.107143	258.426643
13	2021-03	71138.741935	332.434258	58529.064516	175.807742
14	2021-04	63723.733333	297.783600	62970.333333	189.148333
15	2021-05	60734.967742	283.817000	29748.451613	89.357484
16	2021-06	67080.266667	313.468767	13276.700000	39.880100
17	2021-07	43608.064516	203.782323	42530.483871	127.751742
18	2021-08	27840.322581	130.098935	138009.000000	414.547258
19	2021-09	18009.931034	84.161034	138317.100000	415.472700
20	2021-10	12331.064516	57.623548	80986.354839	243.264323
21	2021-11	9517.266667	44.474533	84910.433333	255.051300
22	2021-12	6183.741935	28.896806	200890.870968	603.429839
23	2022-01	102312.612903	478.111000	650778.903226	1954.789548
24	2022-02	119037.178571	556.265607	145005.857143	435.564107
25	2022-03	48845.800000	228.258600	44279.200000	133.004200

8.4.7 Exportando o DataFrame

Por fim, a última coisa que gostaríamos de aprender nesse aulão de Pandas é como salvar o dataframe, resultado de todas as nossas operações, para um novo `csv` ou arquivo de qualquer outro tipo. Assim como tínhamos funções do tipo `read_csv` para trazer arquivos de fora, temos métodos do tipo `to_csv` para salvar um dado dataframe em um novo arquivo a ser utilizado em outros softwares.

A sintaxe é bastante simples:

```
df_covid_merge.to_csv('df_resultado_aula_pandas.csv', sep=',', encoding='utf8', index=False)

os.listdir()
```

```
['df_resultado_aula_pandas.csv',
'EAE1106 - Aula Fundamentos.html',
'EAE1106 - Aula Fundamentos.pdf',
'EAE1106 - Aula Funções.html',
'EAE1106 - Aula Funções.pdf',
'EAE1106 - Aula Instalação.html',
'EAE1106 - Aula Instalação.pdf',
'EAE1106 - Aula Iteração.html',
```

```
'EAE1106 - Aula Iteração.pdf',
'EAE1106 - Aula Matplotlib.html',
'EAE1106 - Aula Matplotlib.pdf',
'EAE1106 - Aula NumPy.html',
'EAE1106 - Aula NumPy.pdf',
'EAE1106 - Aula Objetos.html',
'EAE1106 - Aula Objetos.pdf',
'EAE1106 - Aula Pandas.html',
'EAE1106 - Aula Pandas.pdf',
'EAE1106_Fundamentos_slides.aux',
'EAE1106_Fundamentos_slides.log',
'EAE1106_Fundamentos_slides.nav',
'EAE1106_Fundamentos_slides.out',
'EAE1106_Fundamentos_slides.pdf',
'EAE1106_Fundamentos_slides.snm',
'EAE1106_Fundamentos_slides.synctex.gz',
'EAE1106_Fundamentos_slides.tex',
'EAE1106_Fundamentos_slides.toc',
'EAE1106_Introducao_slides.aux',
'EAE1106_Introducao_slides.log',
'EAE1106_Introducao_slides.nav',
'EAE1106_Introducao_slides.out',
'EAE1106_Introducao_slides.pdf',
'EAE1106_Introducao_slides.snm',
'EAE1106_Introducao_slides.synctex.gz',
'EAE1106_Introducao_slides.tex',
'EAE1106_Introducao_slides.toc',
'notebook_images',
'owid-covid-data_top10.csv']
```

Olha o arquivo que geramos aí, minha gente!

Tente exportar o dataframe em outros formatos e importá-lo em outros softwares. Lembre-se, assim como quando queremos aprender novas línguas como inglês, alemão ou mandarim, em programação a fluência chega apenas com muito treino, tentativa e erro.

8.5 Eficiência e *dtypes*

DataFrames podem ser objetos bastante pesados, com milhões de linhas e milhares de colunas. Isso por vezes pode ser um problemão para nós. Existem bibliotecas que servem ao propósito de trabalhar de forma mais eficiente com bases de dados muito grandes (e.g., [Dask](#)), mas algumas

funções dentro do próprio Pandas já conseguem nos ajudar bastante em grande parte das situações.

Vamos começar relendo a base completa de dados da covid com os 10 países com o maior número de casos. Além disso, imagine que por um descuido nosso tenhamos replicado todas as entradas por 10 vezes seguidas. Qual é o tamanho dessa base e quantos bytes de memória foram alocados para esse objeto?

```
df_covid_duplicada = pd.DataFrame()

for i in range(0,10):
    df_covid_duplicada = pd.concat([df_covid_duplicada, df_covid]).reset_index(drop=True)

df_covid_duplicada.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 76630 entries, 0 to 76629
```

```
Data columns (total 68 columns):
```

#	Column	Non-Null Count	Dtype
0	iso_code	76630 non-null	object
1	continent	76630 non-null	object
2	location	76630 non-null	object
3	date	76630 non-null	object
4	total_cases	76630 non-null	float64
5	new_cases	76410 non-null	float64
6	new_cases_smoothed	74840 non-null	float64
7	total_deaths	73450 non-null	float64
8	new_deaths	73270 non-null	float64
9	new_deaths_smoothed	71730 non-null	float64
10	total_cases_per_million	76630 non-null	float64
11	new_cases_per_million	76410 non-null	float64
12	new_cases_smoothed_per_million	74840 non-null	float64
13	total_deaths_per_million	73450 non-null	float64
14	new_deaths_per_million	73270 non-null	float64
15	new_deaths_smoothed_per_million	71730 non-null	float64
16	reproduction_rate	72920 non-null	float64
17	icu_patients	42780 non-null	float64
18	icu_patients_per_million	42780 non-null	float64
19	hosp_patients	28610 non-null	float64
20	hosp_patients_per_million	28610 non-null	float64
21	weekly_icu_admissions	13770 non-null	float64
22	weekly_icu_admissions_per_million	13770 non-null	float64

23	weekly_hosp_admissions	36030	non-null	float64
24	weekly_hosp_admissions_per_million	36030	non-null	float64
25	new_tests	57550	non-null	float64
26	total_tests	60560	non-null	float64
27	total_tests_per_thousand	60560	non-null	float64
28	new_tests_per_thousand	57550	non-null	float64
29	new_tests_smoothed	71160	non-null	float64
30	new_tests_smoothed_per_thousand	71160	non-null	float64
31	positive_rate	58120	non-null	float64
32	tests_per_case	58120	non-null	float64
33	tests_units	71860	non-null	object
34	total_vaccinations	39500	non-null	float64
35	people_vaccinated	39380	non-null	float64
36	people_fully_vaccinated	38600	non-null	float64
37	total_boosters	20700	non-null	float64
38	new_vaccinations	38580	non-null	float64
39	new_vaccinations_smoothed	41810	non-null	float64
40	total_vaccinations_per_hundred	39500	non-null	float64
41	people_vaccinated_per_hundred	39380	non-null	float64
42	people_fully_vaccinated_per_hundred	38600	non-null	float64
43	total_boosters_per_hundred	20700	non-null	float64
44	new_vaccinations_smoothed_per_million	41810	non-null	float64
45	new_people_vaccinated_smoothed	41810	non-null	float64
46	new_people_vaccinated_smoothed_per_hundred	41810	non-null	float64
47	stringency_index	75800	non-null	float64
48	population	76630	non-null	float64
49	population_density	76630	non-null	float64
50	median_age	76630	non-null	float64
51	aged_65_older	76630	non-null	float64
52	aged_70_older	76630	non-null	float64
53	gdp_per_capita	76630	non-null	float64
54	extreme_poverty	53480	non-null	float64
55	cardiovasc_death_rate	76630	non-null	float64
56	diabetes_prevalence	76630	non-null	float64
57	female_smokers	76630	non-null	float64
58	male_smokers	76630	non-null	float64
59	handwashing_facilities	7660	non-null	float64
60	hospital_beds_per_thousand	76630	non-null	float64
61	life_expectancy	76630	non-null	float64
62	human_development_index	76630	non-null	float64
63	excess_mortality_cumulative_absolute	6920	non-null	float64
64	excess_mortality_cumulative	6920	non-null	float64
65	excess_mortality	6920	non-null	float64

```

66  excess_mortality_cumulative_per_million    6920 non-null    float64
67  ano_mes                                   76630 non-null   object
dtypes: float64(62), object(6)
memory usage: 61.2 MB

```

O DataFrame `df_covid_duplicada` possui 76,630 linhas, 67 colunas e está usando 64.6 MB da nossa memória RAM. O primeiro passo para tornar o objeto mais eficiente é ver se encontramos linhas duplicadas em todas suas colunas, que não nos trazem nenhuma nova informação mas carregam desnecessariamente nossos pentes de memória. Para esse fim utilizaremos a função `drop_duplicates`.

```

df_covid_duplicada.drop_duplicates(inplace=True)

df_covid_duplicada.info(memory_usage='deep')

```

```

<class 'pandas.core.frame.DataFrame'>
Index: 7663 entries, 0 to 7662
Data columns (total 68 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   iso_code                                 7663 non-null   object
1   continent                               7663 non-null   object
2   location                                7663 non-null   object
3   date                                    7663 non-null   object
4   total_cases                             7663 non-null   float64
5   new_cases                               7641 non-null   float64
6   new_cases_smoothed                     7484 non-null   float64
7   total_deaths                           7345 non-null   float64
8   new_deaths                             7327 non-null   float64
9   new_deaths_smoothed                    7173 non-null   float64
10  total_cases_per_million                 7663 non-null   float64
11  new_cases_per_million                   7641 non-null   float64
12  new_cases_smoothed_per_million          7484 non-null   float64
13  total_deaths_per_million                7345 non-null   float64
14  new_deaths_per_million                  7327 non-null   float64
15  new_deaths_smoothed_per_million         7173 non-null   float64
16  reproduction_rate                       7292 non-null   float64
17  icu_patients                            4278 non-null   float64
18  icu_patients_per_million                4278 non-null   float64
19  hosp_patients                           2861 non-null   float64
20  hosp_patients_per_million               2861 non-null   float64
21  weekly_icu_admissions                   1377 non-null   float64

```

22	weekly_icu_admissions_per_million	1377	non-null	float64
23	weekly_hosp_admissions	3603	non-null	float64
24	weekly_hosp_admissions_per_million	3603	non-null	float64
25	new_tests	5755	non-null	float64
26	total_tests	6056	non-null	float64
27	total_tests_per_thousand	6056	non-null	float64
28	new_tests_per_thousand	5755	non-null	float64
29	new_tests_smoothed	7116	non-null	float64
30	new_tests_smoothed_per_thousand	7116	non-null	float64
31	positive_rate	5812	non-null	float64
32	tests_per_case	5812	non-null	float64
33	tests_units	7186	non-null	object
34	total_vaccinations	3950	non-null	float64
35	people_vaccinated	3938	non-null	float64
36	people_fully_vaccinated	3860	non-null	float64
37	total_boosters	2070	non-null	float64
38	new_vaccinations	3858	non-null	float64
39	new_vaccinations_smoothed	4181	non-null	float64
40	total_vaccinations_per_hundred	3950	non-null	float64
41	people_vaccinated_per_hundred	3938	non-null	float64
42	people_fully_vaccinated_per_hundred	3860	non-null	float64
43	total_boosters_per_hundred	2070	non-null	float64
44	new_vaccinations_smoothed_per_million	4181	non-null	float64
45	new_people_vaccinated_smoothed	4181	non-null	float64
46	new_people_vaccinated_smoothed_per_hundred	4181	non-null	float64
47	stringency_index	7580	non-null	float64
48	population	7663	non-null	float64
49	population_density	7663	non-null	float64
50	median_age	7663	non-null	float64
51	aged_65_older	7663	non-null	float64
52	aged_70_older	7663	non-null	float64
53	gdp_per_capita	7663	non-null	float64
54	extreme_poverty	5348	non-null	float64
55	cardiovasc_death_rate	7663	non-null	float64
56	diabetes_prevalence	7663	non-null	float64
57	female_smokers	7663	non-null	float64
58	male_smokers	7663	non-null	float64
59	handwashing_facilities	766	non-null	float64
60	hospital_beds_per_thousand	7663	non-null	float64
61	life_expectancy	7663	non-null	float64
62	human_development_index	7663	non-null	float64
63	excess_mortality_cumulative_absolute	692	non-null	float64
64	excess_mortality_cumulative	692	non-null	float64

```

65  excess_mortality                692 non-null    float64
66  excess_mortality_cumulative_per_million  692 non-null    float64
67  ano_mes                        7663 non-null   object
dtypes: float64(62), object(6)
memory usage: 6.2 MB

```

Legal, conseguimos dropar as linhas duplicadas e reduzimos o comprometimento de memória de 64.6 MB para 6.5MB.

No entanto, os tipos de dados de cada coluna não nos parecem os mais adequados. Uma coluna do tipo `float64` consome 4x mais memória do que uma coluna do tipo `float16` e 8x mais memória do que uma coluna do tipo `int8`, por exemplo. Da mesma forma, uma coluna do tipo `object` consome muito mais memória do que uma coluna que o Pandas entende como do tipo categórica em grande parte dos casos. Mas o que são esses tipos todos?

- **int8 / uint8:** consome 1 byte (8-bit) de memória, o valor numérico deve ser inteiro e estar no intervalo $[-128, 127]$ / $[0, 255]$
- **int16 / uint16:** consome 2 bytes de memória, o valor numérico deve ser inteiro e estar no intervalo $[-32768, 32768]$ / $[0, 65535]$
- **int32 / uint32:** consome 4 bytes de memória e aceita valores numéricos inteiros com até 10 dígitos, da ordem de 2 bilhões.
- **int64 / uint64:** consome 8 bytes de memória e aceita valores numéricos inteiros ainda maiores.
- **float16 / float32 / float64:** consome 2, 4 e 8 bytes de memória, respectivamente, e aceita valores com casas decimais delimitado pelos mesmos intervalos dos tipos anteriores.
- **bool:** consome 1 byte de memória e aceita apenas dois valores `True` e `False`.
- **object:** o quanto consome de memória pode variar, mas, por aceitar valores numéricos e strings de todo tipo, consome ao menos 8 bytes de memória.
- **category:** o quanto consome de memória pode variar e depende da relação entre o número de valores únicos na coluna e o número de linhas de dados.

Beleza, devemos fugir do tipo `object` e tentar reduzir as colunas numéricas para tipos menos demandantes em termos de memória. Como fazer isso? A função `astype` é nossa companheira nesse caso.

Voltando ao nosso dataframe original, as colunas `iso_code`, `continent` e `location` podem ser transformadas em colunas do tipo categórica sem nenhuma perda de informação. Começemos por elas.

```

columns_to_cat = ['iso_code', 'continent', 'location']

for c in columns_to_cat:
    df_covid_duplicada[c] = df_covid_duplicada[c].astype('category')

```

O quanto isso nos ajudou?

```
df_covid_duplicada.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 7663 entries, 0 to 7662
```

```
Data columns (total 68 columns):
```

#	Column	Non-Null Count	Dtype
0	iso_code	7663 non-null	category
1	continent	7663 non-null	category
2	location	7663 non-null	category
3	date	7663 non-null	object
4	total_cases	7663 non-null	float64
5	new_cases	7641 non-null	float64
6	new_cases_smoothed	7484 non-null	float64
7	total_deaths	7345 non-null	float64
8	new_deaths	7327 non-null	float64
9	new_deaths_smoothed	7173 non-null	float64
10	total_cases_per_million	7663 non-null	float64
11	new_cases_per_million	7641 non-null	float64
12	new_cases_smoothed_per_million	7484 non-null	float64
13	total_deaths_per_million	7345 non-null	float64
14	new_deaths_per_million	7327 non-null	float64
15	new_deaths_smoothed_per_million	7173 non-null	float64
16	reproduction_rate	7292 non-null	float64
17	icu_patients	4278 non-null	float64
18	icu_patients_per_million	4278 non-null	float64
19	hosp_patients	2861 non-null	float64
20	hosp_patients_per_million	2861 non-null	float64
21	weekly_icu_admissions	1377 non-null	float64
22	weekly_icu_admissions_per_million	1377 non-null	float64
23	weekly_hosp_admissions	3603 non-null	float64
24	weekly_hosp_admissions_per_million	3603 non-null	float64
25	new_tests	5755 non-null	float64
26	total_tests	6056 non-null	float64
27	total_tests_per_thousand	6056 non-null	float64
28	new_tests_per_thousand	5755 non-null	float64
29	new_tests_smoothed	7116 non-null	float64
30	new_tests_smoothed_per_thousand	7116 non-null	float64
31	positive_rate	5812 non-null	float64
32	tests_per_case	5812 non-null	float64

33	tests_units	7186	non-null	object
34	total_vaccinations	3950	non-null	float64
35	people_vaccinated	3938	non-null	float64
36	people_fully_vaccinated	3860	non-null	float64
37	total_boosters	2070	non-null	float64
38	new_vaccinations	3858	non-null	float64
39	new_vaccinations_smoothed	4181	non-null	float64
40	total_vaccinations_per_hundred	3950	non-null	float64
41	people_vaccinated_per_hundred	3938	non-null	float64
42	people_fully_vaccinated_per_hundred	3860	non-null	float64
43	total_boosters_per_hundred	2070	non-null	float64
44	new_vaccinations_smoothed_per_million	4181	non-null	float64
45	new_people_vaccinated_smoothed	4181	non-null	float64
46	new_people_vaccinated_smoothed_per_hundred	4181	non-null	float64
47	stringency_index	7580	non-null	float64
48	population	7663	non-null	float64
49	population_density	7663	non-null	float64
50	median_age	7663	non-null	float64
51	aged_65_older	7663	non-null	float64
52	aged_70_older	7663	non-null	float64
53	gdp_per_capita	7663	non-null	float64
54	extreme_poverty	5348	non-null	float64
55	cardiovasc_death_rate	7663	non-null	float64
56	diabetes_prevalence	7663	non-null	float64
57	female_smokers	7663	non-null	float64
58	male_smokers	7663	non-null	float64
59	handwashing_facilities	766	non-null	float64
60	hospital_beds_per_thousand	7663	non-null	float64
61	life_expectancy	7663	non-null	float64
62	human_development_index	7663	non-null	float64
63	excess_mortality_cumulative_absolute	692	non-null	float64
64	excess_mortality_cumulative	692	non-null	float64
65	excess_mortality	692	non-null	float64
66	excess_mortality_cumulative_per_million	692	non-null	float64
67	ano_mes	7663	non-null	object

dtypes: category(3), float64(62), object(3)
memory usage: 5.0 MB

Caramba! Esse pequena alteração reduziu o comprometimento de memória em aproximadamente 20%.

Vamos tentar transformar as colunas de `total_cases`, `new_cases`, `total_deaths` e `new_deaths` em colunas do tipo `int32` ao invés de `float64` já que são colunas sabidamente de números

inteiros. Não esqueça de lidar com os valores missing antes, já que dados do tipo `int` não aceitam `NaN` ou `infty` como valores.

```
columns_to_int = ['total_cases', 'new_cases', 'total_deaths', 'new_deaths']

for c in columns_to_int:
    df_covid_duplicada[c] = df_covid_duplicada[c].fillna(-1)
    df_covid_duplicada[c] = df_covid_duplicada[c].astype('int32')
```

O quanto isso nos ajudou?

```
df_covid_duplicada.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 7663 entries, 0 to 7662
```

```
Data columns (total 68 columns):
```

#	Column	Non-Null Count	Dtype
0	iso_code	7663 non-null	category
1	continent	7663 non-null	category
2	location	7663 non-null	category
3	date	7663 non-null	object
4	total_cases	7663 non-null	int32
5	new_cases	7663 non-null	int32
6	new_cases_smoothed	7484 non-null	float64
7	total_deaths	7663 non-null	int32
8	new_deaths	7663 non-null	int32
9	new_deaths_smoothed	7173 non-null	float64
10	total_cases_per_million	7663 non-null	float64
11	new_cases_per_million	7641 non-null	float64
12	new_cases_smoothed_per_million	7484 non-null	float64
13	total_deaths_per_million	7345 non-null	float64
14	new_deaths_per_million	7327 non-null	float64
15	new_deaths_smoothed_per_million	7173 non-null	float64
16	reproduction_rate	7292 non-null	float64
17	icu_patients	4278 non-null	float64
18	icu_patients_per_million	4278 non-null	float64
19	hosp_patients	2861 non-null	float64
20	hosp_patients_per_million	2861 non-null	float64
21	weekly_icu_admissions	1377 non-null	float64
22	weekly_icu_admissions_per_million	1377 non-null	float64
23	weekly_hosp_admissions	3603 non-null	float64

24	weekly_hosp_admissions_per_million	3603	non-null	float64
25	new_tests	5755	non-null	float64
26	total_tests	6056	non-null	float64
27	total_tests_per_thousand	6056	non-null	float64
28	new_tests_per_thousand	5755	non-null	float64
29	new_tests_smoothed	7116	non-null	float64
30	new_tests_smoothed_per_thousand	7116	non-null	float64
31	positive_rate	5812	non-null	float64
32	tests_per_case	5812	non-null	float64
33	tests_units	7186	non-null	object
34	total_vaccinations	3950	non-null	float64
35	people_vaccinated	3938	non-null	float64
36	people_fully_vaccinated	3860	non-null	float64
37	total_boosters	2070	non-null	float64
38	new_vaccinations	3858	non-null	float64
39	new_vaccinations_smoothed	4181	non-null	float64
40	total_vaccinations_per_hundred	3950	non-null	float64
41	people_vaccinated_per_hundred	3938	non-null	float64
42	people_fully_vaccinated_per_hundred	3860	non-null	float64
43	total_boosters_per_hundred	2070	non-null	float64
44	new_vaccinations_smoothed_per_million	4181	non-null	float64
45	new_people_vaccinated_smoothed	4181	non-null	float64
46	new_people_vaccinated_smoothed_per_hundred	4181	non-null	float64
47	stringency_index	7580	non-null	float64
48	population	7663	non-null	float64
49	population_density	7663	non-null	float64
50	median_age	7663	non-null	float64
51	aged_65_older	7663	non-null	float64
52	aged_70_older	7663	non-null	float64
53	gdp_per_capita	7663	non-null	float64
54	extreme_poverty	5348	non-null	float64
55	cardiovasc_death_rate	7663	non-null	float64
56	diabetes_prevalence	7663	non-null	float64
57	female_smokers	7663	non-null	float64
58	male_smokers	7663	non-null	float64
59	handwashing_facilities	766	non-null	float64
60	hospital_beds_per_thousand	7663	non-null	float64
61	life_expectancy	7663	non-null	float64
62	human_development_index	7663	non-null	float64
63	excess_mortality_cumulative_absolute	692	non-null	float64
64	excess_mortality_cumulative	692	non-null	float64
65	excess_mortality	692	non-null	float64
66	excess_mortality_cumulative_per_million	692	non-null	float64

```
67  ano_mes                                7663 non-null    object
dtypes: category(3), float64(58), int32(4), object(3)
memory usage: 4.9 MB
```

Reduzimos mais um pouco o comprometimento de memória, embora a diferença não tenha sido tão grande. Em dataframes com milhões de linhas e milhares de colunas, no entanto, essa redução pode fazer toda a diferença. Dica: dê uma olhada na função `pandas.to_numeric` e no argumento `downcast` caso você não saiba para qual tipo de dado numérico deve ir.

No fim das contas, parte importante da estratégia de otimização na gestão de memória de dataframes vem de conhecermos os tipos mais adequados de dados para cada coluna **antes** da leitura do dataframe. Isso pode ser feito lendo primeiro uma amostra de poucas linhas (e.g., 10 ou 20) do dataframe com o qual iremos trabalhar para entender a base de dados e só a partir daí ler o dataframe completo com a função `read_csv` (ou similar) e passar uma lista de tipo de dados ao argumento `dtype` (exemplo do algoritmo [aqui](#)). Tente isso em casa! **E não se esqueça: fuja de dados do tipo `object`!**

8.6 Exercícios

1. X
2. X

9 Visualização

9.1 Introdução

9.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- os
- pandas
- matplotlib
- seaborn
- plotly

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sbs
import plotly as ply
```

9.2 O que é o Matplotlib?

Matplotlib é uma excelente biblioteca de ferramentas gráficas, projetada para computação científica, que nos permite um grau bastante alto de personalização sobre todos os aspectos da apresentação. Nos permite construir visualizações estáticas em 2D e 3D e exportá-las nos mais diferentes formatos. Embora existam outras bibliotecas (e.g., [altair](#), [bokeh](#), [plotly](#)) que nos permitam construir visualizações dinâmicas e mais complexas, o Matplotlib é uma excelente primeira abordagem no campo de visualização de dados dentro do Python. Para a esmagadora maioria de nossas aplicações, é essa biblioteca que iremos buscar primeiro.

9.3 Construindo nosso 1º gráfico

9.3.1 Base de dados

Em relação aos dados, utilizaremos mais uma vez a base com informações relacionadas a covid-19 nos 10 países com o maior número de casos acumulados. Dessa vez, no entanto, nos concentraremos em algumas poucas colunas e na agregação mensal.

```
# substituir esse caminho de diretório pelo caminho que você estiver utilizando no seu computador
os.chdir('D:/Dropbox/Projetos e trabalhos/FEAUSP/Cursos/EAE1106 - Metodos Computacionais para

columns_to_read = ['location',
                   'date',
                   'total_cases',
                   'new_cases',
                   'total_deaths',
                   'new_deaths',
                   'population',
                   'people_fully_vaccinated',
                   'median_age',
                   'aged_65_older',
                   'cardiovasc_death_rate',
                   'diabetes_prevalence']

df_covid = pd.read_csv('owid-covid-data_top10.csv', sep=',', encoding='utf8', usecols=columns_to_read)
df_covid['ano_mes'] = df_covid['date'].str[0:7]

df_covid_mensal = df_covid.groupby(by=['location', 'ano_mes'], as_index=False).agg(total_cases=
new_cases=
total_deaths=
new_deaths=
population=
people_vac=
median_age=
aged_65_ol=
cardiovasc=
diabetes_p=

df_covid_mensal.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 269 entries, 0 to 268
```

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	location	269 non-null	object
1	ano_mes	269 non-null	object
2	total_cases	269 non-null	float64
3	new_cases	269 non-null	float64
4	total_deaths	255 non-null	float64
5	new_deaths	269 non-null	float64
6	population	269 non-null	float64
7	people_vaccinated	145 non-null	float64
8	median_age	269 non-null	float64
9	aged_65_older	269 non-null	float64
10	cardiovasc_death_rate	269 non-null	float64
11	diabetes_prevalence	269 non-null	float64

dtypes: float64(10), object(2)

memory usage: 25.3+ KB

```
df_covid_mensal
```

	location	ano_mes	total_cases	new_cases	total_deaths	new_deaths	population	people_vaccinated
0	Brazil	2020-02	2.0	2.0	NaN	0.0	213993441.0	NaN
1	Brazil	2020-03	5717.0	5715.0	201.0	201.0	213993441.0	NaN
2	Brazil	2020-04	87187.0	81470.0	6006.0	5805.0	213993441.0	NaN
3	Brazil	2020-05	516137.0	428950.0	29367.0	23361.0	213993441.0	NaN
4	Brazil	2020-06	1412669.0	916328.0	59792.0	30425.0	213993441.0	NaN
...	...	...	...	...	...	...	...	...
264	United States	2021-11	48582403.0	2547313.0	781652.0	34249.0	332915074.0	19991
265	United States	2021-12	54810020.0	6227617.0	827893.0	46241.0	332915074.0	20751
266	United States	2022-01	74984166.0	20174146.0	889490.0	61597.0	332915074.0	21218
267	United States	2022-02	79044330.0	4060164.0	950732.0	61242.0	332915074.0	21510
268	United States	2022-03	79265726.0	221396.0	958437.0	7705.0	332915074.0	21527

Note que temos alguns problemas com valores missing, vamos nesse casos substituí-los por zero.

```
df_covid_mensal.fillna(0,inplace=True)
df_covid_mensal
```

	location	ano_mes	total_cases	new_cases	total_deaths	new_deaths	population	people
0	Brazil	2020-02	2.0	2.0	0.0	0.0	213993441.0	0.0
1	Brazil	2020-03	5717.0	5715.0	201.0	201.0	213993441.0	0.0
2	Brazil	2020-04	87187.0	81470.0	6006.0	5805.0	213993441.0	0.0
3	Brazil	2020-05	516137.0	428950.0	29367.0	23361.0	213993441.0	0.0
4	Brazil	2020-06	1412669.0	916328.0	59792.0	30425.0	213993441.0	0.0
...	...	...	...	...	...	...	...	...
264	United States	2021-11	48582403.0	2547313.0	781652.0	34249.0	332915074.0	19991
265	United States	2021-12	54810020.0	6227617.0	827893.0	46241.0	332915074.0	20751
266	United States	2022-01	74984166.0	20174146.0	889490.0	61597.0	332915074.0	21218
267	United States	2022-02	79044330.0	4060164.0	950732.0	61242.0	332915074.0	21510
268	United States	2022-03	79265726.0	221396.0	958437.0	7705.0	332915074.0	21527

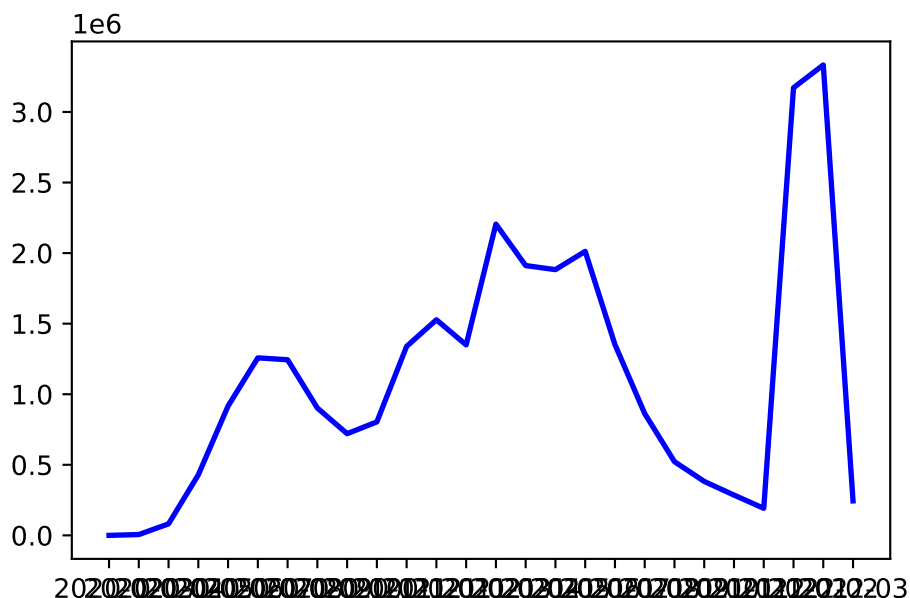
9.3.2 Função *plot*

Com o dataframe criado, podemos plotar o número total de novos casos no Brasil ao longo do tempo usando uma única função: `plot`. Essa função recebe como argumento os valores do eixo `x` (no nosso caso a data) e do eixo `y` (número de novos casos) e nos devolve um gráfico de linha relacionando esses dois conjuntos de dados. Por fim é preciso chamar a função `show()` para mostrar o gráfico que criamos.

```
df_covid_br = df_covid_mensal[df_covid_mensal['location'].isin(['Brazil'])].reset_index(drop=True)

x = df_covid_br['ano_mes']
y = df_covid_br['new_cases']

# construção do gráfico
plt.plot(x, y, 'b-', linewidth=2)
plt.show()
```



O gráfico tá feito, mas ficou um pouco feio. Como podemos melhorar?

- Ajustar o que mostramos no eixo X.
- Ajustar a escala do eixo Y.
- Mudar a cor da linha para algo mais agradável.
- Colocar um marcador em cada ponto de dado.
- Adicionar título ao gráfico.

Sabemos que nossos dados começam em fevereiro de 2020 e vão até março de 2022. Para ajustar o eixo y, porém, é importante conhecer os limites numéricos do número de casos.

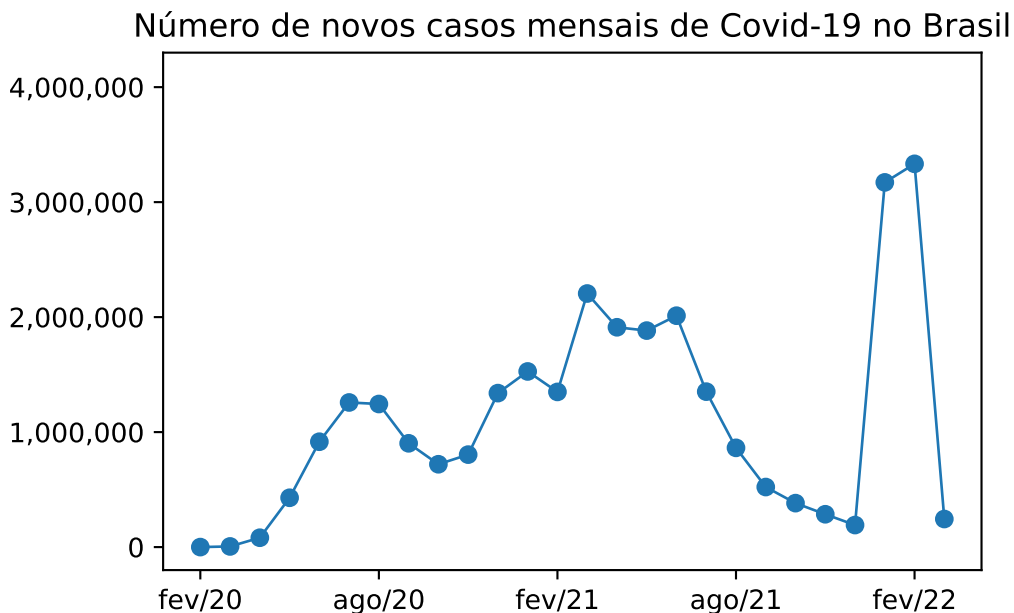
```
df_covid_br.describe()
```

	total_cases	new_cases	total_deaths	new_deaths	population	people_vaccinated	m
count	2.600000e+01	2.600000e+01	26.000000	26.000000	26.0	2.600000e+01	26
mean	1.257560e+07	1.112912e+06	333718.576923	25085.230769	213993441.0	4.313415e+07	33
std	9.654155e+06	9.019196e+05	244101.785272	21076.256589	0.0	5.995921e+07	0.0
min	2.000000e+00	2.000000e+00	0.000000	0.000000	213993441.0	0.000000e+00	33
25%	4.140463e+06	3.939348e+05	127233.000000	8953.750000	213993441.0	0.000000e+00	33
50%	1.166074e+07	9.094320e+05	288658.500000	22273.500000	213993441.0	3.508376e+06	33
75%	2.127057e+07	1.483579e+06	592965.000000	30504.500000	213993441.0	8.424734e+07	33
max	2.904080e+07	3.333041e+06	652216.000000	82392.000000	213993441.0	1.557116e+08	33

Agora podemos usar algumas opções e métodos sobre o `plot` que criamos dentro da célula para aplicar todas essas alterações sugeridas.

```
x_ticks = [0,6,12,18,24]
x_labels = ['fev/20','ago/20','fev/21','ago/21','fev/22']
y_ticks = [0,1000000,2000000,3000000,4000000]
y_labels = ['0','1,000,000','2,000,000','3,000,000','4,000,000']
```

```
plt.plot(x, y, color='#1f77b4', linewidth=1, marker='o') # Saiba que o código "#1f77b4" é um
plt.xticks(x_ticks, x_labels)
plt.yticks(y_ticks, y_labels)
plt.ylim([-200000, 4300000])
plt.title('Número de novos casos mensais de Covid-19 no Brasil')
plt.show()
```



Bem melhor né? Mas ainda dá para melhorar **muito**!

Uma das belezas do Python é sua lógica de programação multiparadigma, o que nos permite trabalhar também com a lógica orientada a objetos. Isso não é diferente com gráficos.

No método que acabamos de utilizar, muitos dos objetos que compõe o gráfico como um todo (eixos, fundo, bordas, etc.) são criados e transmitidos sem se tornarem conhecidos pelo programador. Em geral é preferível um estilo de programação mais explícito, sob o qual

teríamos mais controle sobre as várias dimensões do gráfico. E é assim que trabalharemos com gráficos daqui para frente.

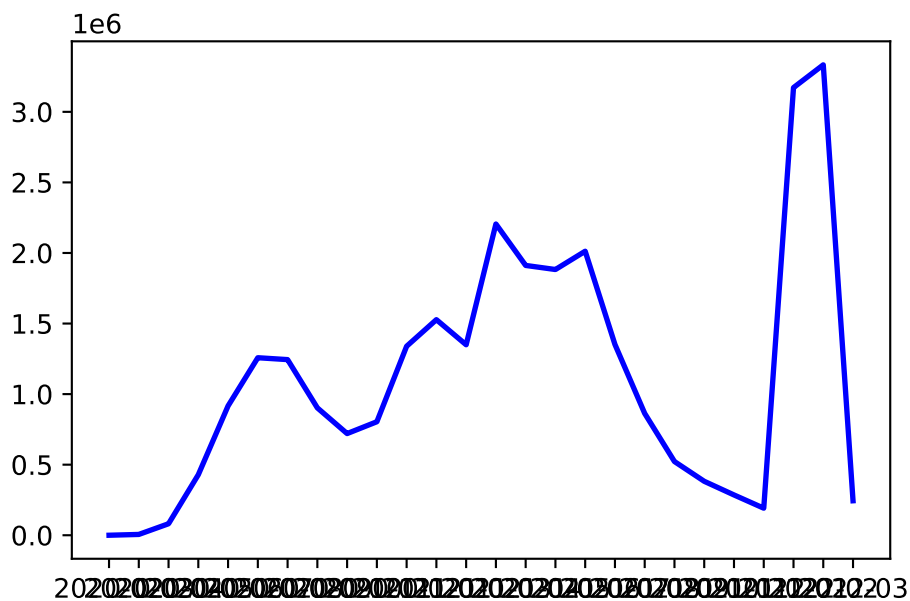
9.4 Personalização

Vamos começar refazendo o primeiro gráfico, antes das alterações na escala e cores.

```
fig, ax = plt.subplots()

ax.plot(x, y, 'b-', linewidth=2)

plt.show()
```



Legal, mas o que significam todas essas coisas?

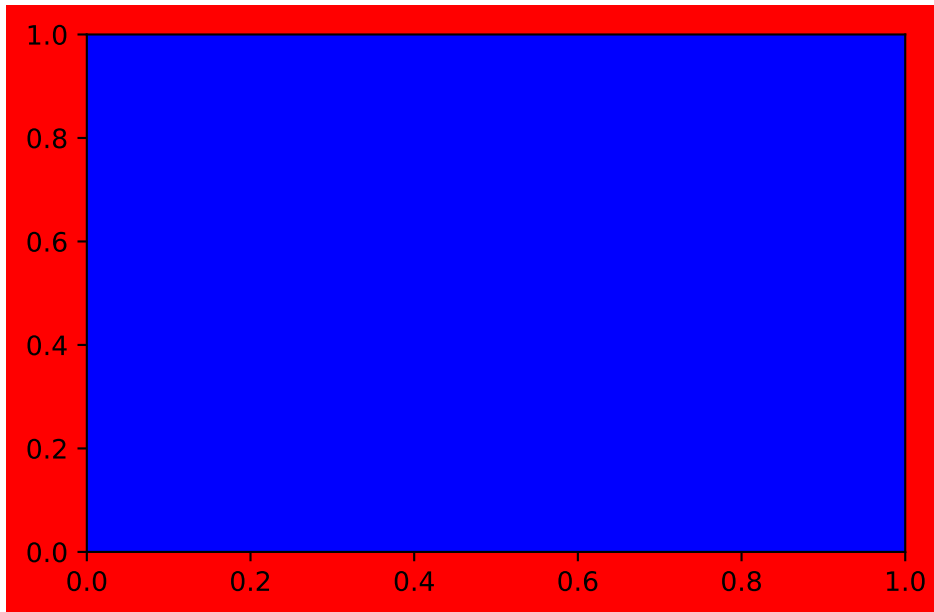
Começamos utilizando a função `subplots()` do `matplotlib.pyplot`, que nos retorna dois objetos aos quais damos os nomes de `fig` e `ax`. Sem entrar nos detalhes de classes e instâncias da lógica de programação orientada a objetos no Python:

- `fig` é como se fosse um fundo branco.
- Pense em `ax` como sendo uma forma de iniciar um quadro sobre o qual construiremos a figura.

```
fig, ax = plt.subplots()

fig.set_facecolor("red")
ax.set_facecolor("blue")

plt.show()
```



Em essência, a função `plot()` é na verdade um método de `ax`.

Embora fazer esse gráfico simples dessa forma exija um pouco mais de código, o uso explícito de objetos nos dá melhor controle sobre a aparência do objeto e de todas as personalizações possíveis. Isso ficará mais claro à medida que avançarmos.

Como replicar o 2º gráfico da seção anterior usando essa lógica orientada a objetos?

```
x_ticks = [0,6,12,18,24]
x_labels = ['fev/20','ago/20','fev/21','ago/21','fev/22']
y_ticks = [0,1000000,2000000,3000000,4000000]
y_labels = ['0','1,000,000','2,000,000','3,000,000','4,000,000']

fig, ax = plt.subplots()

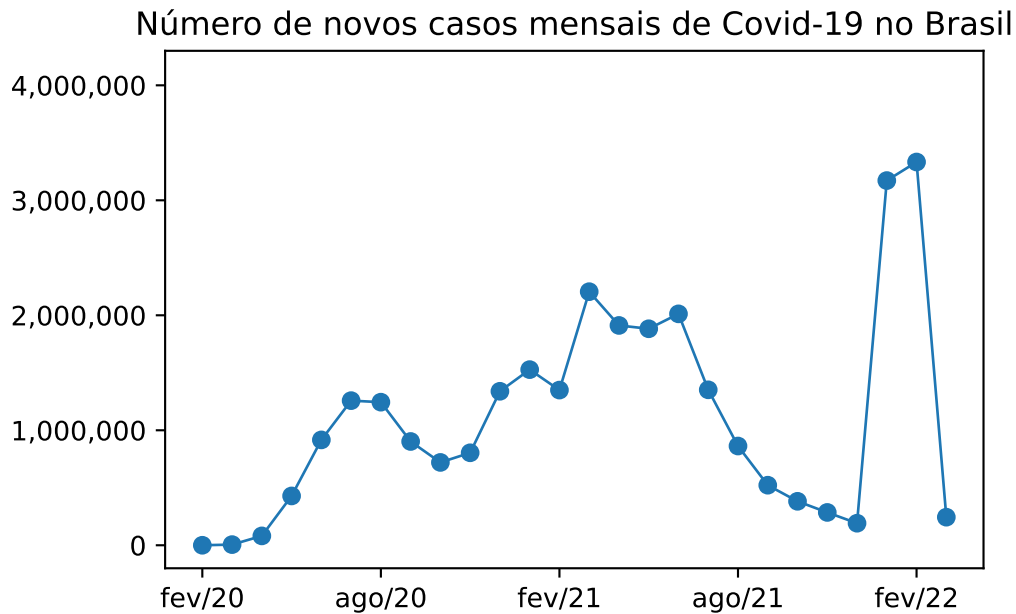
ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o')
ax.set_xticks(x_ticks)
```

```

ax.set_xticklabels(x_labels)
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels)
ax.set_ylim(-200000, 4300000)

plt.title('Número de novos casos mensais de Covid-19 no Brasil')
plt.show()

```



Foram várias linhas de código a mais com essa abordagem mais orientada a objeto, mas que no fim nos permite muito mais personalizações. Vamos tentar deixá-lo ainda melhor?

A ideia por trás de uma figura é sempre a mesma: transmitir alguma informação de forma clara e direta. Seguindo essa ideia e partindo do gráfico inicial, vamos fazer mais algumas alterações uma a uma e ir comparando os resultados.

- Mudar a proporção do gráfico e manter a borda apenas do eixo X

```

height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

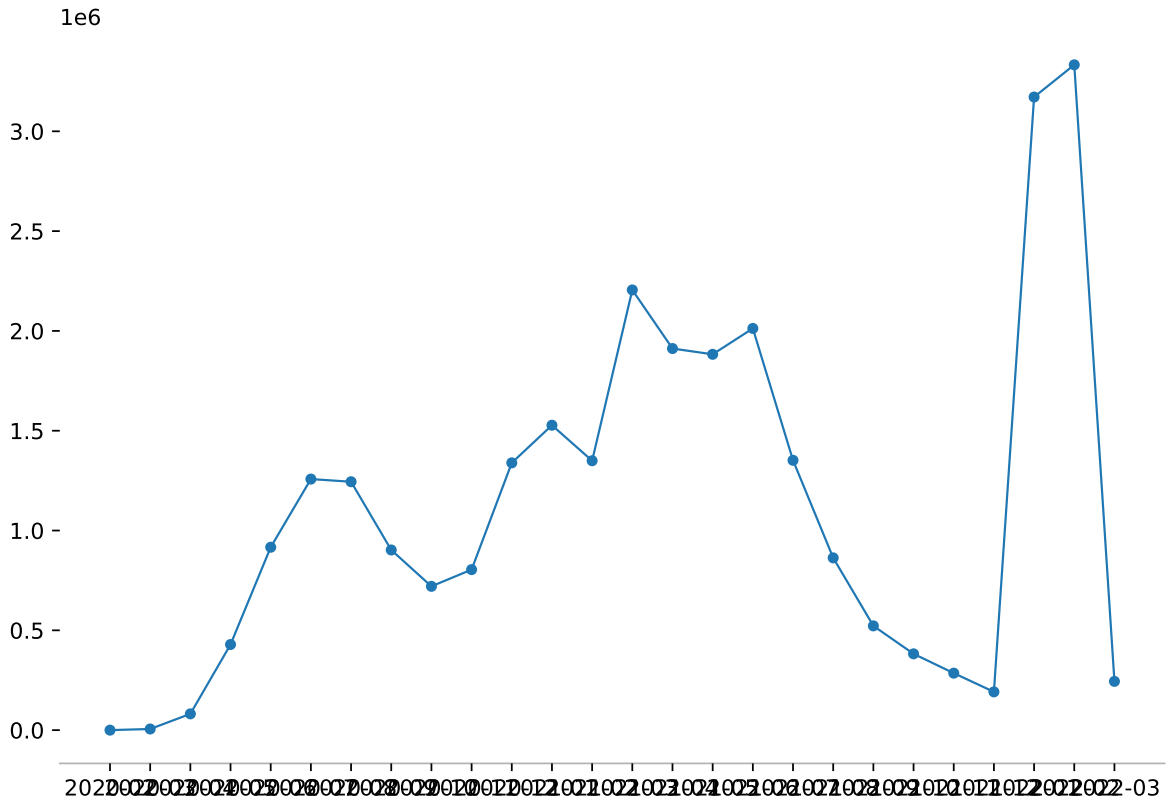
# Tipo de gráfico a ser plotado
ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o', markersize=4)

# Visibilidade das bordas

```

```
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Mostrar o resultado
plt.show()
```



- Colocar linhas de grid para os valores de Y

```
height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Tipo de gráfico a ser plotado
ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o', markersize=4)

# Visibilidade das bordas
```

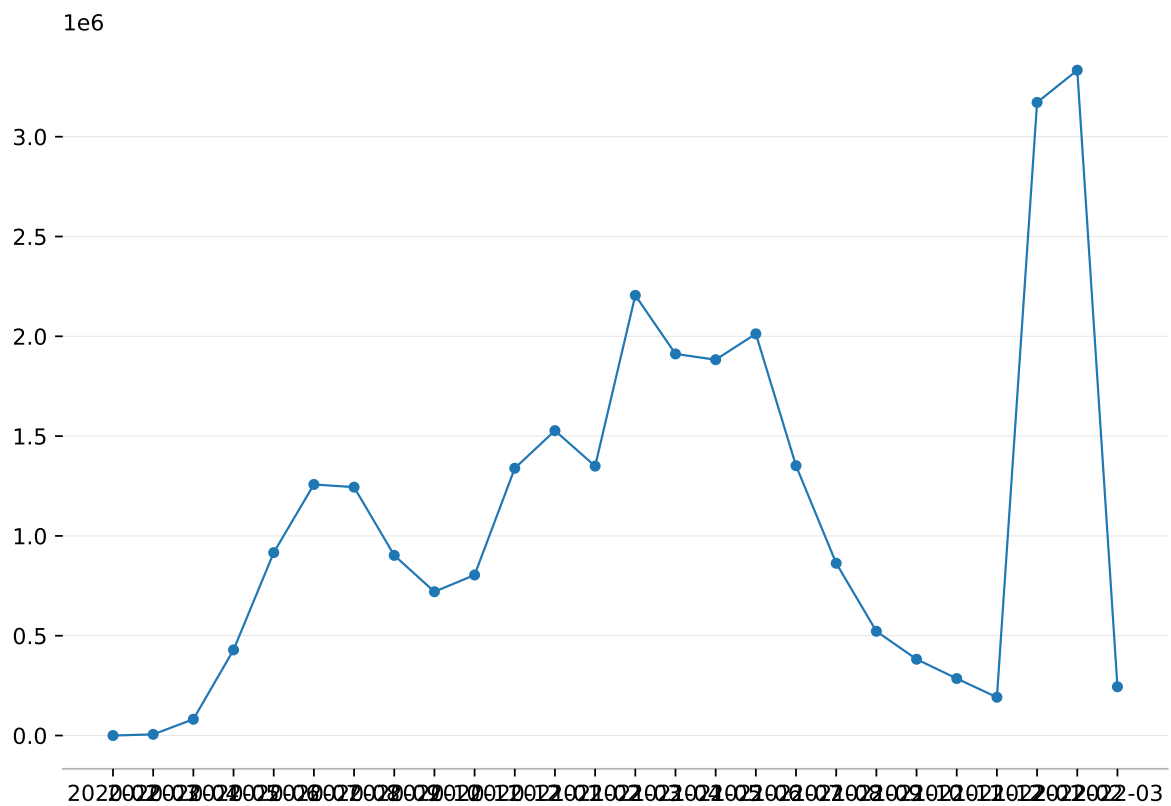
```

ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major', axis='y', ls='-', lw=0.5, c='k', alpha=0.1)

# Mostrar o resultado
plt.show()

```



- Mudar os marcadores do eixo x e do eixo y como já havíamos feito.

```

height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Tipo de gráfico a ser plotado

```

```

ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o', markersize=4)

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

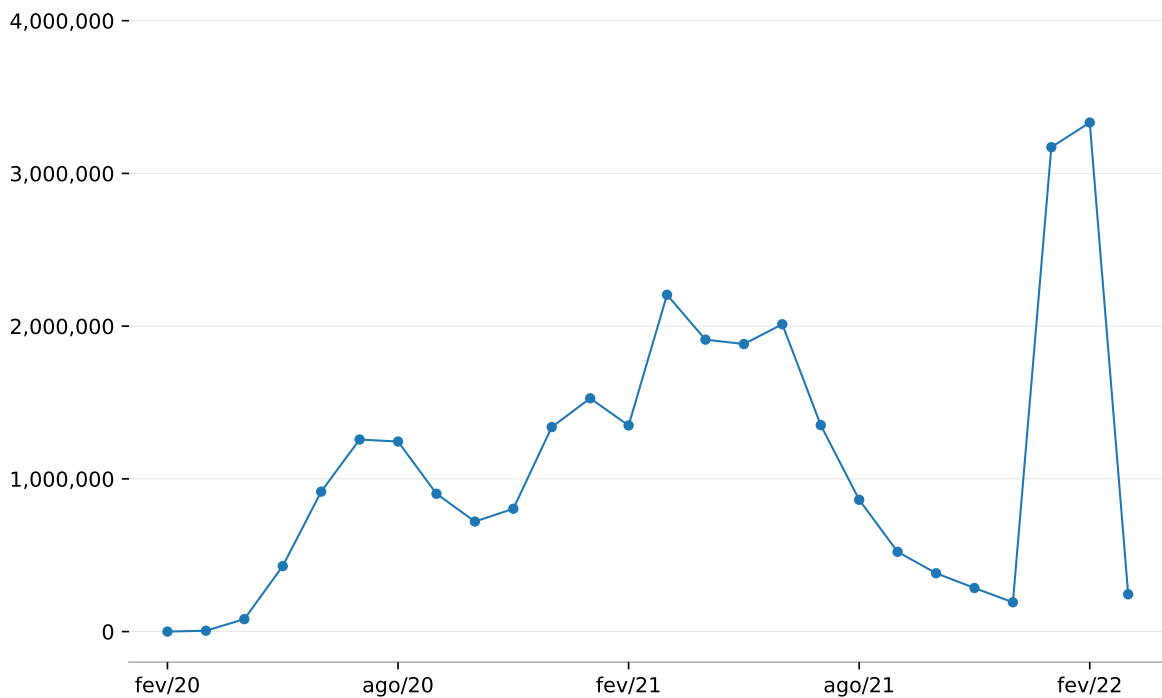
# Linhas de grid
ax.grid(visible=True, which='major', axis='y', ls='-', lw=0.5, c='k', alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

# Limites dos eixos
ax.set_xlim(-1, 26)
ax.set_ylim(-200000, 4300000)

# Mostrar o resultado
plt.show()

```



- Adicionar uma linha de referência que representa a média do número de casos ao longo do tempo.

```
height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Tipo de gráfico a ser plotado
ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o', markersize=4)

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major', axis='y', ls='--', lw=0.5, c='k', alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
```

```

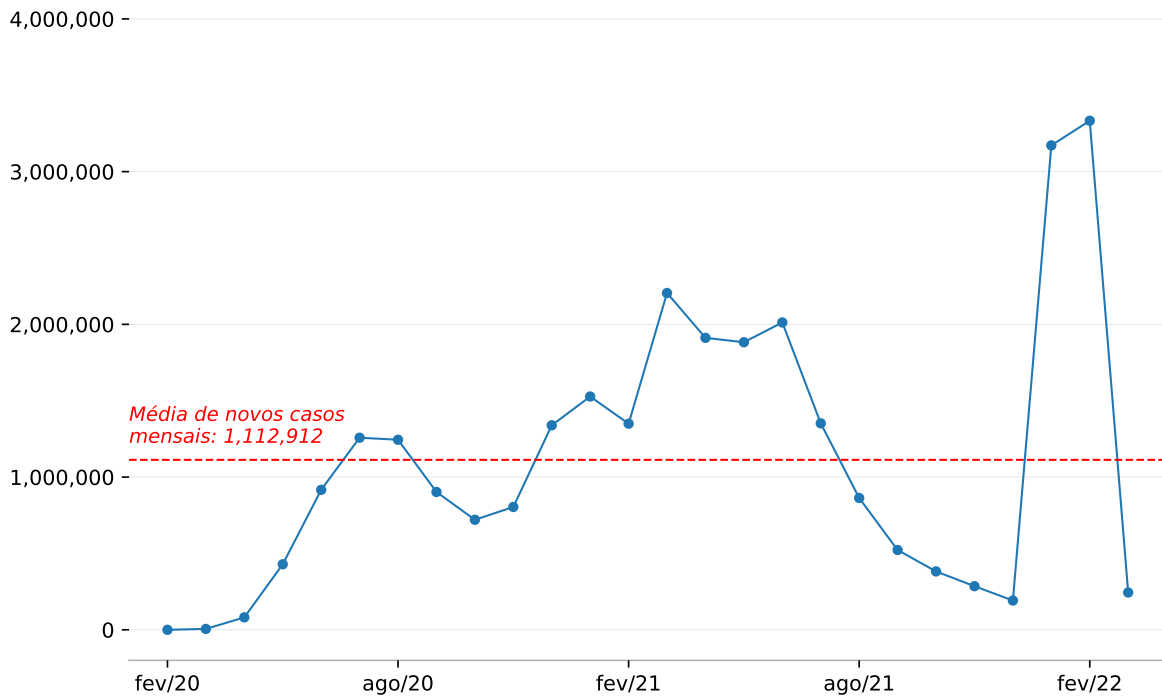
ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

# Limites dos eixos
ax.set_xlim(-1, 26)
ax.set_ylim(-200000, 4300000)

# Linha de referência
ax.plot([-1, 26], [y.mean(), y.mean()], 'r--', lw=1.0)
ax.annotate('Média de novos casos\nmensais: {:.0f}'.format(y.mean()),xy=(-1, 1.10*y.mean()))

# Mostrar o resultado
plt.show()

```



- Deixar o título maior e um pouco afastado do início da figura, juntamente com a fonte dos dados logo abaixo.
- Salvar a figura em formato .png

```

height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Tipo de gráfico a ser plotado
ax.plot(x, y, color='#1f77b4', linewidth=1, marker='o', markersize=4)

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

# Limites dos eixos
ax.set_xlim(-1, 26)
ax.set_ylim(-200000, 4300000)

# Linha de referência
ax.plot([-1, 26], [y.mean(), y.mean()], 'r--', lw=1.0)
ax.annotate('Média de novos casos\nmensais: {:.0f}'.format(y.mean()),xy=(-1, 1.10*y.mean()))

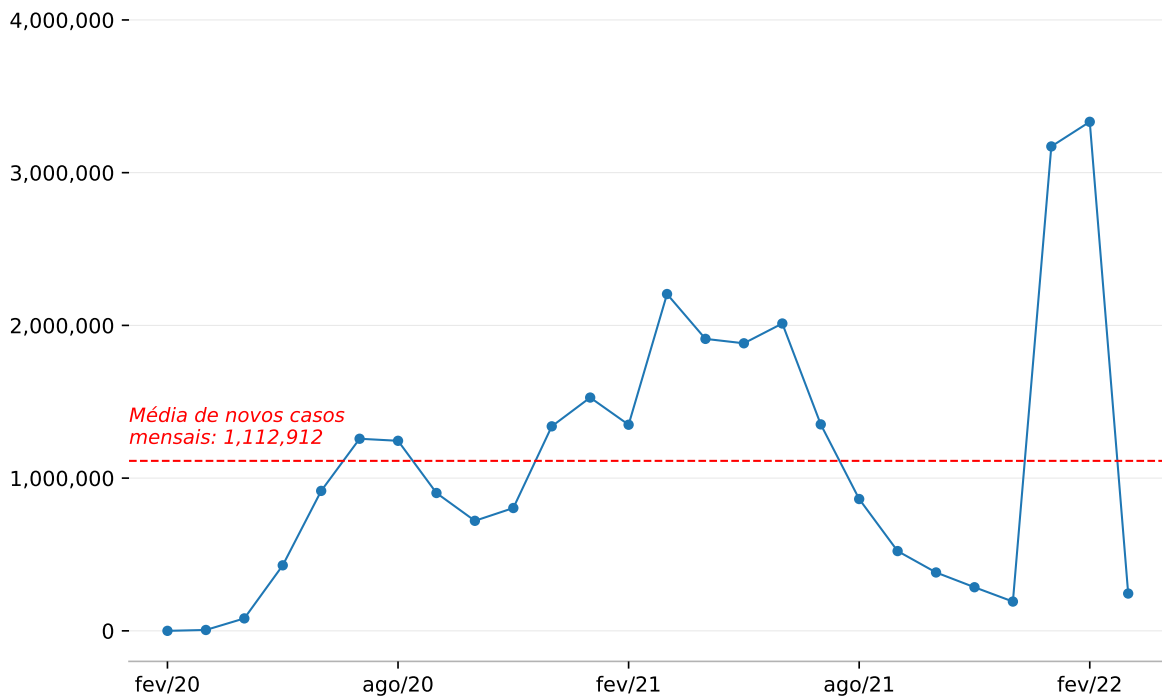
# Título e fonte
plt.suptitle('Número de novos casos mensais de COVID19 - Brasil',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('casos_covid_br.png', bbox_inches='tight')
plt.show()

```

Número de novos casos mensais de COVID19 - Brasil

Fonte: Our World in Data



Uau, que diferença! Foram muitas linhas de código, mas esse é o preço de um bom gráfico e de uma boa visualização. O controle sobre todos os elementos que compõe uma figura é essencial para que possamos cumprir com o objetivo de tornar visualizações cada vez mais informativas e democráticas.

9.5 Gráficos múltiplos

Uma vez aprendida a forma de construção de um gráfico através da função `subplot()` é fácil construir figuras que contenham várias séries em um mesmo plot ou vários plots na mesma figura. Vamos começar pelo caso mais simples.

9.5.1 Várias séries em um mesmo *plot*

Uma vez em posse dos dados de casos de covid-19 por país, uma pergunta que surge naturalmente é: como o número de casos reportados diferiu entre os diferentes países? Um aluno bem atento teria notado que no último gráfico da sessão anterior já havíamos juntado duas séries em um

mesmo plot, sendo uma dessas séries a reta horizontal que representa a média de novos casos. Mas então basta adicionar mais uma linha de código com um segundo `ax.plot()`? É isso aí!

Para essa análise vamos primeiro criar a coluna `new_cases_per_million`, já que comparar o número absoluto de casos entre países com populações muito distintas pode não ser a melhor saída.

```
df_covid_mensal['new_cases_per_million'] = (df_covid_mensal['new_cases'] / df_covid_mensal['population_millions'])
df_covid_mensal['total_cases_per_million'] = (df_covid_mensal['total_cases'] / df_covid_mensal['population_millions'])
df_covid_mensal['total_deaths_per_million'] = (df_covid_mensal['total_deaths'] / df_covid_mensal['population_millions'])

df_covid_br = df_covid_mensal[df_covid_mensal['location'].isin(['Brazil'])].reset_index(drop=True)
df_covid_kr = df_covid_mensal[df_covid_mensal['location'].isin(['South Korea'])].reset_index(drop=True)

x = df_covid_br['ano_mes'][-26:]
y1 = df_covid_br['new_cases_per_million'][-26:]
y2 = df_covid_kr['new_cases_per_million'][-26:]
```

```
x_ticks = [0,6,12,18,24]
x_labels = ['fev/20','ago/20','fev/21','ago/21','fev/22']
y_ticks = [0,10000,20000,30000,40000,50000]
y_labels = ['0','10,000','20,000','30,000','40,000','50,000']
```

```
height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Séries a serem plotadas
ax.plot(x, y1, color='#1f77b4',linewidth=2,marker='o',markersize=4)
ax.plot(x, y2, color='#d62728',linewidth=2,marker='s',markersize=4)

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
```

```

ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

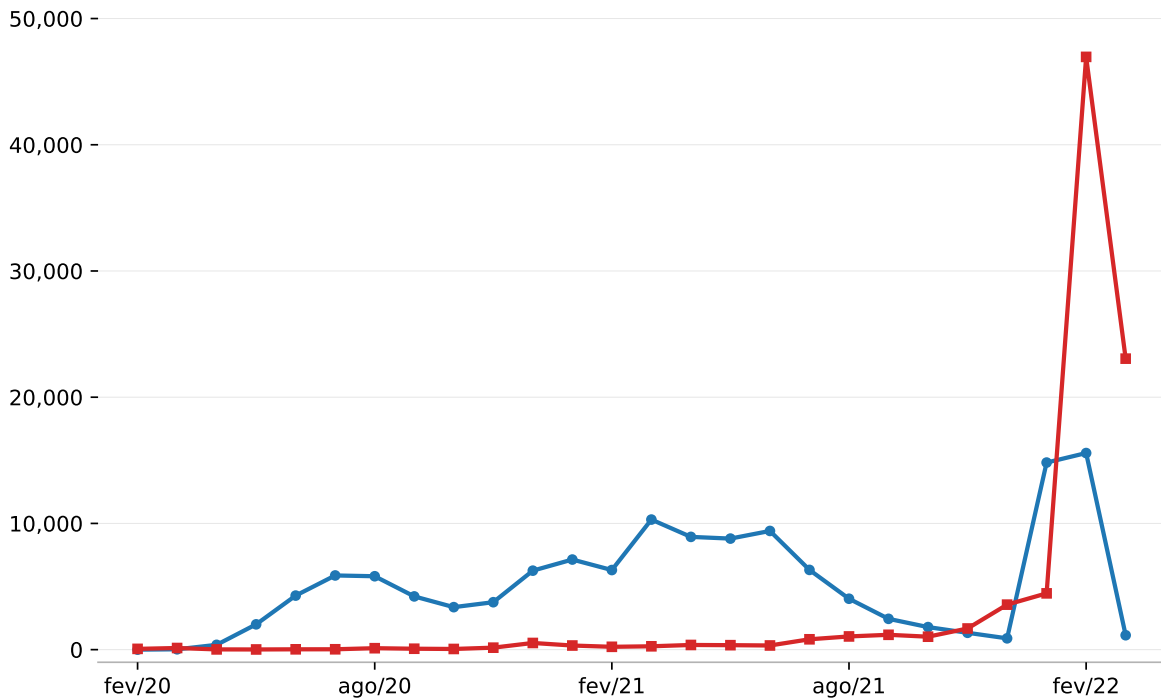
# Limites dos eixos
ax.set_xlim(-1, 26)
ax.set_ylim(-1000, 55000)

# Título e fonte
plt.suptitle('Novos casos por 100 habitantes - Brasil e Coréia do Sul',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('cm_covid_br.png', bbox_inches='tight')
plt.show()

```

Novos casos por 100 habitantes - Brasil e Coréia do Sul
Fonte: Our World in Data



Super direto! Mas ainda falta a legenda. Para isso basta adicionar mais uma única linha de

código.

```
height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Séries a serem plotadas
ax.plot(x, y1, color='#1f77b4',linewidth=2,marker='o',markersize=4,label='Brasil')
ax.plot(x, y2, color='#d62728',linewidth=2,marker='s',markersize=4,label='Coréia do Sul')

# Legenda
ax.legend(loc='center',bbox_to_anchor=(0.3,0.8),framealpha=0,ncol=2,prop={'size': 12})

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

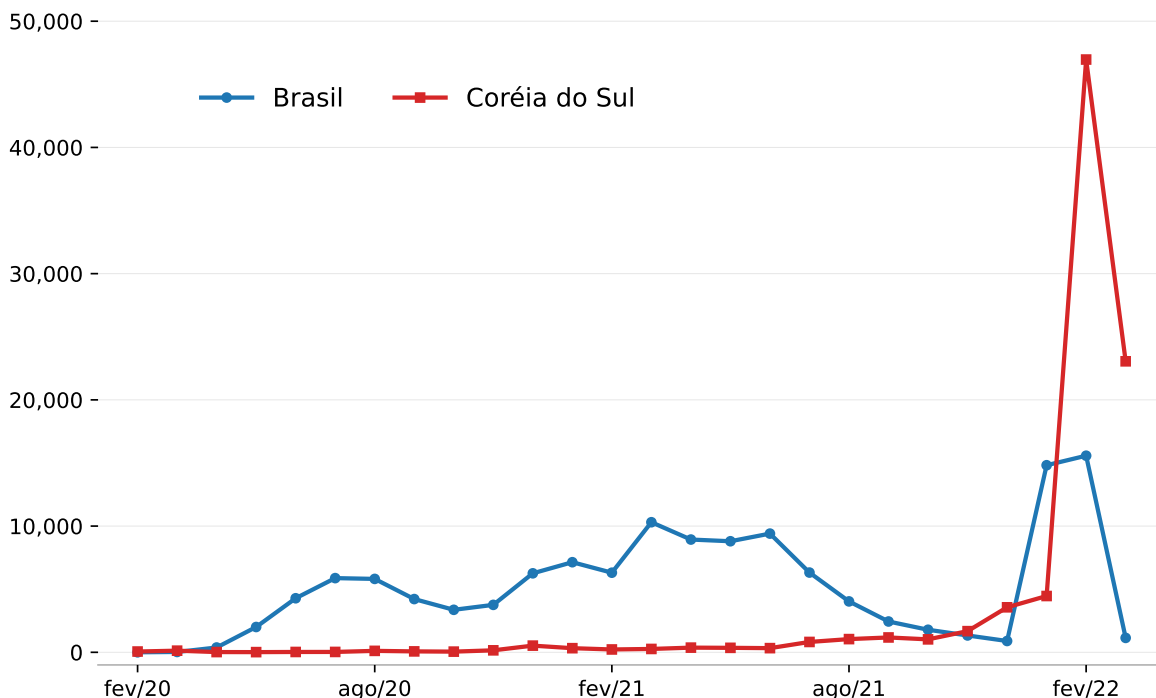
# Limites dos eixos
ax.set_xlim(-1, 26)
ax.set_ylim(-1000, 55000)

# Título e fonte
plt.suptitle('Novos casos por 1,000,000 de habitantes',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('casos_covid_brkr.png', bbox_inches='tight')
plt.show()
```

Novos casos por 1,000,000 de habitantes

Fonte: Our World in Data



Perfeito!

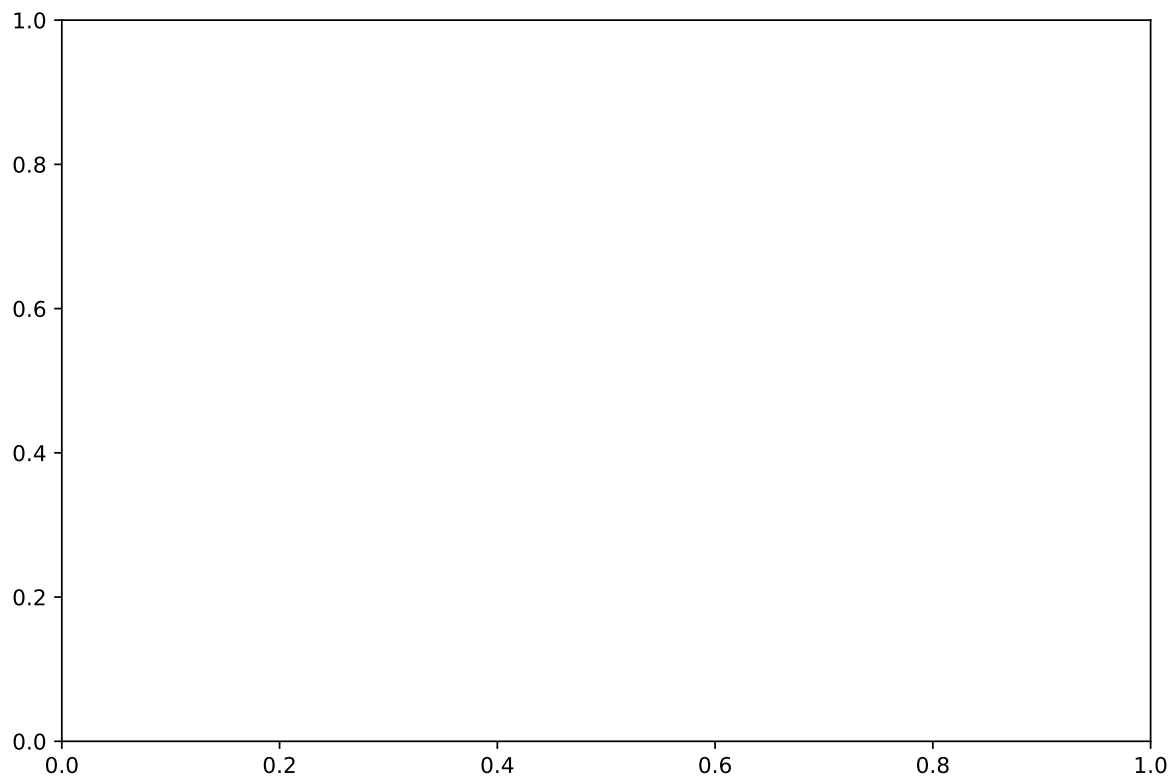
Vamos tentar agora vários plots dentro da mesma figura?

9.5.2 Vários *plots* dentro da mesma figura

Uma outra alternativa ao gráfico acima seria colocar cada série em um plot separado. Para tal, basta alterar os dois primeiros argumentos da função `subplots()`, que representam o número de linhas e o número de colunas da figura final. Nesse caso, porém, o objeto `ax` deixará de ser um simples objeto do matplotlib e se torna um `numpy.ndarray` com vários objetos do matplotlib. Aqui funciona a mesma lógica de indexação de sempre.

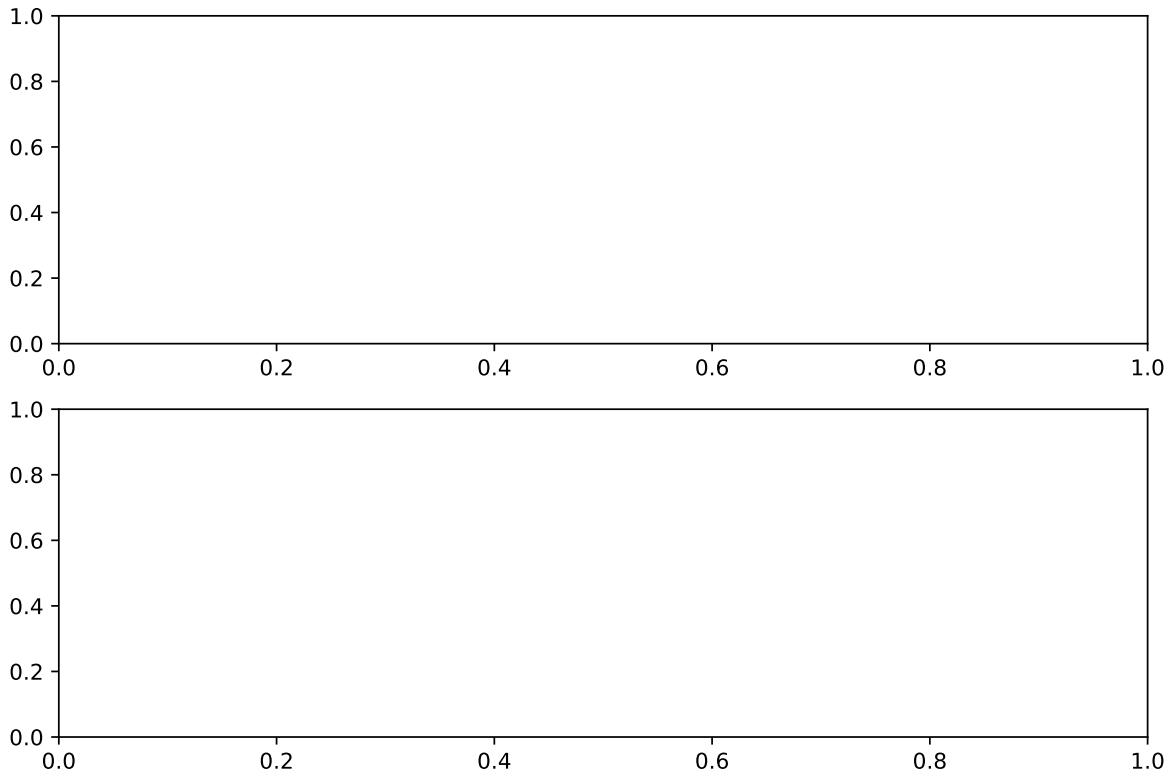
```
num_rows = 1
num_cols = 1
height = 6
fig, ax = plt.subplots(num_rows, num_cols, figsize=(1.50*height, height))
type(ax)
```

matplotlib.axes._axes.Axes



```
num_rows = 2
num_cols = 1
height = 6
fig, ax = plt.subplots(num_rows, num_cols, figsize=(1.50*height, height))
type(ax)
```

numpy.ndarray



Teremos que usar, portanto, algum tipo de loop justamente para poder lidar com as personalizações de cada plot de forma individual.

```
num_rows = 2
num_cols = 1
height = 6
fig, ax = plt.subplots(num_rows, num_cols, figsize=(1.50*height, height))

# Séries a serem plotadas
ax[0].plot(x, y1, color='#1f77b4', linewidth=2, marker='o', markersize=4, label='Brasil')
ax[1].plot(x, y2, color='#d62728', linewidth=2, marker='s', markersize=4, label='Coréia do Sul')

for i in range(0, 2):
    # Legenda
    ax[i].legend(loc='upper left', bbox_to_anchor=(0.05, 0.9), framealpha=0, ncol=2, prop={'size'

    # Visibilidade das bordas
    ax[i].spines['top'].set_visible(False)
    ax[i].spines['right'].set_visible(False)
    ax[i].spines['left'].set_visible(False)
```

```

ax[i].spines['bottom'].set_visible(True)
ax[i].spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax[i].grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Marcadores dos eixos
ax[i].set_xticks(x_ticks)
ax[i].set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax[i].set_yticks(y_ticks)
ax[i].set_yticklabels(y_labels, fontsize=10, fontweight='light')

# Limites dos eixos
ax[i].set_xlim(-1, 26)
ax[i].set_ylim(-1000, 55000)

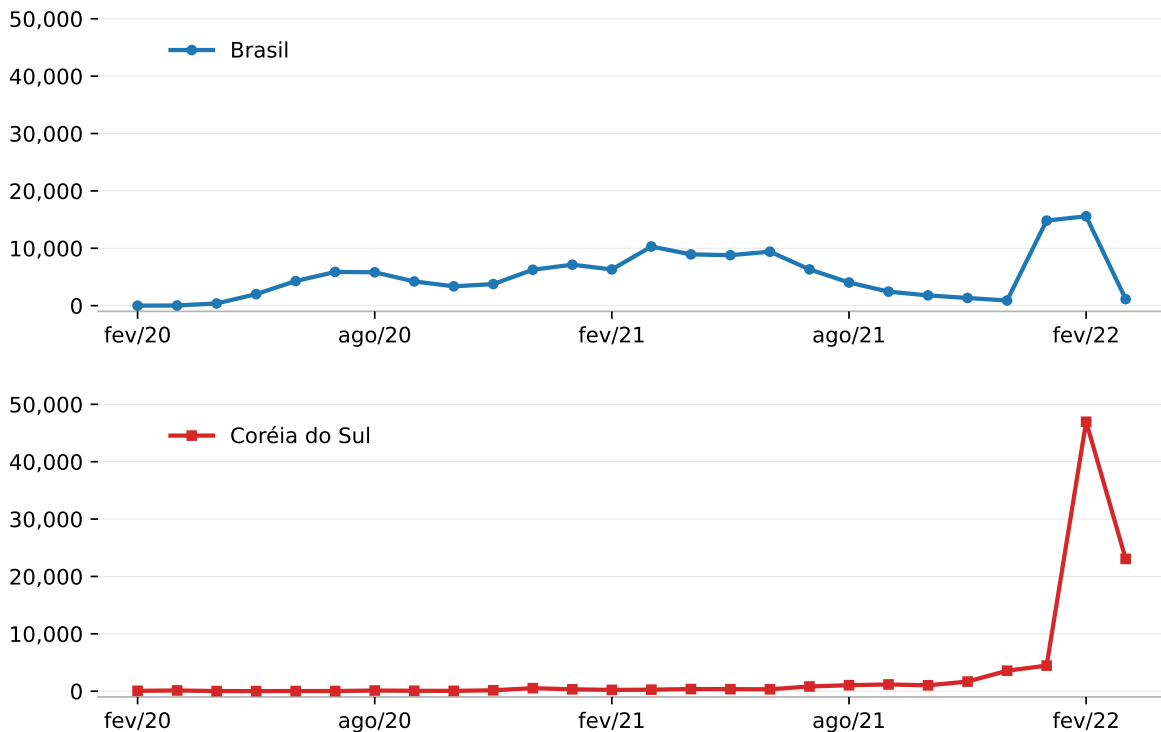
# Título e fonte
plt.suptitle('Novos casos por 100 habitantes',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=195)

# Salvar a figura e mostrar o resultado
plt.savefig('casos_covid_brkr2.png', bbox_inches='tight')
plt.show()

```

Novos casos por 100 habitantes

Fonte: Our World in Data



Como sempre, o céu é o limite!

9.6 Outros tipos de gráficos

9.6.1 Gráfico de dispersão: *scatter*

No entanto, gráficos de linhas não são os mais indicados quando estamos interessados em avaliar a relação entre duas variáveis e não de uma única variável ao longo do tempo, por exemplo. Para isso temos o **gráfico de dispersão** (*scatterplot*, em inglês), que nada mais é do que uma forma de representar pares ordenados de pontos ao longo de dois eixos.

Vamos construir um exemplo de scatterplot utilizando as novas variáveis `total_cases_per_million` e `total_deaths_per_million`. É de se esperar que um número maior de casos por habitantes esteja relacionado diretamente com um número maior de mortes por habitantes, o que implicaria em uma relação positiva entre essas duas variáveis. Como ficaria a representação gráfica dessa relação para a média anual por país?

```

df_covid_mensal['ano'] = df_covid_mensal['ano_mes'].str[0:4]
df_covid_anual = df_covid_mensal[['location','ano','total_cases_per_million','total_deaths_p

df_covid_anual = df_covid_anual.groupby(by=['location','ano'], as_index=False).mean()
x = df_covid_anual['total_cases_per_million']
y = df_covid_anual['total_deaths_per_million']

x_ticks = [0,100000,200000,300000]
x_labels = ['0','100,000','200,000','300,000']
y_ticks = [0,1000,2000,3000]
y_labels = ['0','1,000','2,000','3,000']

```

```

height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Séries a serem plotadas
ax.scatter(x,y,color='#1f77b4',marker='o',s=50)

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Marcadores dos eixos
ax.set_xticks(x_ticks)
ax.set_xticklabels(x_labels, fontsize=10, fontweight='light')
ax.set_yticks(y_ticks)
ax.set_yticklabels(y_labels, fontsize=10, fontweight='light')

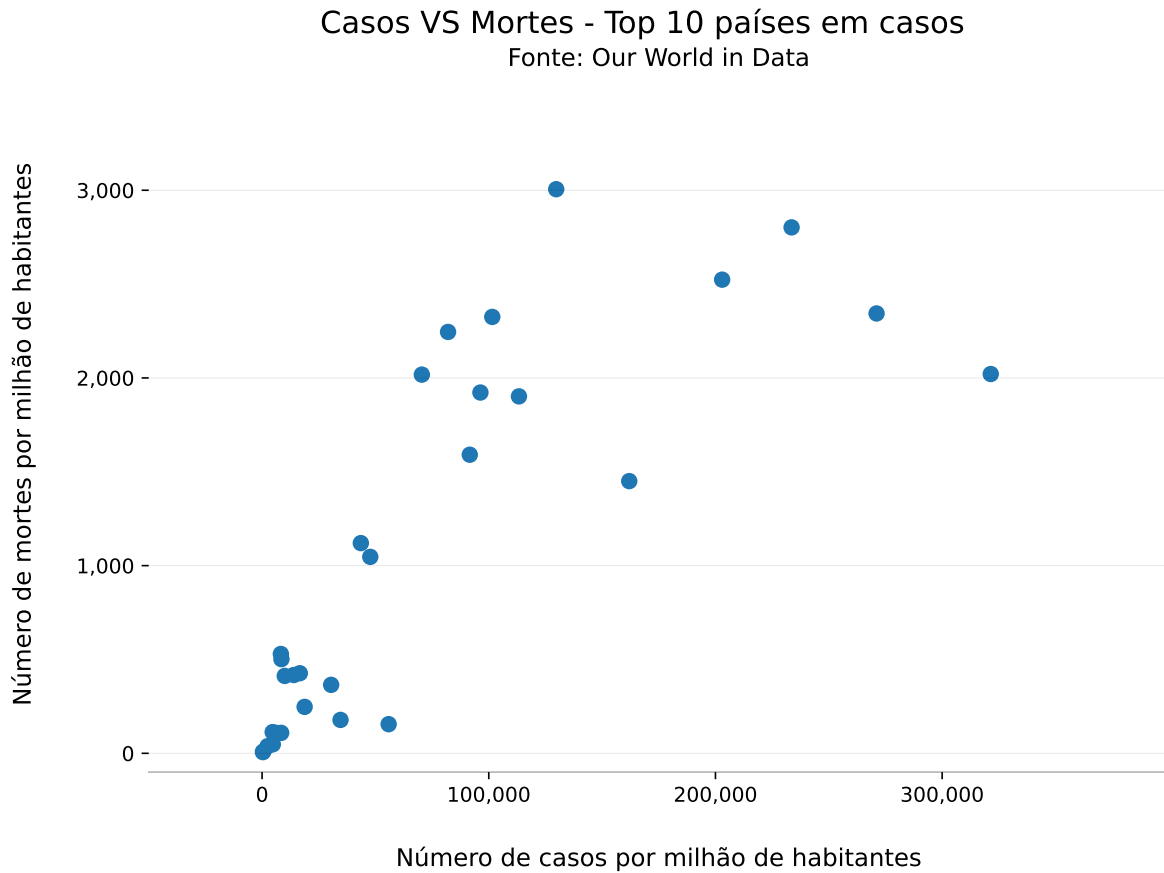
# Limites e título dos eixos
ax.set_xlim(-50000, 400000)
ax.set_ylim(-100, 3500)
ax.set_xlabel('Número de casos por milhão de habitantes',fontsize=12, fontweight='light', la
ax.set_ylabel('Número de mortes por milhão de habitantes',fontsize=12, fontweight='light', la

# Título e fonte
plt.suptitle('Casos VS Mortes - Top 10 países em casos',fontsize=15,fontweight='normal')

```

```
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('casos_mortes_scatter.png', bbox_inches='tight')
plt.show()
```



Parece que tem uma relação positiva de fato. Para bater o martelo sobre a relação para todos os países, no entanto, seria necessário ir atrás de mais dados. Que tal refazer a análise buscando a amostra completa de países e não apenas os 10 países com o maior número absoluto de casos?

9.6.2 Gráfico de barra: *bar* e *barh*

Um outro tipo de gráfico bastante interessante é o gráfico de barra, que serve muito ao propósito, por exemplo, de mostrar diferenças em uma variável de interesse, para um dado instante do

tempo, entre regiões ou grupos diferentes. Dentre os 10 países com o maior número de casos, quais países tem a maior proporção da população com 65 anos ou mais?

Aqui vamos usar apenas as colunas `iso_code` e `aged_65_older` para ordenar os países de acordo com a proporção de idosos em sua população. Como já vimos, os valores de `aged_65_older` são constantes ao longo do tempo no nível país. Podemos então partir da criação de um dataframe que contenha apenas os dados referentes ao último mês de dado e ficar com uma única observação por país.

```
df_covid_top10 = df_covid_mensal[df_covid_mensal['ano_mes']=='2022-03']
df_covid_top10 = df_covid_top10[['location','aged_65_older']].reset_index(drop=True)
df_covid_top10.sort_values(['aged_65_older'],ascending=False,ignore_index=True,inplace=True)

df_covid_top10
```

	location	aged_65_older
0	Japan	27.049
1	Italy	23.021
2	Germany	21.453
3	France	19.718
4	United Kingdom	18.517
5	United States	15.413
6	Russia	14.178
7	South Korea	13.914
8	Brazil	8.552
9	India	5.989

```
# Para não dar problema com os labels do eixo x serem muito grandes, vamos substituir aqueles
df_covid_top10.loc[4,'location'] = 'United\nKingdom'
df_covid_top10.loc[5,'location'] = 'United\nStates'
df_covid_top10.loc[7,'location'] = 'South\nKorea'
```

À partir desses dados, podemos construir 2 tipos de gráfico de barras:

- Barra vertical

```
x = df_covid_top10['location']
y = df_covid_top10['aged_65_older']

height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))
```

```

# Séries a serem plotadas
ax.bar(x,y,align='center',color='#1f77b4')

# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

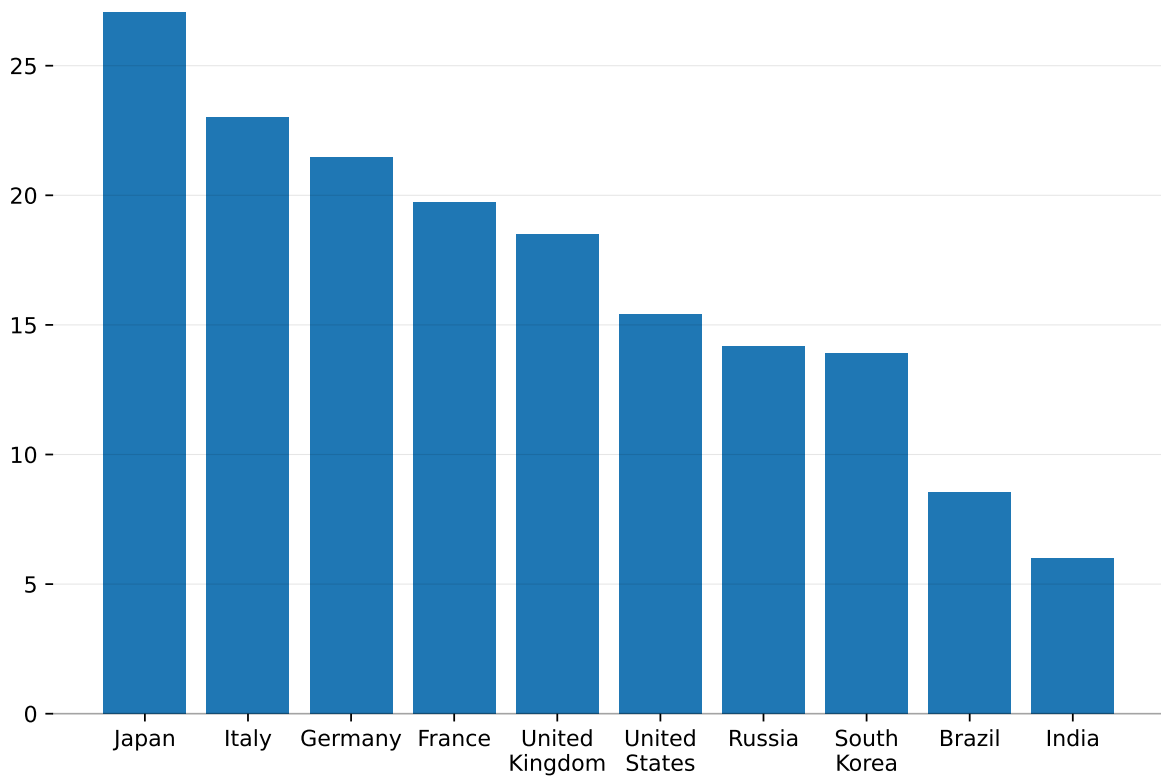
# Linhas de grid
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Título e fonte
plt.suptitle('Fração da população 65+ - Top 10 países em casos',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('frac65plus_bar.png', bbox_inches='tight')
plt.show()

```

Fração da população 65+ - Top 10 países em casos
Fonte: Our World in Data



- Barra horizontal

```
height = 6
fig, ax = plt.subplots(1,1, figsize=(1.50*height, height))

# Séries a serem plotadas
ax.barh(x,y,align='center',color='#1f77b4')

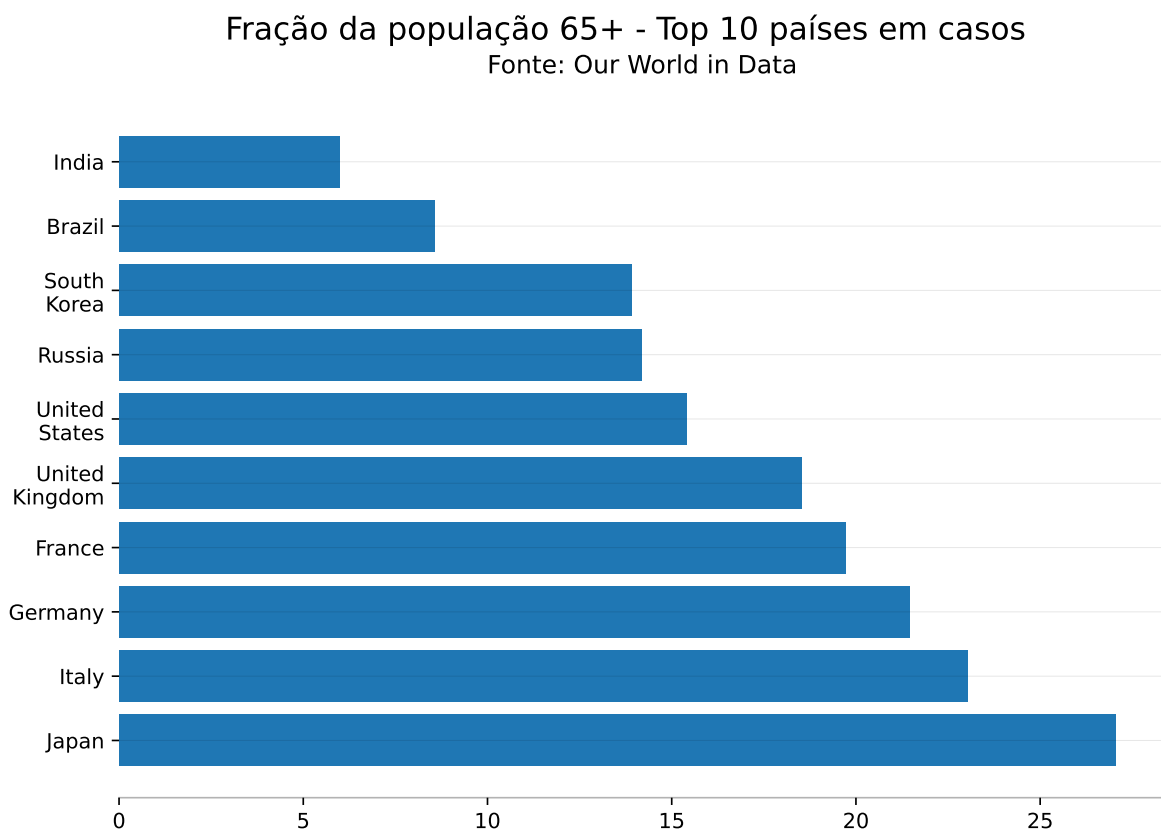
# Visibilidade das bordas
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['bottom'].set_alpha(0.3)

# Linhas de grid
```

```
ax.grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

# Título e fonte
plt.suptitle('Fração da população 65+ - Top 10 países em casos',fontsize=15,fontweight='normal')
plt.title('Fonte: Our World in Data',fontsize=12,fontweight='normal',pad=15)

# Salvar a figura e mostrar o resultado
plt.savefig('frac65plus_barh.png', bbox_inches='tight')
plt.show()
```



9.6.3 Histograma: *hist*

O Matplotlib também nos oferece um conjunto bastante grande de gráficos estatísticos, como por exemplo o *boxplot* e o *histograma*. Vamos utilizar esse último, através da função `hist`, para plotar em uma mesma figura histogramas de 4 variáveis diferentes: `median_age`, `aged_65_older`, `cardiovasc_death_rate` e `diabetes_prevalence`.

```
df_covid_top10 = df_covid_mensal[df_covid_mensal['ano_mes']=='2022-03']

df_covid_top10 = df_covid_top10[['location',
                                  'median_age',
                                  'aged_65_older',
                                  'cardiovasc_death_rate',
                                  'diabetes_prevalence']].reset_index(drop=True)

df_covid_top10
```

	location	median_age	aged_65_older	cardiovasc_death_rate	diabetes_prevalence
0	Brazil	33.5	8.552	177.961	8.11
1	France	42.0	19.718	86.060	4.77
2	Germany	46.6	21.453	156.139	8.31
3	India	28.2	5.989	282.280	10.39
4	Italy	47.9	23.021	113.151	4.78
5	Japan	48.2	27.049	79.370	5.72
6	Russia	39.6	14.178	431.297	6.18
7	South Korea	43.4	13.914	85.998	6.80
8	United Kingdom	40.8	18.517	122.137	4.28
9	United States	38.3	15.413	151.089	10.79

```
num_rows = 2
num_cols = 2
height = 6
fig, ax = plt.subplots(num_rows,num_cols, figsize=(1.50*height, height))

# Séries a serem plotadas
ax[0,0].hist(df_covid_top10['median_age'],color='#1f77b4',bins=10)
ax[0,1].hist(df_covid_top10['aged_65_older'],color='#1f77b4',bins=10)
ax[1,0].hist(df_covid_top10['cardiovasc_death_rate'],color='#1f77b4',bins=10)
ax[1,1].hist(df_covid_top10['diabetes_prevalence'],color='#1f77b4',bins=10)

ax[0,0].set_title('Idade mediana', fontsize=8, fontweight='light')
ax[0,1].set_title('Fração pop. 65+', fontsize=8, fontweight='light')
ax[1,0].set_title('Taxa de mort. cardiovascular', fontsize=8, fontweight='light')
ax[1,1].set_title('Prevalência de diabetes', fontsize=8, fontweight='light')

for r in range(0,2):
```

```

for c in range(0,2):
    # Visibilidade das bordas
    ax[r,c].spines['top'].set_visible(False)
    ax[r,c].spines['right'].set_visible(False)
    ax[r,c].spines['left'].set_visible(False)
    ax[r,c].spines['bottom'].set_visible(True)
    ax[r,c].spines['bottom'].set_alpha(0.3)

    # Linhas de grid
    ax[r,c].grid(visible=True, which='major',axis='y', ls='-',lw=0.5,c='k',alpha=0.1)

    # Opções do eixo y
    ax[r,c].set_yticks([0,1,2,3,4])
    ax[r,c].set_yticklabels(['0','1','2','3','4'], fontsize=10, fontweight='light')
    ax[r,c].set_ylim(0,4.5)

# Ajuste de espaço entre os vários plots
fig.subplots_adjust(hspace = 0.6, wspace = 0.3)

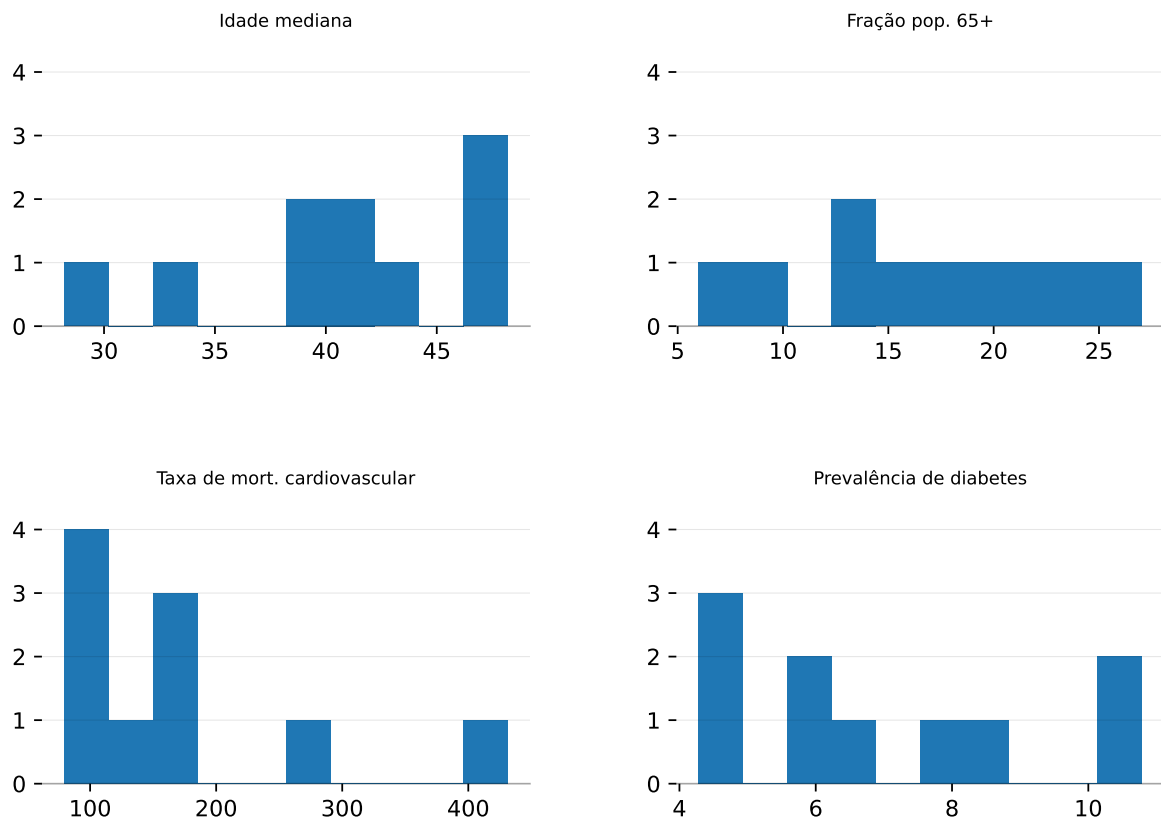
# Título e fonte
plt.suptitle('Características dos países com o maior número de casos de covid19',y=1.05,fontweight='normal')
plt.text(0, 13.3,'Fonte: Our World in Data',fontsize=12,fontweight='normal')

# Salvar a figura e mostrar o resultado
plt.savefig('carac_covid_all.png', bbox_inches='tight')
plt.show()

```

Características dos países com o maior número de casos de covid19

Fonte: Our World in Data



9.7 Seaborn

9.8 Plotly

9.9 Visualização e *storytelling*

É parte importante do dia-a-dia de um economista - e também de grande parte dos profissionais pertencentes ao grupo das ciências sociais aplicadas - ser capaz de comunicar os resultados de uma análise ou pesquisa. E como há muito tempo já se sabe, para comunicar e informar, **uma imagem vale mais do que mil palavras**.

No artigo *An Economist's Guide to Visualizing Data*, publicado no prestigiado *Journal of Economic Perspectives* em 2014, o autor *Jonathan Schwabish* fala sobre a importância de construir visualizações e gráficos que de fato chamem a atenção para aquilo que se deseja comunicar. Ele cita três princípios básicos na construção de uma boa visualização gráfica:

1. **Mostre os dados.** O conjunto de dados é o elemento básico por trás da história que se quer contar. Isso não quer dizer que é preciso mostrar todos os dados disponíveis, alguns gráficos mostram demais e acabam sendo pouco informativos. Mostre os dados, mas o faça da forma mais clara possível.
2. **Reduza a sujeira e a desordem gráfica.** O uso de elementos visuais desnecessários ou que distraem, tenderá a reduzir a eficácia do gráfico em comunicar uma mensagem. Pense duas vezes antes de utilizar bordas, linhas de referências muito grossas e escuras, pontos de referência desnecessários nos eixos, cores muito próximas em conjuntos de dados distintos, texturas, etc.
3. **Elementos visuais e textuais devem funcionar de forma integrada.** O gráfico não pode ser poluído de texto e se transformar naquilo que não é, mas elementos textuais podem ajudar bastante na tarefa de chamar a atenção para aquele que é o ponto principal da mensagem. O gráfico deve funcionar como um complemento ao texto, mas ao mesmo tempo deve se sustentar sem ele.

O artigo vai muito além desses 3 princípios básicos e mostra exemplos de visualizações ruins e de como é possível “consertá-las”. Há, no entanto, outras várias fontes de informação sobre visualização (esse [link](#) é uma ótima referência) e de como construir gráficos e visualizações a partir de uma ideia de *storytelling*, de utilizar dados e ferramentas gráficas para contar uma história e comunicar uma mensagem (o livro *Storytelling com dados* é outra ótima referência). Em resumo, construir um bom gráfico, uma visualização eficiente, é um trabalho de criatividade e de muita tentativa e erro. Uma determinada visualização pode funcionar muito bem para um certo tipo de dado e não funcionar para outros vários tipos. **Personalização** é a palavra chave nessa árdua tarefa de construir um bom gráfico.

9.10 Exercícios

1. X
2. X

10 Análise empírica integrada

10.1 Introdução

10.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- os
- pandas
- matplotlib
- seaborn
- plotly

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sbs
import plotly as ply
```

10.2 O que é o NumPy?

NumPy (Numerical Python) é o pacote fundamental

10.3 Exercícios

1. X
2. X

Parte III

Temas complementares

Após o desenvolvimento dos fundamentos da lógica de programação e a aplicação prática dessas ideias em análise de dados com Python, a parte final do curso tem como objetivo ampliar o repertório conceitual do aluno e apresentar outros paradigmas e ferramentas amplamente utilizados em análise empírica.

Esta etapa não tem como objetivo aprofundar tecnicamente todos os temas abordados, mas sim introduzir conceitos-chave que ajudam o aluno a compreender diferentes estilos de programação e a transitar entre linguagens e ecossistemas distintos. Em particular, são abordados o paradigma de programação orientada a objetos e uma breve introdução à linguagem R, muito utilizada em economia e a linguagem a ser utilizada nos curso de Econometria do Bacharelado em Ciências Econômicas da FEA-USP.

Listo abaixo os principais conceitos trabalhados nos capítulos que compõem esta parte final do curso:

- **Capítulo 11 – Introdução à programação orientada a objetos (OOP).** Apresenta os princípios básicos da programação orientada a objetos, com foco em atributos e classes. O objetivo é introduzir uma forma alternativa de estruturar programas, destacando quando e por que esse paradigma é utilizado, bem como suas vantagens em projetos mais complexos.
- **Capítulo 12 – Introdução ao R.** Introduz a linguagem R, discutindo sua sintaxe básica, principais tipos de objetos e semelhanças e diferenças em relação ao Python. O capítulo apresenta também o ecossistema do **tidyverse**, incluindo ferramentas para manipulação e visualização de dados, e enfatiza o uso de pipelines como forma de organizar análises empíricas.

Juntos, esses capítulos finais têm como objetivo expandir a visão do estudante sobre programação e análise de dados, mostrando que diferentes linguagens e paradigmas podem ser utilizados para resolver problemas semelhantes. Ao final do curso, espera-se que o aluno tenha desenvolvido não apenas familiaridade com Python, mas também a capacidade de reconhecer padrões comuns entre linguagens, facilitando o aprendizado autônomo de novas ferramentas ao longo de sua formação acadêmica e profissional.

11 Introdução à programação orientada a objetos (OOP)

11.1 Introdução

11.1.1 Pré-requisitos

Para trabalhar com funções e todas as suas funcionalidades, utilizaremos dois pacotes externos:

- os
- pandas
- matplotlib
- seaborn
- plotly

```
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sbs
import plotly as ply
```

11.2 Exercícios

1. X
2. X

12 Introdução ao R

12.1 Introdução

12.1.1 RStudio

Assim como no Python, programar em R depende de um ambiente que pode ser configurado de várias formas. Você pode trabalhar com o R em sua forma pura, mas usualmente utilizamos alguma IDE que facilite o processo. E no mundo das IDEs que suportam o R, o **RStudio** talvez seja a principal. Para instalar o **RStudio** é preciso primeiro instalar o R. A partir daí, basta baixar o programa para o seu sistema operacional e seguir as configurações padrão do instalador para que o **RStudio** se torne o programa padrão para arquivos vinculados à linguagem.

Em termos de layout, o **RStudio** se parece muito com o **Spyder**: um ambiente integrado que nos permite escrever o código e acompanhar de maneira simultânea os objetos criados e o resultado daquilo que rodamos.

12.1.2 Pacotes de terceiros

Bla bla bla

12.1.3 Pré-requisitos

Utilizaremos muito o conjunto de funcionalidades que vem com o ambiente tidyverse, que não é nativo do R, e inclui todo um conjunto de pacotes que facilitam a vida daqueles que trabalham com análise e gestão de dados.

- tidyverse
- summarytools
- data.table
- ggplot2

```
library(tidyverse)
library(summarytools)
library(data.table)
library(ggplot2)
```

12.2 Primeiros passos

12.2.1 Diferenças de sintaxe em relação ao Python

12.2.2 R como calculadora

12.3 Objetos

12.3.1 Vetores e matrizes

12.3.2 Dataframes

12.4 Iteração

12.5 Funções

12.6 Gestão e análise de dados

12.6.1 Manipulação e pipelines

12.6.2 Visualização

12.7 Exercícios

1. X

2. X

Bibliografia